rai-bvar: Composable Bayesian VARs with Stochastic Volatility, Validated Samplers and a Fair Out-of-Sample Verdict Engine in Python

Yigit Okar

[**TODO:** affiliation and contact address]

Draft of October 23, 2026

Abstract

A density-forecast gain credited to a richer mean or prior in a Bayesian vector autoregression (BVAR) is interpretable only when the volatility specification is held fixed. We present rai-bvar, an Apache-2.0 Python package built around this position. A BVAR is a composition of a mean, a volatility and a prior component, and the composition selects the estimation engine. Conjugate Minnesota models, with dummy-observation priors, the Giannone–Lenza–Primiceri hierarchy and Lenza–Primiceri COVID-19 scaling, go to an analytic engine. Compositions with stochastic volatility, outliers or Student-t errors run on hand-written JAX Gibbs samplers in recursive structural or corrected reduced form. Both engines return one predictive contract, joint path draws with per-draw conditional Gaussian moments, so log predictive densities are Rao–Blackwellised. For MCMC forecasts, an opt-in estimator integrates the intermediate outcomes out at multi-step horizons. A verdict engine calls a model better only if it differs from its reference along exactly one axis, its log-score gain exceeds three Monte Carlo standard errors, a valid test or the model confidence set separates the two, and both models converged at 95% or more of the forecast origins. The samplers are checked by exact identities, Geweke tests, simulation-based calibration, analytic twins and parity with independent implementations. Eighteen injected sampler bugs measure the power of the first three: in each of 37 mutant-by-tier checks the bug is caught and the unmodified sampler passes. A replication of Clark and Ravazzolo (2015) on FRED-QD illustrates the workflow. A first full run missed two of six acceptance criteria; the remedies, among them a larger MCMC budget and the opt-in estimator, were pre-registered before the acceptance run. Acceptance-run outcome by criterion: [ACC: `criteria[*].met`].

Keywords: Bayesian vector autoregression; stochastic volatility; Gibbs sampling; density forecasts; sampler validation; reproducibility; Python; JAX.

1 Introduction

rai-bvar is a Python package for Bayesian vector autoregressions (BVARs) built around one question, stated in its specification as

> "Does added model complexity improve out-of-sample density forecasts once time-varying volatility is accounted for?"

The qualification matters. The volatility of U.S. macroeconomic shocks has changed over the post-war period (Cogley and Sargent, 2005; Primiceri, 2005; Sims and Zha, 2006), and allowing

for time-varying volatility has been found to improve BVAR density forecasts (Clark, 2011; Clark and Ravazzolo, 2015; Carriero et al., 2016); Chan (2024) surveys BVARs with stochastic volatility (SV), and Carriero et al. (2015) study how other specification choices affect forecast accuracy. When a model with a richer mean or prior is compared with a baseline that also has a different volatility specification, the measured gain combines the two changes. The package therefore adopts a design position, which is a rule of the software and not an empirical finding: a mean or prior effect is measured with the volatility specification held fixed, so a richer mean or prior under SV is compared with the linear Minnesota model under the same SV, never with a constant-volatility baseline.

The credibility of such a comparison depends on six links. We state each as a requirement on the software, not as a claim about existing tools, and point to where this paper addresses it.

1. *The sampler may target the wrong posterior.* A published equation-by-equation sampler for large BVARs with SV (Carriero et al., 2019) did so, as Bognanni (2022) showed; Carriero et al. (2022) corrected it. Samplers need checks of known power (Section 8).

2. *Every forecast origin is a separate MCMC fit, and each must converge*; a gain computed from unconverged fits should not be certified (Sections 6.8, 8.12 and 11.6).

3. *The log score is itself a Monte Carlo estimate.* At horizons beyond one step its error can be large in the tails while the reported standard error stays small (Section 6.4).

4. *Triangular SV models depend on the variable ordering* (Arias et al., 2023; Chan et al., 2024), so a gain should be reported with its spread across orderings (Sections 6.9 and 11.7).

5. *Tests must be valid for the design.* For nested models under an expanding window, tests of equal predictive accuracy in the tradition of Diebold and Mariano (1995) lack their standard limiting distribution (Clark and McCracken, 2001), and the framework of Giacomini and White (2006) requires a rolling or fixed window. A decision then needs a second route, here the model confidence set (Hansen et al., 2011), and the scores compared must be proper (Gneiting and Raftery, 2007) (Section 6.6).

6. *Analysis choices can be made after the scores are seen*, unless the settings are fixed in code before the scores exist (the pre-registration in Section 11.6).

Two infrastructure requirements follow: one estimation interface and output format for conjugate and non-conjugate models (Sections 4.2 and 6.2), and no look-ahead, so that data-dependent prior settings such as Minnesota scales are recomputed inside every estimation window (Section 6.7).

The package metadata describe rai-bvar as "Ablation-ready Bayesian VARs: composable mean, volatility and prior components with a fair out-of-sample verdict engine." Five design commitments shape it. (i) The composition chooses the engine: conjugate Minnesota models, including the hierarchy of Giannone et al. (2015) and the COVID-19 scaling of Lenza and Primiceri (2022), go to an analytic engine, and compositions with SV, outliers or Student-t errors to hand-written JAX Gibbs samplers; the user never selects the engine. (ii) Both engines return one predictive contract. (iii) One backtest engine and one verdict engine score and compare every model. (iv) Validation ships as library code (`rai_bvar.validation`), not only as tests. (v) The implementation is licensed under Apache-2.0, and GPL-licensed packages were run only as numerical oracles, with the qualification given in Section 2. "Ablation-ready" describes the architecture, in which a new mean, volatility or prior component is evaluated against every existing one through one contract and one verdict rule. The release described here (Phases 0–2 of the roadmap) populates the volatility axis;

the mean axis is linear and the prior axis is the Minnesota family (Section 3). It is version 0.0.1, classified as pre-alpha. [**TODO:** Release commit hash, tag and repository URL (OUTLINE TODO 20).]

Our thesis is that a model comparison is only as credible as its weakest link: the sampler, convergence, the Monte Carlo error of the score, the variable ordering, test validity, or post-hoc choices. rai-bvar makes each link checkable, and this paper shows the evidence for each. The econometrics is not new; the contribution is the integration, the contracts and the evidence of correctness:

- **C1, a factorial composition grammar** (Section 4). `BVAR(mean=...,vol=...,lags=...)` resolves to an engine by fixed rules, rejects invalid compositions at construction, and exposes `spec()` and `axes`; together with the hash `spec_hash(model)` they make "differs along exactly one axis" a checkable property.

- **C2, a populated volatility axis** (Section 3). The release provides constant volatility under inverse-gamma, inverse-Wishart or IW-equivalent (Chan, 2022) priors, and SV with AR(1) or random-walk log-variance, with outliers (Carriero et al., 2024), with Student-t errors (Jacquier et al., 2004), or with both, in two recursive structural forms and the corrected reduced form (Carriero et al., 2022), though not every combination in every form (Section 3.5); dummy-observation priors (Doan et al., 1984; Sims and Zha, 1998; Giannone et al., 2015) are available on both engines.

- **C3, one predictive contract** (Sections 6.2 to 6.4). Path draws with per-draw conditional means and Cholesky factors give a Rao–Blackwellised log score with analytic marginalisation and an effective-sample-size-adjusted Monte Carlo standard error (MCSE), following Geweke and Amisano (2010) and Krüger et al. (2021). The contract's simulation-consistency invariant is tested for every registered analytic model and for 12 MCMC compositions. We diagnose a tail failure of the path-conditioned estimator at $h > 1$ and add, for MCMC forecasts, an opt-in estimator that integrates the intermediate outcomes out.

- **C4, a backtest and verdict engine that encode "volatility first"** (Sections 6.6 to 6.8). Look-ahead-free refits, one-axis pairing, grid harmonisation, a three-MCSE gate, a validity policy (Clark and McCracken, 2001; Giacomini and White, 2006) for Diebold–Mariano tests with the Harvey–Leybourne–Newbold correction (Diebold and Mariano, 1995; Harvey et al., 1997), model-confidence-set separation (Hansen et al., 2011) and a convergence gate (Vehtari et al., 2021) combine into one automatic rule. The combination is the claim; the rule's size and power are not yet studied (Section 12).

- **C5, validation as library code** (Section 8). The validation package (4,230 lines) implements the statistics and the independent reference models of identity checks, block-level checks, Geweke tests and simulation-based calibration (SBC); with the analytic twins, parity tests and mutation suite of the test suite they form seven tiers (Geweke, 2004; Cook et al., 2006; Gelman, 2017; Talts et al., 2018; Modrák et al., 2025; Grosse and Duvenaud, 2014; Gandy and Scott, 2021; Säilynoja et al., 2022). The reference modules import no kernel code, which a test enforces, and Geweke and SBC tests are registered for 22 composition rows and Geweke tests for 7 block configurations. [**TODO:** OUTLINE TODO 3: row 18, `reduced_sv_rw` (reduced form × SV(rw)), and a `reduced-SV(rw)` contract fit were committed in 7f410dc, after the acceptance commit c823d20; the SV(rw, t) variant is still not covered. If the release commit includes 7f410dc, the counts become 23 (Geweke/SBC rows) and 13 (contract fits, C3); refresh both from the release records.] The power of the identity, block-level, Geweke and SBC tiers is measured by mutation testing (DeMillo et al., 1978; Jia and Harman, 2011): 18 injected sampler bugs give 37 mutant-by-tier checks,

all caught with passing controls. On testing probabilistic programming systems, see Dutta et al. (2018).

- **C6, observations on validation methods** (Sections 8 and 9.5). Exact-mode SV is exact for any mixture table, so a deliberately coarse table gives Geweke tests power against errors in the exact correction; the log-likelihood as a test quantity catches the triangular-step error of Carriero et al. (2019), which parameter ranks missed in recorded runs, consistent with Modrák et al. (2025); per-replication SBC effective sample sizes cannot be certified from 99 kept draws; and dummy-observation rows admit a closed-form variance marginal and an exact rejection envelope.

- **C7, two float64 failures fixed in the kernel without jitter** (Section 9). A Householder-QR draw of the contemporaneous coefficients replaces a Cholesky draw whose formed precision, with eigenvalues 1.9×10^{17} and 14.8 at a failing Geweke state, lost the small one, and a cancellation-free excess-pivot recursion factorises the random-walk SV path system at $\sigma \approx 5 \times 10^{-9}$. The recursion applies subtraction-free elimination (Grassmann et al., 1985; Alfa et al., 2002; Ye, 2008); we claim the application, not the algorithm.

- **C8, reproducible JAX MCMC with stated limits** (Section 4.6). Random keys are counter-based (Salmon et al., 2011) and derived per chain, sweep, block and equation. Bitwise invariances are tested with their scope stated: toy kernels for chunking and plans, production chain prefixes under `chains="map"`. Production chain prefixes under `chains="vmap"` and production `shard_map` runs agree with their references to a relative tolerance of 10^{-9}, and nothing is guaranteed across platforms or JAX versions.

- **C9, an empirical illustration** (Section 11). A pre-registered qualitative replication of Clark and Ravazzolo (2015) on FRED-QD with all 24 variable orderings illustrates the workflow; it does not test the econometric claim. In a first full run criteria 1, 2, 3 and 5 of six were met and criteria 4 and 6 were not; the diagnosis was that marginal $\hat{R}$ misses let the convergence gate decide one criterion and that the path-conditioned score degenerated at a few crisis origins, which drove the other. The settings were chosen in response to that diagnosis, so the pre-registration is explicitly not blind; what it fixes is that they were committed before the acceptance run was launched, before any of its scores existed. The acceptance run's outcome is [ACC: `criteria[*].met`].

[**TODO:** Prior-art searches (OUTLINE TODO 11) before any novelty or "to our knowledge" wording is added anywhere.] The priors, SV samplers, mixture approximations, the ancillarity–sufficiency interweaving strategy, the corrected triangular algorithm, precision samplers, tests of predictive ability, scoring rules, Geweke and SBC tests, mutation testing and subtraction-free elimination are all existing methods, credited where they are used.

Section 2 relates the package to existing software and states its licensing practice. Sections 3 to 5 present the models, the architecture and the MCMC samplers. Section 6 covers the predictive contract, the log-score estimators, the backtest and the verdict engine, and Section 7 walks through two examples. Section 8 validates the samplers, Section 9 describes the numerical failures that validation and replication runs exposed, and Section 10 reports performance. Section 11 presents the replication, Section 12 the limitations, and Section 13 a checklist and a roadmap.[1]

[1]The development process and the use of AI assistance are disclosed in Appendix J; the licensing statement and the notes in which GPL source was read are in Appendix H.

2 Related software, numerical oracles and licensing

Several existing packages were named as numerical oracles or references. We list them with the role each actually had and the versions used; parity results are in Section 8.9.

- The R package BVAR (Kuschnig and Vashold, 2021), version 1.0.5 (Kuschnig and Vashold, 2024), was run as the oracle for the hierarchical prior of Giannone et al. (2015), for the sum-of-coefficients and single-unit-root dummy observations, and for Metropolis–Hastings sampling of the hyperparameters.
- stochvol (Kastner, 2016; Hosszejni and Kastner, 2021), version 3.2.1, was run as the oracle for univariate SV (the design document names 3.2.9, but the committed fixtures were generated with 3.2.1).
- bayesianVARs (Gruber et al., 2026), version 0.1.8, was planned as the oracle for the corrected reduced-form sampler but could not be built in the development environment, which had no Fortran compiler (`gfortran`); we state no finding about it. The corresponding Phase 2 exit criterion was met by substitution: an exact system-wide Gibbs sampler written in R for this purpose serves as the reduced-form oracle.
- The Python package bvar of the Bank of England (Labonne et al., 2026), version 0.3.5 (MIT licence), is a second oracle for the analytic engine.
- bvartools (Mohr, 2026) was an architectural reference and was never run as an oracle.

Oracles named for later phases are factorstochvol (Kastner, 2026), bvars (Liu et al., 2026), bsvars (Woźniak, 2026), bsvarSIGNs (Wang and Woźniak, 2026) and bvarsv (Krueger, 2015); none is run in this release.

Other BVAR software is publicly available in Python (bvar-toolkit-python, Chan, 2026b; impulso, Pinder, 2026; srvar-toolkit, Shaw, 2026; and the `BayesianVARMAX` model of pymc-extras, PyMC Developers, 2026), in R and Python (bvhar, Kim and Baek, 2026), in MATLAB (the BEAR toolbox, Dieppe et al., 2016; bvar-toolkit, Chan, 2026a) and in R (BGVAR, Boeck et al., 2022), and frequentist VAR estimation is available in statsmodels (Seabold and Perktold, 2010); we do not compare features. [**TODO:** Re-verify at submission that every package named in this paragraph exists under this name (names only; OUTLINE TODO 12).]

The MCMC engine is written in JAX (Bradbury et al., 2018; Frostig et al., 2018), the analytic engine in NumPy (Harris et al., 2020) and SciPy (Virtanen et al., 2020), forecasts and MCMC posteriors are returned as xarray objects (Hoyer and Hamman, 2017), and the continuous ranked probability score (CRPS), its threshold-weighted version and the energy and variogram scores come from scoringrules (Zanetta and Allen, 2024); a comparable R package is scoringRules (Jordan et al., 2019). Reference values for the convergence-diagnostic tests were computed with ArviZ (Kumar et al., 2019) and hard-coded, and a further comparison runs only when ArviZ is installed; ArviZ is not a dependency. The samplers are hand-written Gibbs kernels by design, not models in a probabilistic programming language such as PyMC (Abril-Pla et al., 2023) or NumPyro (Phan et al., 2019). Each kernel exposes its exact conditional or its Metropolis–Hastings acceptance ratio, on which the identity tests (Section 8.4) and the deterministic replay of failures (Section 4) rely; we make no claim about how the two approaches compare in mixing.

rai-bvar is released under the Apache License, version 2.0. The R oracles BVAR and stochvol are GPL-licensed, and the project's specification therefore requires algorithms to be implemented from the published papers, not translated from GPL source. [**TODO:** Confirm the licence of each R package

named in this section on CRAN at submission (dossier 01, section 4.2).] The development practice is stated as follows:

> No code was translated from GPL sources. Algorithms were implemented from the published papers. GPL packages were run as numerical oracles, and in a few documented cases (named in the repository notes) their source was read to pin down an oracle's behaviour, defaults or constants, never to write implementation code.

Not every package whose source was read is an oracle of this release: the notes also record reading the sources of bayesianVARs, bvarsv, bvartools, factorstochvol and bsvars to confirm published constants and observable behaviour. This is weaker than a clean-room process in the strict sense, in which implementers never see the source, and we do not claim that stronger property. Five repository notes record a reading of GPL source,[2] and Appendix H lists them with the version read and the purpose, together with the notes in which no GPL source was read. The source of the MIT-licensed Bank of England package was read to document its conventions as an oracle. The committed parity fixtures that the R oracles produced are outputs of GPL programs, and the project treats them as data, not code. [**TODO:** Decide whether to cite the FSF GPL FAQ (bib key `fsf2026gplfaq`) on the status of program output, as a web page and not as legal advice (OUTLINE TODO 17).] The FRED-MD and FRED-QD databases (McCracken and Ng, 2016, 2021) are downloaded on demand and checked against SHA-256 checksums, never bundled, because some series carry third-party copyrights; committed test fixtures use only series from U.S. federal agencies.

3 The model framework

This section states the models that rai-bvar currently estimates. None of the model classes is new, and each is credited where it is defined; where the package makes its own choice within a class (the Mode I scaling and the default SOC scope below), the text says so.

3.1 Notation

The data are n variables y_t, observed at $t = 1, \dots, T$ after p initial observations, with regressors $x_t = (d_t', y_{t-1}', \dots, y_{t-p}')' \in \mathbb{R}^k$, $k = n_d + np$, where d_t holds $n_d \in \{0, 1, 2\}$ deterministic terms (none, a constant, or a constant and a linear trend). Y (columns y_i) and X stack the observations, $Y_{<i}$ holds the first $i-1$ columns of Y, and the column order of the data is the recursive ordering. We write κ for the Minnesota overall tightness (code field `lambda_`), keeping λ for structural variances. Equations are numbered $i = 1, \dots, n$ in the recursive order; when this position enters a prior formula as a number we write it ι, as in $\mathrm{IG}(1 + \iota/2, \psi_i/2)$ (the code numbers equations from 0, so there ι is the index plus one). $\mathrm{IG}(c, C)$ has shape c and scale C, $\mathrm{IW}(\Psi, d)$ has mean $\Psi/(d - n - 1)$, and $\mathrm{N}(m, V)$ has variance V. Table A.1 in Appendix A lists the notation with the code names of the main hyperparameters.

[2] `docs/phase2/02_sv_sampler.md`: the sources of stochvol 3.2.9, bayesianVARs 0.1.8 and bvarsv 1.1, read for behaviour and constants; `docs/phase2/01_omori_mixture.md`: the published mixture constants and observable behaviour, confirmed in six GPL packages; `docs/phase2/03_sv_outliers_t.md`: text searches of the stochvol sources, and of a MATLAB replication code by Carriero, Clark, Marcellino and Mertens that carries no licence; `docs/phase2/07_jax_patterns.md`: a search of one stochvol header for the ten published mixture constants, in a cross-check; `tests/parity/BVAR_PARAMETRIZATION.md`: the R sources of BVAR 1.0.5, read to establish its behaviour.

3.2 The recursive structural form

Every model estimated by MCMC is written as

$$Ay_t = \Gamma x_t + \Lambda_t^{1/2}\varepsilon_t, \qquad \varepsilon_t \overset{\text{iid}}{\sim} \mathrm{N}(0, I_n), \tag{1}$$

with A unit lower triangular (entries a_{ij}, $j < i$), Γ with rows γ_i', and $\Lambda_t = \mathrm{diag}(\lambda_{1,t}, \ldots, \lambda_{n,t})$. The reduced form is $y_t = B'x_t + u_t$, $u_t \sim \mathrm{N}(0, \Sigma_t)$, with $B' = A^{-1}\Gamma$ and $\Sigma_t = A^{-1}\Lambda_t A^{-1\prime}$. All current models have this linear mean. As $|A| = 1$, the log likelihood given the first p observations is

$$\log p(Y \mid A, \Gamma, \lambda) = \sum_{i=1}^{n}\sum_{t=1}^{T}\Big[-\tfrac{1}{2}\log(2\pi\lambda_{i,t}) - \frac{e_{i,t}^2}{2\lambda_{i,t}}\Big], \qquad e_{i,t} = y_{i,t} + \textstyle\sum_{j<i} a_{ij}y_{j,t} - x_t'\gamma_i, \tag{2}$$

the reduced-form Gaussian log density with no Jacobian term. It is one of the quantities that decide the engine's convergence flag (Section 5) and a test quantity of the validation suite (Section 8).

If the priors are independent across equations given the hyperparameters, the equation posteriors are independent conditional on the volatilities: equation i is a heteroskedastic regression with $k + i - 1$ regressors, and a sweep costs $O(nk^3) = O(n^4p^3)$ rather than the $O(n^6p^3)$ of a joint draw of all coefficients. This factorisation is exact in the structural form. A prior on the reduced-form B instead couples the equations; for that case the equation-by-equation algorithm of Carriero et al. (2019) does not target its intended posterior (Bognanni, 2022), and the package uses the corrected algorithm of Carriero et al. (2022) (Section 5).

3.3 Priors

All coefficient priors in the current release are of the Minnesota type (Litterman, 1986), in a conjugate form (analytic engine) or a structural or reduced form (MCMC engine); `form="auto"` is resolved from the volatility arm (Section 4).

Conjugate form. The analytic engine uses the normal–inverse-Wishart (NIW) prior (Kadiyala and Karlsson, 1997; Bańbura et al., 2010)

$$\Sigma \sim \mathrm{IW}(\mathrm{diag}\,\psi, d), \qquad \mathrm{vec}\,B \mid \Sigma \sim \mathrm{N}(\mathrm{vec}\,b, \Sigma \otimes \Omega), \qquad \Omega_{(l,j)} = \frac{\kappa^2(d-n-1)}{l^{\alpha}\psi_j}, \qquad d = n+2, \tag{3}$$

with Ω diagonal, (l, j) the coefficient on lag l of variable j, α the lag decay and $\Omega = \kappa_d$ (`var_const`) for deterministic terms. The scales ψ_j default to the residual variances of univariate AR(p) regressions with a constant on the estimation window (with the COVID-19 scaling below, on its periods before t^* when enough of them exist); the other defaults are $\kappa = 0.2$, $\alpha = 2$ and $\kappa_d = 10^7$. The mean b is zero except on the own first lag, where by default it is 1 for series in levels or log levels (transformation codes 1 and 4) and 0 for differenced series. The Kronecker structure ties own- and cross-lag tightness (Kadiyala and Karlsson, 1997), and the inverse-Wishart prior fixes the implied prior on the contemporaneous coefficients (Appendix B.1), so the cross-lag and contemporaneous scales κ_c and κ_a introduced below must be 1 in this form. Sum-of-coefficients (SOC) and single-unit-root (SUR) dummy rows (Doan et al., 1984; Sims and Zha, 1998) are built as in Giannone et al. (2015), except that by default (`soc_scope="persistent"`, a convention of this package) SOC rows exist only for variables with a non-zero own-lag prior mean, so that differenced variables are not pulled towards a unit root; `soc_scope="all"` restores the rows of Giannone et al. (2015). For the hyperparameters ϑ, `hierarchy="fixed"` keeps given values; `"eb"` (the default) and

`"mh"` use the hierarchy of Giannone et al. (2015) with their default hyperpriors (plus a prior on α, which Giannone et al. (2015) fix at 2, used only when α is estimated), the first by maximising $\log p(Y \mid \vartheta) + \log p(\vartheta)$, the second by random-walk Metropolis with (B, Σ) drawn exactly given each ϑ. With dummy rows, $p(Y \mid \vartheta)$ is the marginal likelihood of dummies and data divided by that of the dummies alone. The COVID-19 treatment of Lenza and Primiceri (2022) divides the rows of Y and X by a scale s_t common to all variables, which is 1 before a start period t^* (by default the period containing March 2020), free in t^* and the next two periods, and decays geometrically towards 1 afterwards; the model stays conjugate, and the log marginal likelihood gains the Jacobian $-n \sum_t \log s_t$.

Structural form. This is the default on the MCMC engine. The coefficients are centred conditionally on the contemporaneous row,

$$\mathrm{E}[\gamma_i \mid a_i] = M \tilde{a}_i, \qquad \tilde{a}_i = (a_{i1}, \ldots, a_{i,i-1}, 1, 0, \ldots, 0)', \tag{4}$$

with M the reduced-form prior mean b. Since $\gamma_i = B\tilde{a}_i$, this centring (D3 in the design documents) gives $\mathrm{E}[B' \mid A] = A^{-1}\mathrm{E}[\Gamma \mid A] = M'$ for every A. The same linear conditional centring, under a normalisation with unit-variance structural shocks, appears in Sims and Zha (2006), who attribute it to earlier work of theirs. [**TODO:** Confirm that their "Sims and Zha (1998a)" is Sims and Zha (1998) and that the conditional centring is in its text, then cite it here: N05 did not obtain the 1998 paper (form confirmed only via the MSBVAR documentation and the 2006 NBER draft).] In the coordinates $\delta_i = \gamma_i - M_{<i} a_i$ (unit Jacobian; $M_{<i}$ is the first $i-1$ columns of M), a_i and δ_i are independent a priori with diagonal precisions, and equation i is the regression

$$y_i = (XM_{<i} - Y_{<i})\, a_i + X\delta_i + e_i, \tag{5}$$

whose contemporaneous regressors are minus the prior-mean residuals of the earlier variables.

Three prior modes share this form. *Mode I*, independent and moment-matched, is the package's own default (design decision D-10) and is used by `Constant("ig")` and by every SV arm. With the fixed scale $\bar{\sigma}_i^2 = \psi_i/\iota$,

$$a_{ij} \sim \mathrm{N}(0, \kappa_a \bar{\sigma}_i^2/\psi_j), \qquad \delta_i \sim \mathrm{N}(m_i, \bar{\sigma}_i^2\, \Omega^{(i)}), \tag{6}$$

where κ_a (`contemp`) scales the contemporaneous prior variance, m_i is column i of M, and $\Omega^{(i)}$ is the Ω of (3) with the entries of lags of the other variables multiplied by κ_c^2; κ_c (`cross`) thus scales a prior standard deviation. At $\kappa_c = \kappa_a = 1$ Mode I reproduces three moments of the NIW prior exactly (the reduced-form mean for every A, the marginal variances of B and $\mathrm{E}\,\Sigma = \operatorname{diag}\psi$) but not its distribution. It thereby avoids the growth $\mathrm{E}\,\Sigma_{ii} = 2^{i-1}\psi_i$ along the ordering (Chan et al., 2024) that the unscaled choice $a_{ij} \sim \mathrm{N}(0, \psi_i/\psi_j)$, with $\mathrm{E}\,\lambda_i = \psi_i$, would imply. *Mode II* (`Constant("iw")` with a structural mean) replaces $\bar{\sigma}_i^2$ by λ_i in both blocks, with $\lambda_i \sim \mathrm{IG}(1+\iota/2, \psi_i/2)$. At $\kappa_c = \kappa_a = 1$ it is exactly the structural form of the NIW prior (3), so its posterior is the NIW posterior. Appendix B.1 gives a proof sketch, which uses the partition theorem for the inverse Wishart (Gupta and Nagar, 2000); this direction (inverse Wishart to structural normal–inverse-gamma) is Corollary 1 of Chan (2022), and his Proposition 1 is the converse. With $\kappa_c \neq 1$ it is the asymmetric conjugate prior of Chan (2022) with the centring (4). Finally, `Constant("iw_equivalent")` scales only a_i by λ_i and keeps the Mode I prior on δ_i (or, in reduced form, the coefficient prior below). At $\kappa_a = 1$ the implied Σ is exactly $\mathrm{IW}(\operatorname{diag}\psi, n+2)$ (Chan, 2022, Proposition 1), and in reduced form the model is invariant to the ordering of the variables.

Reduced form. The columns π_j of B get independent priors $\mathrm{N}(m_j, \psi_j \Omega^{(j)})$, which match the NIW marginal variances at $\kappa_c = 1$. The prior on (A, λ) is the structural one, so the two forms differ only in the coefficient prior.

Hyperparameters on the MCMC engine. Hyperparameters are fixed during MCMC: at given values (`hierarchy="fixed"`) or, by default (`"eb"`), at the empirical-Bayes mode of the analytic conjugate twin, fitted at $\kappa_c = \kappa_a = 1$ on the same window. Two cases differ when $\kappa_c \neq 1$: in structural form the mode of the per-equation closed-form marginal likelihood of Chan (2022), plus the same log hyperprior, is used instead, with κ_c and κ_a held fixed and the twin's mode as the starting point; in reduced form, where no closed form exists, the twin's mode is used and a note is recorded with the fit. This value depends only on the window, so every volatility arm in a comparison gets the same mean prior. It is the empirical-Bayes mode of the model being estimated only when that model is Mode II; for every other composition (Mode I, `iw_equivalent`, the reduced form and the SV arms) it is a documented plug-in. `hierarchy="mh"` and the COVID-19 scaling raise an error on MCMC compositions, for which SV with outliers is the available treatment of the 2020 observations.

3.4 Volatility arms

Constant. Under `Constant`, $\lambda_{i,t} = \lambda_i \sim \mathrm{IG}(1 + \iota/2, \psi_i/2)$, with mean ψ_i/ι, in all three modes. In Mode II and `iw_equivalent` the λ-scaled coefficient blocks enter the inverse-gamma full conditional of λ_i through one coupling channel, which adds half their number to the shape and half their λ-free prior quadratic form to the scale.

Stochastic volatility. Under `SV` the log-variance $h_{i,t} = \log \lambda_{i,t}$ of each structural equation follows

$$h_{i,t} = \mu_i + \phi_i (h_{i,t-1} - \mu_i) + \sigma_i \eta_{i,t}, \qquad \eta_{i,t} \overset{\text{iid}}{\sim} \mathrm{N}(0,1), \tag{7}$$

or, with `rw=True`, the random walk $h_{i,t} = h_{i,t-1} + \sigma_i \eta_{i,t}$. The priors use the parametrisation of `stochvol` (Kastner, 2016; Hosszejni and Kastner, 2021) with the package's own defaults: $\mu_i \sim \mathrm{N}(\log(\psi_i/\iota), 10)$, centred in each window on the log of the Mode I scale; $(\phi_i + 1)/2 \sim \mathrm{Beta}(20, 1.5)$; $\sigma_i^2 \sim B_\sigma \chi_1^2$ with $B_\sigma = 1$ (0.1 for the random walk), or an inverse gamma; $h_{i,0}$ from the stationary law, or $\mathrm{N}(\log(\psi_i/\iota), 4)$ for the random walk. [**TODO:** Credit the triangular VAR-SV form itself here (Section 1 cites Cogley and Sargent (2005); Primiceri (2005); Clark (2011) for SV in VARs); check each paper's exact specification before naming it as the source of (1) with (7).]

Robust SV. Outliers and Student-t errors are transitory scales of the structural residual,

$$e_{i,t} = o_{i,t}\, \tau_{i,t}^{1/2}\, e^{h_{i,t}/2}\, \varepsilon_{i,t}, \qquad \lambda_{i,t} = e^{h_{i,t}}\, o_{i,t}^2\, \tau_{i,t}, \tag{8}$$

and $\lambda_{i,t}$ in (2) is then this effective variance. With `outliers=True`, as in Carriero et al. (2024), $o_{i,t} \in \{1, \ldots, 20\}$ (their discretisation of a uniform outlier scale on $[2, 20]$) is 1 with probability $1 - p_i$ and otherwise uniform; the Beta prior on p_i encodes, by default, one outlier every 4 years over 10 prior years (every 10 years with t errors). [**TODO:** The outline credits the outlier grid to Stock and Watson (2016) ("in the spirit of"); N03 has the 2016 specification second-hand only (published text not accessed). Confirm against the REStat text before citing it here.] With `tails="t"`, $\tau_{i,t} \sim \mathrm{IG}(\nu_i/2, (\nu_i - 2)/2)$, a standardised t with $\mathrm{E}\,\tau_{i,t} = 1$, or, with `t_scale="unit"`, the $\mathrm{IG}(\nu_i/2, \nu_i/2)$ scale mixture of Jacquier et al. (2004); ν_i is uniform on $\{3, \ldots, 40\}$ and drawn by an exact collapsed discrete step. The two components can be combined.

Table 1: Components of the current release by axis, with the engine and form that estimate them and the registered validation rows (Section 8) in which they appear. Rows are numbered as in `src/rai_bvar/validation/joint.py`: 1–17, and 201–205 with SOC/SUR rows; rows 15 and 16 change the σ_i^2 and $h_{i,0}$ priors, and row 17 is a zero-mean univariate SV model. Rows as committed at c823d20. "Later" follows the repository's `SPEC.md`, §4.4 and §13; the COVID-19 entry follows the engine's error message (Phase 3).

Axis	Component	Engine (form)	Validation rows
Mean	`Linear`, $x_t'\gamma_i$	both engines	all except 17
	Later: BART (Phase 4); TVP, threshold and Markov-switching means (Phase 5).		
Prior	`Minnesota`, conjugate NIW; hierarchy fixed, EB or MH	analytic	none (analytic tests, parity)
	structural, Mode I	MCMC	1–4, 12, 14–16, 201, 204
	structural, Mode II	MCMC	5, 202
	structural, `iw_equivalent`	MCMC	6, 205
	reduced form	MCMC	7–11, 13, 203
	SOC/SUR dummy rows	both engines	201–205
	Later: horseshoe, normal-gamma, Dirichlet–Laplace, R2D2 and SSVS priors; the asymmetric conjugate prior as a prior class (Phase 3).		
Volatility	`Constant("iw")`	analytic; MCMC (Mode II)	5, 202
	`Constant("ig")`	MCMC (both forms)	1, 7, 201, 203
	`Constant("iw_equivalent")`	MCMC (both forms)	6, 11, 205
	`SV()`, AR(1) log-variance	MCMC (both forms)	2, 8, 15–17, 204
	`SV(rw=True)`	MCMC (both forms)	14 (structural only)
	`SV(outliers=True)`	MCMC (both forms)	3, 9
	`SV(tails="t")`	MCMC (both forms)	4, 10
	`SV(outliers=True, tails="t")`	MCMC (both forms)	12, 13
	Later: common SV (Carriero et al., 2016) (Phase 3); factor SV (Kastner, 2019) and order-invariant SV (Chan et al., 2024) (Phase 5).		
COVID-19	scaling of Lenza and Primiceri (2022)	analytic only	none (analytic tests)
	`SV(outliers=True)`	MCMC	3, 9, 12, 13
	Later: the scaling on the MCMC engine (Phase 3).		
Benchmarks	`RandomWalk`, `OLSVAR`	own simulator	none

3.5 What the current release delivers

Table 1 lists the components by axis, the validation rows in which each appears, and the roadmap. "Ablation-ready" describes the architecture: the current release populates the volatility axis with five volatility families (`Constant`, with three covariance priors; `SV` with AR(1) or random-walk log-variance; and SV with outliers, t errors or both), while the mean axis is linear and the prior axis is Minnesota. The random-walk arms in reduced form, `SV(rw=True)` (the primary arm of the replication in Section 11) and its Student-t variant, have no validation row of their own (Section 8). [**TODO:** TODO 3: row 18, `reduced_sv_rw` (reduced form × SV(rw)), and a `reduced-SV(rw)` contract fit were committed in 7f410dc, after the acceptance commit c823d20; the SV(rw, t) variant is still not covered. If the release commit includes it, re-check this sentence, the caption of Table 1 ("1–17") and its `SV(rw=True)` row; Sections 8.10, 11.8 and 12 make the same coverage statement.] The equivalences above are tested: the analytic log marginal likelihood satisfies the Chib identity; Mode I has the NIW moments; the prior draws of Mode II pass Kolmogorov–Smirnov tests against NIW prior draws, and its closed-form log marginal likelihood equals the analytic engine's to an absolute 10^{-10} in five configurations; and the Σ implied by `iw_equivalent` passes Kolmogorov–Smirnov tests against IW(diag $\psi, n+2$). The Mode II identity makes the analytic engine an exact posterior oracle for the Mode II `Constant` compositions of the MCMC engine; the Mode I and `iw_equivalent` tests check priors only (Section 8).

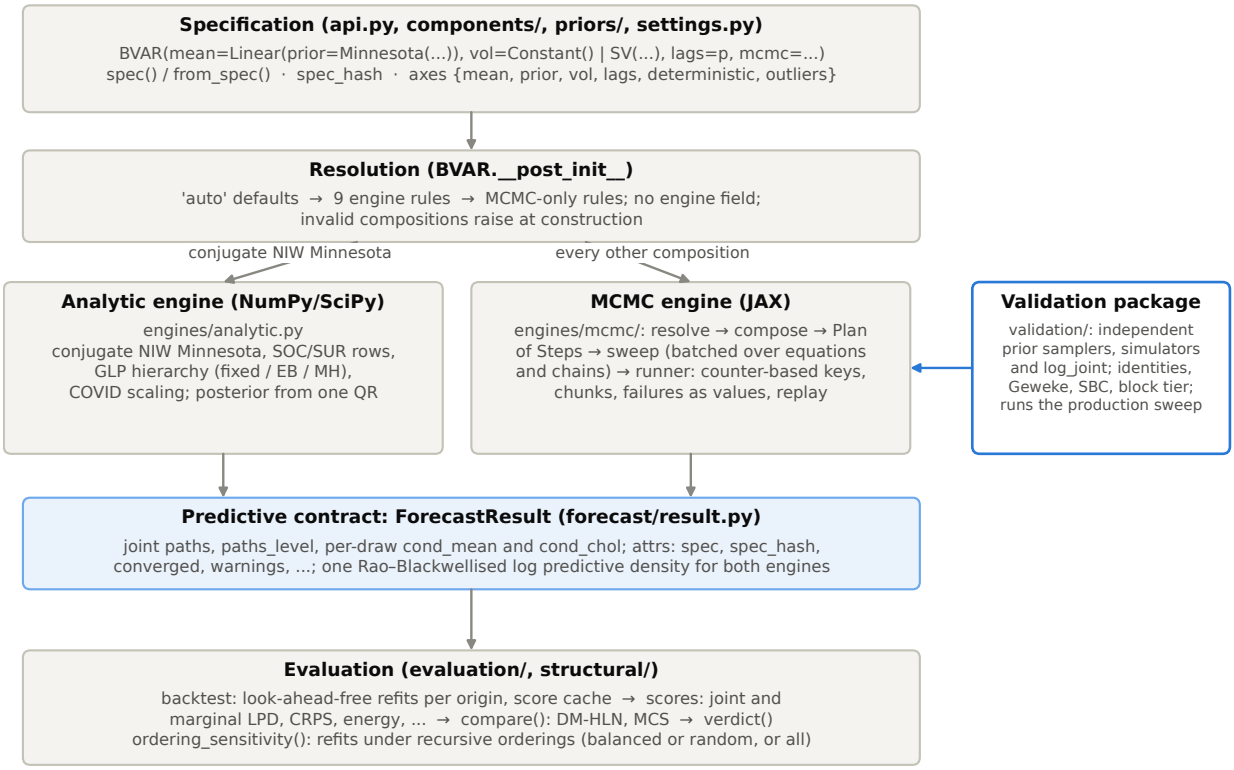


Figure 1: Architecture of rai-bvar. Specifications and engine resolution select the analytic (NumPy/SciPy) or the MCMC (JAX) engine. Both return the same `ForecastResult` contract, which feeds the backtest, the scores, `compare()` and `verdict()`. The validation package wraps the production sweep.

4 Software architecture

4.1 Design rules

Nine design rules fix the shape of the package. Five shape the interface: composition over inheritance (R1: a model is `BVAR(mean, vol, ...)`, with no model-specific classes), volatility first (R2), one predictive contract for both engines (R3), the variable ordering surfaced rather than hidden (R8), and no look-ahead (R9: everything data-dependent is recomputed inside each estimation window). Three govern the internals: float64 throughout, no explicit inverses, precision-based samplers and log-space densities (R4); no sampler merged without passing Geweke and simulation-based calibration tests (R6; Section 8); and one MCMC backend, hand-written Gibbs kernels in JAX (Bradbury et al., 2018; Frostig et al., 2018) with no probabilistic programming language in the core (R7). R5 concerns licensing (Section 2). Figure 1 shows the resulting layers.

4.2 Composition and engine resolution

A model is a frozen dataclass `BVAR(mean, vol, lags, deterministic, outliers, mcmc)` with no engine field. Its constructor resolves the `"auto"` defaults (a Minnesota form left at `"auto"` becomes `"conjugate"` with a `Constant` prior left at `"auto"` or set to `"iw"`, and `"structural"` otherwise; `Constant("auto")` then becomes `"iw"` or `"ig"`), applies the nine rules of Table 2, and, for MCMC compositions, rejects what the engine does not support: Lenza–Primiceri scaling and the Metropolis–Hastings hyperparameter hierarchy (both planned for a later phase), and empirical

#	Prior form	Volatility	Engine and mode
1	conjugate	`Constant("iw")`	analytic (conjugate NIW)
2	conjugate	any other	`ValueError`
3	structural	`Constant("ig")`	MCMC, Mode I
4	structural	`Constant("iw")`	MCMC, Mode II (NIW twin); prior label `scaling=conjugate`
5	structural	`Constant("iw_equivalent")`	MCMC, Mode I δ prior with the (a, λ) prior of Chan (2022)
6	structural	`SV(...)`	MCMC, Mode I
7	reduced	`Constant("ig")`, `SV(...)`	MCMC, corrected triangular (CCCM) step
8	reduced	`Constant("iw_equivalent")`	MCMC, CCCM step with Chan's (a, λ) prior; order-invariant
9	reduced	`Constant("iw")`	`ValueError`

Table 2: Engine resolution: the rules that map a resolved (prior form, volatility prior) to an engine and mode. The first match applies, and errors are raised at construction. `BVAR()`, `BVAR(vol=Constant("ig"))` and `BVAR(vol=SV())` reach rules 1, 3 and 6.

Bayes over ψ with cross-lag tightness $\kappa_c \neq 1$ in structural form. Every error is raised at construction. A parametrised test encodes the seven non-error rows of the table case by case, and further tests check each construction error with its message. Because the resolved components are stored, `BVAR()` keeps the meaning and hash it had before the MCMC engine existed.

`spec()` returns plain JSON, `from_spec(spec())` round-trips (a property-based test), and a golden test checks that every Phase 1 (conjugate) model keeps the specification hash, name and axes recorded with the Phase 1 code. The property `axes` splits a model into mean, prior, volatility, lags, deterministic terms and outliers, and each axis label encodes every non-default setting, so "differs along exactly one axis" is a property that `compare()` checks (Section 6.8). Mode II changes the mean prior, so its label carries `scaling=conjugate`, and it is never paired with a Mode I model along the volatility axis. `import rai_bvar` imports no JAX, which a subprocess test checks; the MCMC engine is loaded at the first MCMC fit.

4.3 The analytic engine

The conjugate engine uses NumPy (Harris et al., 2020) and SciPy (Virtanen et al., 2020). One QR decomposition of the stacked Minnesota dummy rows, SOC/SUR rows and data gives the posterior; $(R^\top R)^{-1}$ is never formed, and draws use the Bartlett decomposition. Empirical Bayes maximises the log marginal likelihood plus the log hyperprior by L-BFGS-B, and the hierarchical option is the random-walk Metropolis–Hastings scheme of Giannone et al. (2015). The engine also computes recursive impulse responses and the forecast-error variance decomposition.

4.4 The MCMC engine in six ideas

1. *The static/traced split is the compilation cache key.* Choices that fix a shape or a branch live in a frozen, hashable `StaticConfig`; data, priors, hyperparameters, seeds and the sample length are traced. Time is padded, by default to $T_{\text{pad}} = 32\lceil T/32 \rceil$, with zero-weight rows, masked before any logarithm, so backtest windows in one bucket share compiled programs; a test of two such windows sees no recompilation.

2. *Equations and chains are `vmap` axes.* Given the volatilities the structural equations are conditionally independent, so kernels are written for one equation and batched, and the

batched phase is an exact Gibbs block. Chains run under `vmap` (the default), `lax.map` or `shard_map`.

3. *The Gibbs order is data.* A `Plan` of `Step`s with declared read and write sets is built in one function and printed by `fit.plan.describe()`. Invariants are checked for every composition, including a tree diff showing that each step writes only what it declares. The mixture indicators are local to the volatility step and never part of the state, so the rule of Del Negro and Primiceri (2015) that nothing is drawn between the indicators and the log volatilities holds by construction.

4. *Kernels are pure and take injected noise:* `sample_core(draw_noise(key, ...), ...)`. Every exact draw exposes its conditional and every Metropolis–Hastings step its log acceptance ratio; the density-ratio identity tests of Section 8 use both, and the failure replay evaluates the exposed conditionals.

5. *Randomness is counter-based* (Salmon et al., 2011). Keys are `fold_in` chains over stream, data salt, chain, absolute sweep, block, equation and sub-draw, with unsigned 32-bit counters and no key in any loop carry, so a draw depends on its position in the chain and not on how sweeps are grouped into compiled calls.

6. *Failures are values.* Nothing raises inside compiled code and no jitter is added. Each chain keeps its first failure as (code, *it*), with code $= 1 + 1000 \cdot \text{block} + \text{equation}$; the host replays the failing chunk deterministically, evaluates the failing block's exposed conditional, and raises `MCMCNumericalError` with the block, equation, iteration and diagnostics. No chain is dropped. Section 9 describes failures that surfaced this way.

4.5 Settings and runtime

`MCMCSettings` is part of the specification: by default 4 chains of 2000 kept draws after a burn-in of 1000, with convergence thresholds $\hat{R} < 1.01$ and bulk ESS > 400 on every primary monitored quantity. `Runtime` is not: it holds the device count (default 1), the kept draws per compiled call, a memory budget that only decides whether to raise before sampling, a memory-mapped store, and whether the host keeps replay copies. The backtest never passes a `Runtime`, so cached scores do not depend on the machine's device count. An `MCMCFit` holds the draws as an xarray `DataTree` (Hoyer and Hamman, 2017), diagnostics, a `summary()` (prior mode, hyperparameters, implied reduced-form prior moments, ordering), the `converged` flag, `forecast()`, `irf()` and the final state for warm starts.

4.6 Reproducibility, stated with its limits

Bit-for-bit invariance is tested with the runner's test doubles ("toys") for chunk and burn-in piece size, aligned thinning, fused versus unfused plan execution, `vmap` over chains versus a loop, and memory-mapped versus in-memory storage, and with the production kernels for chain-count prefixes under `chains="map"`. With the production kernels, chain-count prefixes under `vmap` (SV and CCCM kernels) and `vmap` versus `shard_map` over four devices (structural SV) are equal only to rounding, because XLA vectorises reductions differently for different batch sizes; the tests compare them to a relative tolerance of 10^{-9} over a few burn-in sweeps. In-loop and post-hoc forecasts agree to 10^{-12}. The toy chain-prefix test is bitwise on macOS and holds only to rounding on x86-64 Linux. Chunk invariance with the production kernels follows from the key layout but is not tested,[**TODO:** Dossier 03 Q17: add a production-kernel chunk test, or keep "not tested".] and nothing is

guaranteed across platforms, JAX versions or PRNG implementations (Table G.1 in Appendix G). Every fit records the package and JAX versions, PRNG implementation, platform, device count, seed, salt, resolved numerical paths, ordering and specification hash. One hazard is avoided by layout: under partitionable threefry, words drawn directly from a parent key equal the key data of its `fold_in` children, which JAX's key-reuse checker cannot see. The key layout therefore gives each consumer its own sub-key instead of drawing from a key whose children are derived; a test checks this for the volatility core's forecast draws, where a draw from the block key would reuse the words of the transitory sub-keys.

4.7 Extension protocol

New components implement `MeanBlock`, `VolBlock`, `CoefPrior` or `GlobalStep` and register an independent reference implementation, from which the validation package builds their per-block and per-composition Geweke configurations. That later components will plug in without editing the sweep, runner or forecast core is a design intention, not yet tested.

5 The MCMC samplers

We claim no new sampler: each block below follows published work, credited where it is used, and what was derived in the design notes rather than taken from a paper (such as the single accept/reject of the non-centred step in exact mode) is marked as such. The full pseudocode is in Appendix C. We write $\lambda^{\text{eff}}_{i,t} = e^{h_{i,t}} o^2_{i,t} \tau_{i,t}$ for the effective variance of structural equation i ($\lambda_i o^2_{i,t} \tau_{i,t}$ under constant volatility); sums over t run over observed rows, and padded rows carry weight zero.

5.1 One sweep

In structural form each equation, under `vmap`, draws (δ_i, a_i) jointly from one Gaussian conditional, recomputes its residual and runs its volatility block atomically (Algorithm 1). In the volatility block the transitory steps run first ($o \to p \to \nu \to \tau$), given the current persistent variance; in the SV core the mixture indicators are then drawn immediately before the log-volatility path, followed by the centred and the non-centred parameter steps. In reduced form a pre-phase step draws $\Pi \mid A, \lambda$ (Section 5.3), and each equation then draws $a_j \mid \Pi, \lambda_j$. The coefficient steps condition on the exact Gaussian likelihood given λ^{eff} and are marginal of the indicators, as Del Negro and Primiceri (2015) require; the transitory steps with τ integrated out are partially collapsed Gibbs blocks (van Dyk and Park, 2008) whose trimmed draw is never conditioned on before it is redrawn.

5.2 The structural coefficient draw

In the δ-coordinates of Section 3, every equation regresses on one shared design $F = [X \mid XM_{<n} - Y_{<n}]$; the slots a_{ij}, $j \geq i$, are inactive (prior mean 0, precision 1, zero columns) and are zeroed after the draw, which leaves the active draw exact. The weighted Gram $F^\top \Lambda_i^{-1} F$, $\Lambda_i = \operatorname{diag}(\lambda^{\text{eff}}_{i,\cdot})$, comes by default from a packed product of the columns of F stored once per fit, so under `vmap` the Grams of all chains and equations are one matrix product. The draw is a Jacobi-scaled Cholesky precision draw,

$$\theta_i = s \odot L^{-\top}\big(L^{-1}(s \odot b) + z\big), \qquad LL^\top = \operatorname{diag}(s)\, K \operatorname{diag}(s), \qquad s = \operatorname{diag}(K)^{-1/2}, \tag{9}$$

with $z \sim N(0, I)$, so $\theta_i \sim N(K^{-1}b, K^{-1})$ exactly. Under constant volatility without a transitory scale the weights do not vary over time, so the data part of the precision is G/λ_i with G fixed per

Algorithm 1 One Gibbs sweep of one chain (compact form).

Input: chain key, absolute sweep index it (unsigned 32-bit), state

if reduced form **then** $\Pi \leftarrow$ CCCM scan over $\pi_j \mid \pi_{-j}, A, \lambda^{\text{eff}}$ ▷ Section 5.3

for all equations i **in parallel** (`vmap`); keys `fold_in`(it, block, i, sub-draw) **do**

  structural: $(\delta_i, a_i) \sim N(K_i^{-1} b_i, K_i^{-1})$; reduced: $a_i \mid \Pi, \lambda^{\text{eff}}_{i,\cdot}$ by QR ▷ Section 5.2

  $e_{i,t} \leftarrow$ structural residual from the new coefficients

  $o_i \to p_i \to \nu_i \to \tau_i$ given e_i and the persistent variance ▷ Section 5.6

  $\log \tilde{e}^2_{i,t} \leftarrow \log e^2_{i,t} - 2 \log o_{i,t} - \log \tau_{i,t}$

  Constant: $\lambda_i \sim$ IG (13); SV: $s_i \to h_i \to \theta_i^{\text{SV}}$ (Algorithm 2)

end for

keep the first failure (code, it), $\text{code} = 1 + 1000 \cdot \text{block} + \text{equation}$

fit: $K(\lambda_i) = \text{diag}(d) + G/\lambda_i$ in Mode I, and $K = \tilde{K}/\lambda_i$ with a λ-free $\tilde{K}$ in Mode II, where the prior precision also scales with $1/\lambda_i$. A deterministic rule then picks an exact fast path: `eigen` for Mode I (one graded eigendecomposition per fit, then $O(q^2)$ per draw), `fixed_factor` for Mode II and `cholesky_const` for `iw_equivalent`. The paths share the target, but two exact samplers of one Gaussian agree only to $O(\epsilon_{\text{mach}} \, \text{cond}(K))$, so the tests use a conditioning-aware envelope (Section 8). Where a_i is drawn on its own, as in the reduced form, the engine takes a Householder QR of the whitened stacked design [diag(prior precision)$^{1/2}$; $\Lambda_i^{-1/2} Z$] with $\text{diag}(R) > 0$; R is then the upper Cholesky factor of the precision, the draw is the same function of z as (9), and the precision is never formed; preferring QR to the normal equations is standard least-squares practice (Higham, 2002; Björck, 1996), and Section 9 shows why it matters here.

5.3 The corrected reduced-form step

The columns π_j of Π are drawn from their exact full conditionals (Carriero et al., 2022):

$$K_j = V_j^{-1} + X^\top \text{diag}(W_{\cdot j}) \, X, \qquad b_j = V_j^{-1} m_j + X^\top \big(g_{\cdot j} + W_{\cdot j} \odot X \pi_j \big), \tag{10}$$

where $W_{tj} = (\Sigma_t^{-1})_{jj} = \sum_{i \geq j} A_{ij}^2 / \lambda^{\text{eff}}_{i,t}$ (with $A_{jj} = 1$) and $g_{tj} = (\Sigma_t^{-1} u_t)_j$, with u_t the current reduced-form residual, so A is never inverted. K_j does not depend on Π, so by default all n precisions are factored in parallel before the sequential scan (a deterministic shape rule turns this precompute off above a memory bound). The scan is block Gauss–Seidel on the joint precision of $\text{vec}\,\Pi$ (Fox and Parker, 2017). The earlier version (Carriero et al., 2019), which used equation j of the triangular system alone, targets the wrong posterior (Bognanni, 2022); its conditional is correct only for the last equation, and it survives as an injected bug for the mutation tests.

5.4 The stochastic-volatility core

Algorithm 2 works on $y_t^* = \log(\tilde{e}_t^2 + c)$ with, by default, the relative offset $c = 10^{-12} \psi$ (an absolute offset can be set). The $\log \chi_1^2$ density is approximated by the ten-component mixture of Omori et al. (2007), with component means m_j and variances v_j (the seven-component mixture of Kim et al. (1998) is an option), and the indicators are drawn by inverse CDF. The path $(h_0, \ldots, h_T)$ is drawn in one block from its tridiagonal Gaussian conditional (Rue, 2001; Chan and Jeliazkov, 2009; McCausland et al., 2011), the "all without a loop" (AWOL) sampler in the terminology of Kastner and Frühwirth-Schnatter (2014), in non-centred scaled form $\tilde{Q} = \sigma^2 Q$. By default (`exact=True`) an exact Metropolis–Hastings correction removes the mixture approximation: with g the mixture

Algorithm 2 The SV core of one equation (AR(1) log variance, exact mode).

Input: $\log \tilde{e}_t^2$; state $(h_{0:T}, \mu, \phi, \sigma)$; injected noise
1: $y_t^* \leftarrow \log(\tilde{e}_t^2 + c)$; $s_t \sim \Pr(s_t = j \mid y_t^*, h_t)$ by inverse CDF
2: $h' \leftarrow \mu + \sigma\tilde{x}'$, $\tilde{x}' \sim N(\tilde{Q}^{-1}\tilde{b}, \tilde{Q}^{-1})$, $\tilde{Q}$ tridiagonal ▷ AWOL
3: accept h' with probability $\min\{1, e^{w(h')-w(h)}\}$ ▷ (11)
4: centred: (γ, ϕ), $\gamma = \mu(1-\phi)$, by independence MH; then σ^2
5: non-centred: $\tilde{h} \leftarrow (h-\mu)/\sigma$; ϕ by MH given $\tilde{h}$
6: $(\mu', \sigma') \leftarrow 2 \times 2$ Gaussian regression of $y_t^* - m_{s_t}$ on $(1, \tilde{h}_t)$
7: accept with $\log\alpha = \log\alpha_{\text{aux}} + w(\mu' + \sigma'\tilde{h}) - w(h)$ ▷ $w(h)$ after line 3
8: on acceptance: **if** $\sigma' < 0$ **then** $(\sigma', \tilde{h}) \leftarrow (-\sigma', -\tilde{h})$; $(\mu, \sigma, h) \leftarrow (\mu', \sigma', \mu' + \sigma'\tilde{h})$

density and

$$w(h) = \sum_t \big[\log N(\tilde{e}_t; 0, e^{h_t}) - \log g(y_t^* - h_t)\big], \tag{11}$$

the path proposal is accepted with probability $\min\{1, e^{w(h')-w(h)}\}$ (Del Negro and Primiceri, 2015, Algorithm 3), an idea with a precedent in Stroud et al. (2003).[**TODO:** N01 marks Stroud et al. (2003) "not opened": confirm its eq. (11), or drop the citation.][**TODO:** N01 read Del Negro–Primiceri as NY Fed Staff Report 619 (rev. Oct. 2014) with its online appendix (App. B.2, eq. 7); confirm that "Algorithm 3" is numbered so in the cited RES corrigendum, or cite the staff report.] The chain is then exact for any mixture table and offset $c \geq 0$, which the validation exploits with a deliberately coarse mixture (Section 8). The centred steps need no correction: they do not move h, and the indicator conditional does not involve the parameters. The non-centred step (on by default, `asis=True`) interweaves (Yu and Meng, 2011; Kastner and Frühwirth-Schnatter, 2014); in exact mode it is accepted or rejected once, with a ratio combining the auxiliary ratio and the change in w, a combination derived in the design notes (Appendix B.3) and not claimed as new.[**TODO:** TODO 11: prior-art search for this combined ratio (docs/phase2/01_omori_mixture.md §10.3; Appendix B.3).] A negative σ' is folded together with $\tilde{h}$, which preserves the non-centred posterior by the sign symmetry used by Frühwirth-Schnatter and Wagner (2010). Each chain starts with one unconditional AWOL draw of h, because a prototype found $\sigma = 0$ absorbing from a constant initial path. For a random-walk log variance the centred step draws σ^2 only, and the non-centred regression is on (h_0, σ).

5.5 The random-walk path as a path Laplacian

For $h_t = h_{t-1} + \sigma\eta_t$, $h_0 \sim N(m_0, V_0)$, the scaled precision is $\tilde{Q} = \sigma^2 Q = \text{Lap}_w + \text{diag}(\delta)$: a path-graph Laplacian with unit edge weights w_t (zero after the last live state) plus $\delta_0 = \sigma^2/V_0$ and $\delta_t = \sigma^2/v_{s_t}$. Algorithm 3 carries each pivot as its excess q_t over the next edge:

$$q_0 = \delta_0, \quad q_t = \delta_t + \frac{w_{t-1}\, q_{t-1}}{w_{t-1} + q_{t-1}}, \quad p_t = q_t + w_t, \quad \log|\tilde{Q}| = \sum_t \big[\log w_t + \text{log1p}(q_t/w_t)\big], \tag{12}$$

with $\log q_t$ where no next edge exists. The p_t are the Cholesky pivots of $\tilde{Q}$ (Appendix B.8), but every term is non-negative, so nothing cancels; assembling $\tilde{Q}$ first rounds σ^2/v_{s_t} away once it is below the unit in the last place of the diagonal, and the pivot of the level direction becomes rounding noise (Section 9). We read the recursion as an instance of GTH-type subtraction-free elimination (Grassmann et al., 1985; Alfa et al., 2002; Ye, 2008); we claim its application here, not the algorithm.[**TODO:** TODO 11: confirm this positioning against GTH, Alfa–Xue–Ye, Ye (2008) and GMRF samplers.]

Algorithm 3 Random-walk SV path draw by excess pivots.

Input: $\delta_{0:T} \geq 0$; edge weights $w_{0:T-1} \in \{0,1\}$, $w_T = 0$; $\tilde{b}$; $z \sim N(0, I)$

1: $q_0 \leftarrow \delta_0$; $q_t \leftarrow \delta_t + w_{t-1} q_{t-1}/(w_{t-1} + q_{t-1})$, $t = 1, \ldots, T$ ▷ all terms ≥ 0

2: $\ell_t \leftarrow \sqrt{q_t + w_t}$; $c_t \leftarrow -w_{t-1}/\ell_{t-1}$

3: forward $v_t \leftarrow (\tilde{b}_t - c_t v_{t-1})/\ell_t$; backward $\tilde{x}_t \leftarrow (v_t + z_t - c_{t+1}\tilde{x}_{t+1})/\ell_t$

4: $\log|\tilde{Q}|$ by (12); $h \leftarrow \sigma\tilde{x}$

5.6 Transitory components

With $z_t^2 = e_t^2/\lambda_t^{\text{pers}}$, the outlier scale $o_t \in \{1, \ldots, K\}$ ($K = 20$ by default) has logits $\ell_{tk} = \log \varpi_k - \log k - z_t^2/(2k^2)$, $\varpi_1 = 1 - p$, $\varpi_{k>1} = p/(K-1)$ (Carriero et al., 2024; Stock and Watson, 2016), and $p \mid o \sim \text{Beta}(a_p + N_1, b_p + T - N_1)$ with $N_1 = \#\{t : o_t > 1\}$. For Student-t errors (Jacquier et al., 2004), ν is drawn on its grid ($\{3, \ldots, 40\}$ by default) with τ integrated out, then $\tau_t \sim \text{IG}\big((\nu+1)/2, (\nu - 2 + r_t^2)/2\big)$ in the standardised parametrisation (the default; ν replaces $\nu - 2$ under `t_scale="unit"`), $r_t = z_t/o_t$; with both components the outlier step also integrates τ out. The log-gamma variates are drawn by a vectorised Marsaglia and Tsang (2000) sampler with two pre-drawn candidates per element and a loop over further candidates only for the elements that reject both, rather than by `jax.random.loggamma`, because under `vmap` a rejection loop runs until the last lane accepts; it stays exact, since the first accepted candidate of an independent sequence is the rejection sampler.

5.7 The constant-volatility core

Under constant volatility

$$\lambda_i \mid \cdot \sim \text{IG}\Big(c_i + \tfrac{T}{2} + \tfrac{n_{\text{active}}}{2},\ C_i + \tfrac{1}{2}\textstyle\sum_t \tilde{e}_{i,t}^2 + \text{quad}\Big), \tag{13}$$

where $(n_{\text{active}}, \text{quad})$, the number of coefficients whose prior scales with λ_i and half their prior quadratic form, is passed from the coefficient step of the same equation: zero in Mode I, all $k + \iota - 1$ coefficients in Mode II, and the $\iota - 1$ contemporaneous ones under `iw_equivalent` (ι the one-based equation index). This is the only interaction between the prior and the volatility block that the plan allows. In Mode II the SOC/SUR rows are weighted by $1/\lambda_i$ like data rows, so the λ step counts them too (Section 9).

6 Forecasts, the predictive contract and the verdict engine

Both engines return the same object, a `ForecastResult`, and everything downstream of a fit (scores, backtests, tests and verdicts) reads only that object. None of the scoring rules, tests or estimators below is new; what the package adds is one contract that makes them available, with their Monte Carlo error, to the compositions of Section 3 (the path-integrated estimator of Section 6.4 to MCMC forecasts only).

6.1 Joint path simulation

Let $s = 1, \ldots, S$ index forecast draws and T be the forecast origin. For each selected posterior draw, the MCMC engine first simulates the future log variances, including future outlier and Student-t

scales; $\lambda^{\text{eff}}_{s,T+h}$ denotes the effective variances (with those scales) and $\lambda^{\text{pers}}_{s,T+h}$ the persistent ones. It then runs over $h = 1, \dots, H$, building $x_{s,T+h}$ from the simulated path:

$$m_{s,h} = B_s' \, x_{s,T+h}, \qquad L_{s,h} = A_s^{-1} \operatorname{diag}\big(\lambda^{\text{eff}}_{s,T+h}\big)^{1/2}, \qquad y_{s,T+h} = m_{s,h} + L_{s,h} \, \varepsilon_{s,h}, \tag{14}$$

with $\varepsilon_{s,h} \sim N(0, I_n)$ and $B_s' = A_s^{-1}\Gamma_s$ in structural form ($B_s = \Pi_s$ in reduced form). Because A_s is unit lower triangular, $L_{s,h}$ is the Cholesky factor of the conditional covariance at no extra cost, and $y_{T+h} \mid \theta_s, y_{s,T+1:T+h-1}, \lambda_{s,T+h} \sim N(m_{s,h}, L_{s,h}L_{s,h}')$ holds exactly. The analytic engine draws (B, Σ) from the NIW posterior and uses $L = \text{chol}(\Sigma)$, scaled by the future Lenza–Primiceri factor when that treatment is active (Lenza and Primiceri, 2022). Forecast draws are evenly spaced stored draws of each chain, and their shocks are keyed by (forecast seed, chain, kept-draw index), so a forecast computed inside the sampling loop equals the one computed afterwards from the stored draws to 10^{-12}. Impulse responses use $\Phi_0 = I$, $\Phi_j = \sum_{l=1}^{\min(j,p)} A_l \Phi_{j-l}$ with A_l the reduced-form lag matrices; unit-shock responses $\Phi_j A^{-1}$ are date-free, while one-standard-deviation responses depend on the date and use the persistent variance by default; the forecast error variance decomposition is computed from the responses.

6.2 The predictive contract

A `ForecastResult` wraps an `xarray` dataset (Hoyer and Hamman, 2017) with the fields `paths` (joint paths on the transformed scale), `paths_level` (their pathwise inverse to levels, never applied to quantiles), `cond_mean` ($m_{s,h}$) and `cond_chol` ($L_{s,h}$), indexed by chain, draw, horizon and variable, and attributes for the origin, model, engine, spec and spec hash, transformation codes, seed, convergence flag and warnings. MCMC forecasts also record the variable ordering and sampler settings; analytic forecasts carry the ordering only as the order of the `variable` coordinate. The contract has three invariants: $L_{s,h}^{-1}(\texttt{paths} - \texttt{cond_mean})$ is iid $N(0, I)$; each draw is one joint path across horizons and variables; and stacking chains and draws is chain-major (Table C.2 in Appendix C). They are tested for every registered benchmark and analytic-engine model and for 12 MCMC compositions, eight structural and four reduced-form. Reduced form × SV(t), SV(outliers, t) and SV(rw) are not among them; the last is the primary arm of the replication in Section 11. [**TODO:** TODO 3: row 18, `reduced_sv_rw` (reduced form × SV(rw)), and a `reduced-SV(rw)` contract fit were committed in 7f410dc, after the acceptance commit c823d20; the SV(rw, t) variant is still not covered. If the release commit includes it, update this sentence (13 compositions) and the coverage statements of Sections 8.10, 11.8 and 12.] Accelerator variables may ride along and are never contract fields: MCMC forecasts attach a factored form of $L_{s,h}$, the transitory parameters and, since D-34, each draw's lag matrices (`lag_coefs`).

6.3 The Rao–Blackwellised log predictive density

The default estimator (SPEC §7) is the mixture of the conditional Gaussian densities,

$$\log \hat{p}_h(y) = \operatorname{logsumexp}_s \log N\big(y;\, m_{s,h},\, L_{s,h}L_{s,h}'\big) - \log S, \tag{15}$$

so no kernel density estimate is needed; mixture log scores of this kind are standard (Geweke and Amisano, 2010; Krüger et al., 2021). A leading block $\{1, \dots, j\}$ of the ordering uses $L_{s,h}[1{:}j, 1{:}j]$, and any other subset factors the selected rows of $L_{s,h}$ by QR, so marginal scores are exact. With weights $w_s = \exp(\log p_s - \max_r \log p_r)$, the reported Monte Carlo standard error is the delta-method value

$$\widehat{\text{se}}\big(\log \hat{p}_h\big) = \frac{\text{sd}(w)}{\sqrt{S}\,\bar{w}} \times \sqrt{S/\widehat{\text{ESS}}(w)}, \tag{16}$$

with the sample standard deviation; the second factor, with the ESS of the weights over chains and draws, applies to MCMC forecasts only and is 1 when that ESS is undefined. An opt-in option (`rb_transitory=True`) also integrates the outlier and t scales of $T+h$ out analytically.

6.4 The $h > 1$ tail problem and the path-integrated estimator

Failure. For $h > 1$ the moments in (15) are conditioned on the simulated path $y_{s,T+1:T+h-1}$ of draw s. For an outcome far in the predictive's tail, only draws whose paths happened to wander towards it receive weight, and in the limit one draw carries all of it; the estimate is then biased low, and (16) does not show it. The failure surfaced in the replication of Section 11: for its Constant arm, unemployment, $h = 8$, origin 2008Q2, a review computation on the replication's quick run, recorded in the repository notes, found -93.6 with 1000 forecast draws, -46 to -63 with 8000 and -36 with 64,000, against -19.1 for the path-integrated estimator below on the same 1000 draws, while the MCSE stayed near 1 nat. The value -19.1 is free of the path-conditioning error only; it is itself a Monte Carlo estimate over the same 1000 draws. A synthetic case reproduces the failure: for a persistent VAR(2) with constant volatility, $S = 200$ draws of one parameter value, $h = 8$ and an outcome 6 standard deviations out, the conditioned estimator gives -35.2 (MCSE 1.0) against an exact -22.3. [**TODO:** TODO 14 (keep out of the text until Proposition B.4 in Appendix B.4 is confirmed and unit-tested; risk 6). Observation: for $w \geq 0$, $\sum_s w_s^2 \leq (\sum_s w_s)^2$ gives $\mathrm{sd}(w)^2 \leq S\bar{w}^2$, so the iid factor of (16) is at most 1 nat whatever the error, with equality iff one draw carries all the weight ($\sqrt{(S-1)/S}$ with the population sd). This would explain the saturation near 1 (0.997 above) and the replication script's RB-degenerate flag at 0.9 (`RB_CEILING`, `benchmarks/clark_ravazzolo.py` l.355 at c823d20); the ESS factor can push MCMC values above 1. Lesson: an MCSE is a model of the error, not a certificate; inspect the weight ESS (Kish, 1965).]

Fix (D-34, `lpd_method="integrated"`). Given θ_s and the drawn scale path $\lambda_{s,T+1:T+h}$, the intermediate outcomes can be integrated out. With the simulated shocks $u_{s,k} = y_{s,T+k} - m_{s,k}$ and the moving-average coefficients $\Phi_{s,j}$ of draw s, $y_{T+h} = \tilde{m}_{s,h} + \sum_{k=1}^{h} \Phi_{s,h-k} u_{s,k}$ with independent $u_{s,k} \sim N(0, L_{s,k} L_{s,k}')$, so

$$
\begin{gathered}
y_{T+h} \mid \theta_s, \lambda_{s,T+1:T+h} \sim N\big(\tilde{m}_{s,h},\, M_{s,h} M_{s,h}'\big), \qquad \tilde{m}_{s,h} = m_{s,h} - \sum_{k=1}^{h-1} \Phi_{s,h-k}\, u_{s,k}, \\
M_{s,h} = \big[\Phi_{s,h-1} L_{s,1} \mid \Phi_{s,h-2} L_{s,2} \mid \cdots \mid \Phi_{s,0} L_{s,h}\big] \quad (n \times hn),
\end{gathered}
\tag{17}
$$

where $\tilde{m}_{s,h}$ is the zero-shock mean (deterministic terms cancel). The estimator uses these moments in (15) and the MCSE (16); subset factors come from the QR decomposition of $M_{s,h}[\mathrm{idx}, :]'$, so $M_{s,h} M_{s,h}'$ is never formed, and no additional random numbers are drawn. The weights no longer depend on where a draw's simulated path went. At $h = 1$ the two estimators coincide and the conditioned code runs, bitwise. With one parameter draw and constant volatility the estimator is the exact predictive density: it matches a companion-form oracle at $h = 2, \ldots, 8$ for joint, leading and non-leading subsets and 6-s.d. outcomes within the test tolerance of 10^{-10} (the commit message of b9c66c0 records 1.8×10^{-14}), and it returns the exact -22.3 in the synthetic case. It is equivariant to rescaling variables by $10^{\pm 6}$, and on real MCMC fits each draw's density matches moments obtained by `jax.jacfwd` of the forecast core to 10^{-10}. Figure 2 compares the two estimators.

[**TODO:** Figure 2, right panel: needs `docs/paper/data/rb_tail_cell.json` from `rb_tail_cell.py` (TODO 6); until then its numbers are recorded only (NOTES 8b).]

Residual error. The integrated estimator is still an average over draws whose weights concentrate in the tails. A review of the estimator, recorded in the repository notes and the pre-registration

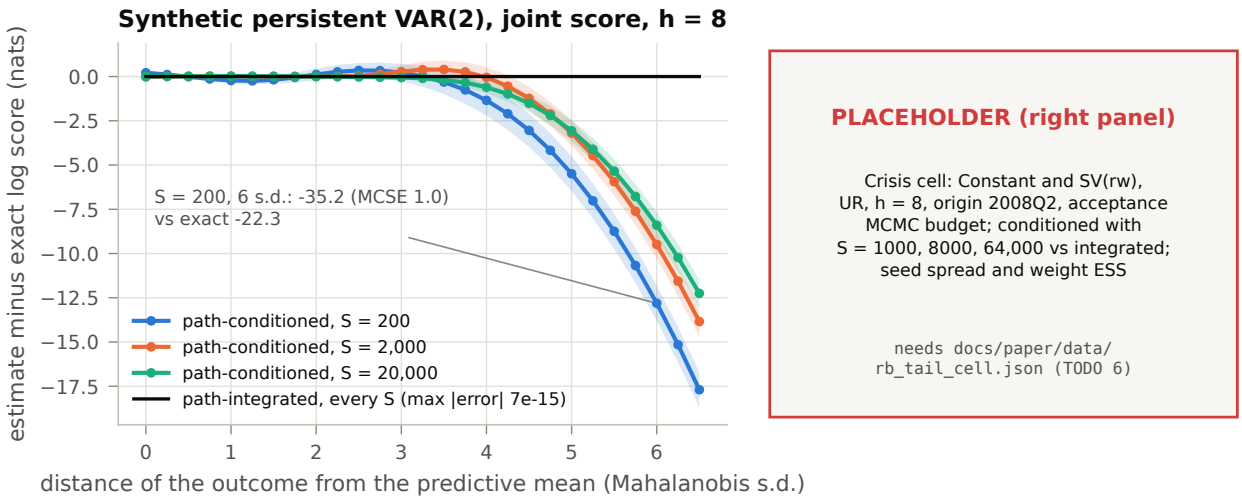


Figure 2: Path-conditioned (SPEC §7) against path-integrated (D-34) Rao–Blackwellised log score at $h > 1$. Left: synthetic persistent VAR(2) at $h = 8$ (one parameter draw, constant volatility); estimate minus the exact log score as the outcome moves from 0 to 6.5 (Mahalanobis) standard deviations from the predictive mean, for the conditioned estimator ($\pm$ its MCSE) with $S = 200$, 2000 and 20,000 and for the integrated one; the test's case (6 s.d., $S = 200$) is annotated. Right: Constant (and SV(rw)), unemployment, $h = 8$, origin 2008Q2, at the acceptance MCMC budget; conditioned with $S = 1000$, 8000 and 64,000 against integrated, with the spread across fit and forecast seeds and the weight ESS.

of Section 11, refitted the same cell at the acceptance budget ($4 \times 20{,}000$ draws): with 1000 forecast draws the Constant arm's weight ESS was 2–6, and across thinning offsets the estimate had a standard deviation of 0.75–1.06 nats and a downward bias of about 0.3–0.6 nats; with 8000 draws the standard deviation was 0.09–0.19 nats and the bias negligible. More forecast draws do not remove the MCMC error: 80,000-draw values still differed by about 1.3 nats across fit seeds. The pre-registered acceptance run therefore uses 8000 forecast draws.

Scope. The estimator is opt-in; the SPEC default is unchanged. It needs `lag_coefs`, which MCMC forecasts of linear or zero means carry; analytic, benchmark and ordering-pooled forecasts raise at $h > 1$, and so does combining it with `rb_transitory` at $h > 1$, which would require integrating the transitory scales of $T + 1, \ldots, T + h - 1$ jointly. [**TODO:** Risk 4: if D-34 is not on the release commit, cut the Fix and Residual-error paragraphs and the integrated series of Figure 2.]

6.5 Scores

All scores are computed per origin, horizon and model on the transformed scale: the joint and per-variable log score with the chosen estimator (an integrated run also stores the conditioned values from the same forecast); the fair CRPS in its probability-weighted-moment form, computed with scoringrules (Zanetta and Allen, 2024); quantile-weighted CRPS with centre and tail weights (Gneiting and Ranjan, 2011) and optional threshold-weighted CRPS; energy (Gneiting and Raftery, 2007) and variogram ($p = 0.5$; Scheuerer and Hamill, 2015) scores on a subsample of at most 1000 draws; squared and absolute errors; and PIT values with 50% and 90% coverage. The log score, which is strictly proper (Gneiting and Raftery, 2007), is the primary score.

6.6 Tests of equal predictive ability

For loss differences $d_t = \ell_{M,t} - \ell_{B,t}$ over P common origins (for the log score, $\ell = -\log \hat{p}$), the Diebold–Mariano statistic (Diebold and Mariano, 1995) uses a rectangular-kernel long-run variance with $h-1$ autocovariances $\hat{\gamma}_k$ and the small-sample correction of Harvey et al. (1997),

$$\mathrm{DM}_{\mathrm{HLN}} = \frac{\bar{d}}{\sqrt{\hat{\omega}/P}} \sqrt{\frac{P + 1 - 2h + h(h-1)/P}{P}}, \qquad \hat{\omega} = \hat{\gamma}_0 + 2\sum_{k=1}^{h-1} \hat{\gamma}_k, \tag{18}$$

compared with Student-t_{P-1} quantiles; Bartlett (Newey–West) weights (Newey and West, 1987) are used only if $\hat{\omega}$ is not positive. Following Giacomini and White (2006), the package treats the p-values as formally valid only with rolling or fixed windows; with expanding windows and nested models the statistic is not asymptotically standard (Clark and McCracken, 2001), and the p-values are labelled descriptive. The test is DM with the HLN correction applied to log-score losses; Amisano and Giacomini (2007) give the analogous weighted likelihood ratio test for density forecasts. The second route is the model confidence set (Hansen et al., 2011): the $T_{\max}$ statistic on the joint log-score losses, with a circular block bootstrap (block length $\max(1, \mathrm{round}(P^{1/3}))$, 2000 replications). Two consequences follow. Under expanding windows a "better" verdict can only come through the MCS route (Section 6.8), so results should report the route. And the comparison table prints two-sided DM p-values while the verdict uses one-sided ones (0.029 against 0.014 in Listing 1).

6.7 Backtests

`backtest()` runs the origins from a start to an end date, every `step` periods (or a given list of origins), with an `Expanding` window (tests descriptive) or a `Rolling` window (tests valid). At each origin it refits the model on the data available at that origin, cut to the window (`window.select(data.upto(origin))`), so every data-dependent prior quantity is recomputed from them (R9). Each origin has its own forecast generator and, for MCMC models, its own fit seed, both derived from the backtest seed and the origin; a data salt keeps chains independent across windows and orderings. Scores are cached per origin under a key of spec, sample, outcomes, origin and settings, with tags for the MCMC seeding and a non-default estimator, so runs resume and adding a model fits only that model. A pool of spawned workers reproduces the serial results to 10^{-12}. Warm starts are opt-in and require strictly increasing origins.

6.8 `compare()` and the verdict

`compare()` backtests every model on identical origins, windows and outcomes and evaluates every statistic on the origins where all models have a finite joint log score. For a model M and a reference B at horizon h, let ΔLPD be the summed difference of their log scores over those origins, $\mathcal{D}(M,B)$ the set of declared axes (mean, prior, volatility, …) on which their axis labels differ, p_{DM} the one-sided p-value of (18) and $\mathcal{M}_{25\%}$ the set of models whose MCS p-value is at least 0.25 (the model confidence set at $\alpha = 0.25$). The default rule is

$$\boxed{\begin{aligned} \text{better} \iff\ & \big|\mathcal{D}(M,B)\big| = 1 \ \wedge\ \Delta\mathrm{LPD} > 0 \ \wedge\ |\Delta\mathrm{LPD}| > 3\,\mathrm{MCSE} \\ & \wedge\ \big[(\text{rolling} \wedge p_{\mathrm{DM}} < 0.10) \ \vee\ (M \in \mathcal{M}_{25\%} \wedge B \notin \mathcal{M}_{25\%})\big], \\ & \text{with MCSE} = \Big(\textstyle\sum_t \big(\widehat{\mathrm{se}}^2_{M,t} + \widehat{\mathrm{se}}^2_{B,t}\big)\Big)^{1/2}, \end{aligned}} \tag{19}$$

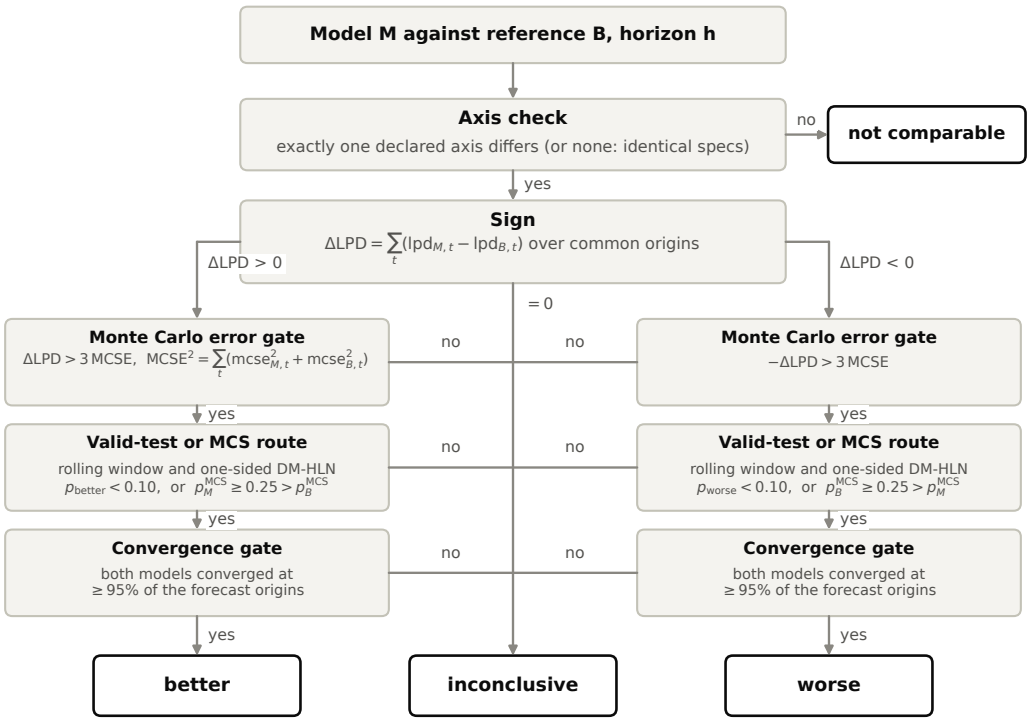


Figure 3: The verdict rule (19) as a decision flow: axis check, sign, Monte Carlo error gate, valid-test or MCS route, convergence gate. The outcomes are better, worse, inconclusive and not comparable.

"worse" is its mirror image, and anything else is "inconclusive". A convergence gate then forces "inconclusive" if either model converged at fewer than 95% of the origins. [**TODO:** No committed test calls `verdict(min_converged=...)` at d58c9ef; the gate is exercised only by Listing 2 and the replication script. Add a unit test.] A fit is converged when no chain raised an error, there are at least two chains, any empirical-Bayes step converged, and every primary quantity (log-likelihood, own-first-lag coefficients and the volatility parameters, including $h_{i,T}$ under SV) has a rank-normalised split $\hat{R}$ below 1.01 and a bulk ESS above 400 (Vehtari et al., 2021). Pairs that differ along more than one axis, or whose specs differ without a declared axis, are "not comparable". With `verdict(axis="vol")` each model is paired with the arm that has the baseline's value on that axis and agrees on all others, and `grid()` harmonises the arms so that this pairing is meaningful: in a volatility grid the Constant baseline becomes the MCMC Mode I model `Constant("ig")`, not the analytic NIW model, and Mode II is labelled a prior change. Figure 3 shows the rule as a decision flow. Each ingredient is standard; their combination into one automatic rule is the package's. The rule's size and power have not been studied, and the quadrature MCSE treats the two models' errors as independent although both use the same per-origin forecast seed (Section 12).

6.9 Ordering sensitivity

In general, a composition with a triangular factorisation depends on the variable ordering, through the prior or through the model class (Arias et al., 2023; Chan et al., 2024). The check (`rb.ordering_sensitivity()`) re-evaluates a model under all, Latin-square-balanced or random permutations of the variables: in prior mode it simulates the prior under each ordering without a fit, and in forecast or backtest mode it refits, recomputing every data-dependent quantity from the permuted

window. A metric such as the range of log scores over orderings is flagged only when it exceeds its threshold (0.05 nats per variable, 0.10 joint) and its signal-to-noise ratio against the noise floor $\sigma_{\text{MC}} = \max(\text{sd across seeds}, \text{MCSE})$ exceeds 3. Optionally, the check returns the equal-weight mixture over orderings, which is exactly order-invariant when all orderings are used. MCMC fits of compositions that are not order-invariant warn from $n = 4$ variables on (R8). Besides the analytic NIW engine, the only order-invariant composition in Phase 2 is the MCMC model reduced form × `Constant("iw_equivalent")`, whose implied Σ prior is inverse Wishart (Chan, 2022). That model is the Constant arm of the replication, where it serves as a negative control for the ordering check (Section 11; see Section 12 for how the check treated it in the first full run).

7 Using rai-bvar: a walk-through

This section shows the user interface on two short scripts. Both use the same simulated data, a stationary three-variable quarterly VAR(1) for output growth, inflation and an interest rate, stored as raw levels with FRED-style transformation codes, so they run without network access. Listing 1 uses the analytic engine and Listing 2 the MCMC engine; each is followed by its standard output, verbatim. The scripts and their outputs are in the paper's repository; the analytic output is deterministic given the seeds, and the MCMC output is bitwise reproducible only on the same platform and package versions (Section 4). [**TODO:** Length: with both listings and their outputs this section runs to about four pages in the assembled build against the two budgeted in OUTLINE §7. Options: move the two output blocks to Appendix C, or typeset each script and its output side by side.]

7.1 Data

`VARData.from_frame(raw, tcodes={...})` takes a data frame of raw levels indexed by dates or periods. Its column order is the recursive ordering used by every triangular model. Gaps in the index, duplicate columns, infinite values, logs of non-positive values and missing values inside the sample raise errors; leading and trailing incomplete periods are trimmed, with a warning except for the leading periods that the transformations themselves consume. `upto(origin)`, `tail(n)`, `select(variables)` and `realized(origin, h)` give the data available at an origin, rolling windows, reorderings and future outcomes on the transformed scale. The transformation codes 1–7 are those of FRED-MD and FRED-QD (McCracken and Ng, 2016), each with a pathwise inverse, and they set the own-first-lag prior mean (1 for levels and log levels, 0 for differences; Section 3). For real data, `rb.datasets.load("fred-qd", vintage="2026-09")` downloads a pinned vintage of FRED-QD (McCracken and Ng, 2021) on demand, checks its SHA-256 checksum and caches it; the files are not bundled with the package. Its `variables` argument takes a named set or a list of series (the order given is the ordering), and `tcodes` takes `"rai"`, `"official"` or a mapping. The named variable sets (`"small"`, `"medium"`, `"large"`) contain only series published by U.S. federal agencies, and interior publication gaps are interpolated (linearly, in logs for logged series) with a warning that lists every filled period (`fill_gaps=True`). [**TODO:** Q11/TODO 21: confirm the official FRED-QD 2026-09 transformation codes against the `"rai"` codes before the paper states them.]

7.2 An analytic BVAR, a forecast and a comparison

Listing 1: `ex_a.py`: data, an analytic BVAR with SOC/SUR dummies and GLP empirical Bayes, a forecast, and a rolling-window comparison with its verdicts.

```
6  import numpy as np
7  import pandas as pd
```

```python

import rai_bvar as rb

# A tiny quarterly data set: a stationary VAR(1) in (GDP growth, inflation, rate),
# stored as raw levels (two log-levels, one rate) with FRED-style transformation codes.
rng = np.random.default_rng(0)
A = np.array([[0.4, 0.0, -0.001], [0.1, 0.6, 0.0], [15.0, 25.0, 0.85]])
mu = np.array([0.0075, 0.006, 4.0])
sd = np.array([0.007, 0.003, 0.25])
corr = np.array([[1.0, 0.2, 0.3], [0.2, 1.0, 0.2], [0.3, 0.2, 1.0]])
L = np.linalg.cholesky(corr * np.outer(sd, sd))
y, Y = mu.copy(), []
for _ in range(200 + 240):
    y = (np.eye(3) - A) @ mu + A @ y + L @ rng.standard_normal(3)
    Y.append(y)
Y = np.array(Y[200:])
raw = pd.DataFrame(
    {"gdp": 100 * np.exp(np.cumsum(Y[:, 0])), "prices": 50 * np.exp(np.cumsum(Y[:, 1])),
     "rate": Y[:, 2]},
    index=pd.period_range("1960Q1", periods=240, freq="Q"),
)
data = rb.VARData.from_frame(raw, tcodes={"gdp": 5, "prices": 5, "rate": 1})
print(data)
print("prior means of own first lag:", data.prior_means())

# Analytic engine: conjugate Minnesota with SOC/SUR dummies and GLP empirical Bayes.
bvar = rb.BVAR(mean=rb.Linear(prior=rb.Minnesota(soc=1.0, sur=1.0)), lags=2)
print(bvar.engine, "|", bvar.name)
fit = bvar.fit(data)
print(fit.summary())
fc = fit.forecast(horizon=4, draws=2000, seed=0)
print(fc)
print(fc.quantiles([0.05, 0.5, 0.95]).sel(horizon=1).round(4).to_pandas())
print("paths_level h=1 median:", fc.median(level=True).sel(horizon=1).round(3).values)

# Pseudo out-of-sample comparison on rolling windows (valid DM/AG p-values).
res = rb.compare(
    {"RW": rb.RandomWalk(), "OLS-VAR(2)": rb.OLSVAR(lags=2), "BVAR(2)": bvar},
    data,
    origins=rb.Origins(start="2010Q1", end="2019Q3"),
    window=rb.Rolling(size=120),
    horizons=[1, 4],
    baseline="RW",
)
pd.set_option("display.width", 200)
pd.set_option("display.max_columns", 20)
pd.set_option("display.max_colwidth", 200)
cols = ["origins", "dlpd", "dlpd_mcse", "crps_rel", "dm_pvalue", "tests_valid", "in_mcs_25", "converged"]
print(res.table()[cols].round(3))
v = res.verdict()
print(v[["model", "reference", "axis", "horizon", "verdict"]])
for r in v.itertuples():
    print(f"{r.model} vs {r.reference} h={r.horizon}: {r.reasons}")
```

Output of Listing 1 (stdout, verbatim; nothing was written to stderr)

```
VARData(variables=['gdp', 'prices', 'rate'], T=239, 1960Q2..2019Q4, freq='Q-DEC', vintage=None)
prior means of own first lag: {'gdp': 0.0, 'prices': 0.0, 'rate': 1.0}
analytic | BVAR(Minnesota(conjugate, eb, SOC+SUR), p=2)
            mode
lambda  0.268149
soc     1.119513
sur     1.176805
ForecastResult(origin=2019Q4, horizon=4, variables=['gdp', 'prices', 'rate'], samples=2000)
variable     gdp  prices    rate
quantile
0.05     -0.0053  0.0027  4.2629
```

```
0.50      0.0053  0.0073  4.6754
0.95      0.0158  0.0120  5.1097
paths_level h=1 median: [349.266 182.507   4.675]
                  origins    dlpd  dlpd_mcse  crps_rel  dm_pvalue  tests_valid  in_mcs_25  converged
model      horizon
RW         1             39   0.000      0.000     1.000        NaN         True      False        1.0
OLS-VAR(2) 1             39  14.447      0.102     0.907      0.029         True       True        1.0
BVAR(2)    1             39  15.368      0.094     0.893      0.020         True       True        1.0
RW         4             36   0.000      0.000     1.000        NaN         True      False        1.0
OLS-VAR(2) 4             36  33.191      0.884     0.860      0.001         True       True        1.0
BVAR(2)    4             36  35.051      0.946     0.842      0.002         True       True        1.0
        model reference                                          axis  horizon         verdict
0  OLS-VAR(2)        RW                                         model        1          better
1     BVAR(2)        RW  deterministic,lags,mean,model,outliers,prior,vol        1  not comparable
2  OLS-VAR(2)        RW                                         model        4          better
3     BVAR(2)        RW  deterministic,lags,mean,model,outliers,prior,vol        4  not comparable
OLS-VAR(2) vs RW h=1: ΔLPD=+14.45 (MC s.e. 0.10); DM p=0.014 (valid: rolling window); MCS25: model in,
    ↪ reference out
BVAR(2) vs RW h=1: differs along ['deterministic', 'lags', 'mean', 'model', 'outliers', 'prior', 'vol']; a
    ↪ verdict needs exactly one axis (SPEC R2)
OLS-VAR(2) vs RW h=4: ΔLPD=+33.19 (MC s.e. 0.88); DM p=0.000 (valid: rolling window); MCS25: model in,
    ↪ reference out
BVAR(2) vs RW h=4: differs along ['deterministic', 'lags', 'mean', 'model', 'outliers', 'prior', 'vol']; a
    ↪ verdict needs exactly one axis (SPEC R2)
```

Listing 1 runs on the analytic engine. The log difference (code 5) consumes the first period (239 of 240 raw rows remain), and the own-first-lag prior means follow the codes: 0, 0 and 1. The model of line 34 adds SOC and SUR dummy observations to the Minnesota prior and leaves the volatility at its default, so the composition resolves to the conjugate NIW model on the analytic engine, and empirical Bayes in the hierarchy of Giannone et al. (2015) sets the tightness (printed as `lambda`, κ in Section 3) and the SOC and SUR weights. The forecast is summarised on the transformed scale and, through the pathwise inverse, on the level scale.

The comparison runs 39 rolling-window origins at $h = 1$ and 36 at $h = 4$. The OLS VAR(2) is "better" than the random walk along the `model` axis at $h = 1$: its ΔLPD is +14.45 with an MCSE of 0.10, the one-sided DM–HLN p-value is 0.014 on a rolling window, and the MCS keeps the VAR and excludes the random walk. The table prints the two-sided p-value, 0.029, for the same pair. The comment in the listing says "DM/AG"; the test implemented is the Diebold–Mariano test with the Harvey–Leybourne–Newbold correction applied to log-score losses (Section 6.6). The BVAR has the larger ΔLPD (+15.37 at $h = 1$), yet its verdict against the random walk is "not comparable": the two differ along seven axes, and a verdict needs exactly one (R2). A comparison that credits the prior would pair the BVAR with an arm that differs from it only in the prior.

7.3 The same model family on the MCMC engine

Listing 2: `ex_b.py` (lines 37–75; lines 8–36 hold the imports, the `pandas` display options and the data block of Listing 1): stochastic volatility on the MCMC engine, summary, forecast LPD, impulse responses, and a volatility-axis comparison.

```
37 print(platform.platform(), "| rai-bvar", rb.__version__)
38
39 # MCMC engine: same Linear mean, stochastic volatility; form="auto" resolves to "structural".
40 mc = rb.MCMCSettings(chains=4, draws=1000, burn=1000)
41 sv = rb.BVAR(vol=rb.SV(), lags=2, mcmc=mc)
42 print(sv.engine, "|", sv.name, "|", sv.mean.prior.form, "|", sv.axes)
43 fit = sv.fit(data.upto("2019Q3"))
44 s = fit.summary()
45 print("prior mode:", s.attrs["prior_mode"], "| hyper:", s.attrs["hyper"], "| converged:", fit.converged)
46 print(s[["mean", "sd", "rhat", "ess_bulk"]].round(3))
47 fc = fit.forecast(horizon=4, draws=2000, seed=0)
```

```python
print(fc, "| engine attr:", fc.dataset.attrs["engine"])
y_obs = data.values[-1]  # 2019Q4, transformed scale
print("joint LPD h=1 and MCSE:", fc.log_pred_density(y_obs, horizon=1, return_mcse=True))
print("marginal LPD (rate):", fc.log_pred_density(y_obs[2:], horizon=1, variables=["rate"]))
irf = fit.irf(horizon=8, draws=500, seed=0)
print(irf)
print(irf.median().sel(horizon=[0, 4, 8], shock="rate").round(4).to_pandas())

# Volatility-axis comparison: both arms structural Mode I, differing only in vol (R2).
models = {
    "Linear x Constant": rb.BVAR(vol=rb.Constant("ig"), lags=2, mcmc=mc),
    "Linear x SV": sv,
}
for k, m in models.items():
    print(k, m.axes)
res = rb.compare(
    models, data,
    origins=rb.Origins(start="2017Q1", end="2019Q3"),
    window=rb.Rolling(size=120),
    horizons=[1],
    draws=1000,
    baseline="Linear x Constant",
)
print(res.table()[["origins", "dlpd", "dlpd_mcse", "dm_pvalue", "tests_valid", "in_mcs_25", "converged"]].
    ↪ round(3))
v = res.verdict(axis="vol")
print(v[["model", "reference", "axis", "horizon", "verdict"]])
for r in v.itertuples():
    print(r.reasons)
```

Output of Listing 2 (stdout, verbatim; MCMC numbers are platform-specific)

```
macOS-26.6.2-arm64-arm-64bit | rai-bvar 0.0.1
mcmc | BVAR(Minnesota(structural, eb), SV, p=2, mcmc=45b567) | structural | {'mean': 'Linear', 'prior': '
    ↪ Minnesota(structural, eb)', 'vol': 'SV', 'lags': '2', 'deterministic': 'c', 'outliers': 'none'}
prior mode: Mode I | hyper: {'hierarchy': 'eb', 'lambda': 0.24549046712778935, 'alpha': 2.0, 'cross': 1.0, '
    ↪ contemp': 1.0, 'var_const': 10000000.0, 'eb_method': 'conjugate_twin', 'eb_converged': True, '
    ↪ offset_rule': 'relative', 'c_rel': 1e-12} | converged: False
                    mean     sd   rhat  ess_bulk
quantity
loglik          1928.304  7.948  1.031   179.391
B_own1[gdp]        0.461  0.069  1.000  3010.071
B_own1[prices]     0.558  0.065  0.999  2735.739
B_own1[rate]       0.948  0.054  1.001  3537.035
mu[gdp]          -10.108  0.129  1.003   665.300
mu[prices]       -11.834  0.160  1.001   757.856
mu[rate]          -2.892  0.155  1.002   692.202
phi[gdp]           0.806  0.118  1.035   131.873
phi[prices]        0.831  0.111  1.034   125.015
phi[rate]          0.861  0.110  1.017   197.937
sigma[gdp]         0.156  0.112  1.036   134.415
sigma[prices]      0.200  0.113  1.042   109.911
sigma[rate]        0.088  0.062  1.008   362.438
h_T[gdp]         -10.226  0.356  1.009   756.713
h_T[prices]      -11.766  0.361  1.010  1264.266
h_T[rate]         -2.831  0.246  1.002   887.965
ForecastResult(origin=2019Q3, horizon=4, variables=['gdp', 'prices', 'rate'], samples=2000) | engine attr:
    ↪ mcmc
joint LPD h=1 and MCSE: (8.575356603927622, 0.007996504228586188)
marginal LPD (rate): -0.4000308979305567
IRFResult(horizon=8, variables=['gdp', 'prices', 'rate'], shock_size='sd', samples=500)
response     gdp  prices    rate
horizon
0         0.0000     0.0  0.2397
4        -0.0002     0.0  0.1230
8        -0.0001    -0.0  0.0456
Linear x Constant {'mean': 'Linear', 'prior': 'Minnesota(structural, eb)', 'vol': 'Constant', 'lags': '2', '
```

```
    ↪ deterministic': 'c', 'outliers': 'none'}
Linear x SV {'mean': 'Linear', 'prior': 'Minnesota(structural, eb)', 'vol': 'SV', 'lags': '2', 'deterministic':
    ↪  'c', 'outliers': 'none'}
                         origins   dlpd  dlpd_mcse  dm_pvalue  tests_valid  in_mcs_25  converged
model            horizon
Linear x Constant 1             11  0.000      0.000        NaN         True       True        1.0
Linear x SV       1             11 -0.107      0.102      0.842         True       True        0.0
        model          reference axis  horizon       verdict
0  Linear x SV  Linear x Constant  vol        1  inconclusiveΔ
LPD=-0.11 (MC s.e. 0.10, not precise enough); DM p=0.421 (valid: rolling window); MCS25: model in, reference
    ↪ in; converged at only 0% of origins
```

With the volatility set to `SV()`, the default mean, `Linear(prior=Minnesota())` (without the dummy observations of Listing 1), moves to the MCMC engine, in structural form with the Mode I prior; there is no engine argument to set. The Minnesota hyperparameters are an empirical-Bayes plug-in from the conjugate twin on the same window (`eb_method: conjugate_twin`), so the mean prior is the one every volatility arm on that window receives; for the SV model this is a plug-in value, not a type-II maximum-likelihood estimate. The summary reports $\hat{R}$ and bulk ESS for every primary quantity. On this homoskedastic DGP the SV parameters mix slowly: ϕ_i and σ_i have bulk ESS between 110 and 362 from 4000 kept draws and $\hat{R}$ up to 1.042, so the fit is flagged as not converged; the library's warning (on stderr, not shown) reports that 7 of the 16 primary quantities fail, which by the table are the six SV parameters ϕ_i, σ_i and the log-likelihood ($\hat{R}$ 1.031, bulk ESS 179). The forecast container and its log-density interface are those of the analytic case (R3); the `engine` attribute records which engine produced it, and the MCMC forecast also carries the ordering, sampler metadata and accelerator variables (Section 6.2). With the interest rate ordered last, a one-s.d. rate shock has no impact effect on output growth or inflation in the recursive identification.

The final comparison is along the volatility axis only: both arms are structural Mode I models with the same mean prior, and their axis labels differ only in `vol`. Over 11 rolling-window origins, SV has a ΔLPD of -0.11 with an MCSE of 0.10, too imprecise for any verdict, and the one-sided DM p-value is 0.421. The SV arm converged at none of the 11 origins, so the convergence gate forces "inconclusive" whatever the other evidence says. This is the intended behaviour: the gate refuses to certify a comparison with an unconverged fit. A longer chain is the first remedy to try; Section 11 shows how the gate decided a criterion of the replication and the fit-level diagnostic that set the MCMC budget before the acceptance run.

7.4 Inspecting fits and failures

An MCMC fit exposes its Gibbs plan: `fit.plan.describe()` prints the order of one sweep with each step's block and write set (Figure C.1 in Appendix C). It also exposes the diagnostics table, `summary()`, the `converged` flag, the list of `warnings` and the timing of each stage. A numerical failure inside a kernel reaches the user as `MCMCNumericalError`, which names the block, the equation, the chain and the sweep and carries the diagnostics of the deterministic replay (Section 4.4; Section 9 describes the failures found this way). [**TODO:** OUTLINE §7.4: a volatility-axis grid, `rb.grid(rb.BVAR, vol=[rb.Constant(), rb.SV(), rb.SV(outliers=True)])` with `verdict(axis="vol")`, is [TC]. Add it as a third script under `docs/paper/code/` and to `run_examples.py` only if it is run on the release commit.]

8 Validating the samplers

8.1 Principle

The specification of rai-bvar contains a merge rule: no sampler enters the code base until it passes Geweke and simulation-based calibration (SBC) tests. The rule answers the first threat of Section 1: a published equation-by-equation algorithm for large BVARs with stochastic volatility (Carriero et al., 2019) targeted a different posterior from the intended one (Bognanni, 2022) until it was corrected (Carriero et al., 2022). A passing suite is informative only if one knows what it would have caught, so we measure the power of ours by injecting known errors into the production samplers (Section 8.11). The harness ships as library code, `rai_bvar.validation` (4,230 lines).

8.2 Harness principles

Two implementations. Everything a test treats as the truth (prior samplers, simulators of observables and the log joint density) was written in a work package whose owner wrote no kernel code, with one exception: the reference for the SOC/SUR dummy rows was added later by the owners of the kernels it checks, from the model rather than from the kernels. No reference imports a kernel module; a subprocess test enforces this. The algebra differs too: the references are joint densities on the natural parameters, the kernels conditional formulas, and for the SOC/SUR dummy rows the reference uses the covariance form ($m \times m$ systems, Matheron's rule; Journel and Huijbregts, 1978; Chilès and Delfiner, 2012; Wilson et al., 2020) where the kernels use the precision form ($q \times q$ Gram updates). The mixture constants are transcribed twice, and a test requires agreement on every printed digit. A design review introduced the two-implementation rule after rejecting validation of each block with its own prior sampler (erratum E9, Appendix I). An oracle shares any convention we copied into it, whereas Geweke and SBC tests share only our prior and simulator, so the two are complementary.

Production code, demanding configurations. One step of a test chain is the production `prepare(data, prior)` followed by one production sweep, never a copy. Test configurations make every term matter: tight proper priors, $M \neq 0$ in (4), an outlier probability $\sim \text{Beta}(2, 18)$ so that outliers occur at $T = 20$, $\nu \in \{5, \ldots, 12\}$, T_{eff} never a multiple of the 32-row padding bucket, and hyperparameters that do not depend on y.

Failures fail. A non-finite test function gives $p = 0$ and is counted, and a failed SBC replication fails the test. The only filter allowed acts on the simulated data alone ($\tilde{y}$ finite, $\max|\tilde{y}| < 10^3$), which leaves ranks uniform because uniformity holds conditionally on the data (Modrák et al., 2025).

Fixed seeds and a sequential wrapper. Keys are derived from (seed, tier, CRC-32 of the composition name, chain), so results do not depend on how chains are vectorised. With fixed seeds a false alarm is deterministic, so marginal results go to the sequential procedure of Gandy and Scott (2021, Algorithm 3). With family level α, k stages and a size multiplier Δ,

$$\beta_1 = \alpha/k, \qquad \gamma_{\text{GS}} = \beta_1^{1/k}, \qquad \beta_{i+1} = \beta_i/\gamma_{\text{GS}}, \qquad \Pr(\text{reject} \mid H_0) \leq \sum_{i=1}^{k} \gamma_{\text{GS}}^{i-1}\beta_i = \alpha. \tag{20}$$

Stage i fails if its Bonferroni-adjusted p-value $q_i \leq \beta_i$, passes if $q_i > \gamma_{\text{GS}} + \beta_i$, and otherwise defers to a fresh, independent experiment, enlarged once by Δ. Per PR, $\alpha = 10^{-3}$, $k = 3$ and $\Delta = 4$ give

Table 3: Validation tiers: content, decision rule, cadence as designed and as run in continuous integration (the job named "nightly" runs weekly), and parametrised test IDs at commit 62d15f6, counted under the tier's pytest marker where one exists. Tiers U, B and A have no tier-wide marker (for B see note a); the G count includes the kernel-level Geweke tests.

Tier	Content	Decision rule	Cadence (designed / run)	IDs
U	density-ratio and Metropolis–Hastings identities; dense oracles with injected noise; closed forms; infrastructure	relative error below a fixed tolerance (10^{-8} for identities)	PR / PR	–
B	block-conditional SBC ($N = 200$, $L = 99$); one-block invariance ($N = 1000$, $L = 20$)	Bonferroni at $\alpha = 10^{-3}$; sequential wrapper	PR / kernel level PR, composition level weekly	43[a]
G	Geweke: G-small ($C = 1000$, $M_c = 100$, $M_1 = 2 \cdot 10^5$) and G-full ($C = 4000$, $M_c = 500$, $M_1 = 10^6$)	$\min_g p_g \leq \alpha/G$, $\alpha = 10^{-3}$ (small) or 10^{-2} (full); sequential wrapper	small PR, full nightly / small PR, full weekly	95
S	SBC, $N = 1000$, $L = 99$, two-pass thinning, ESS gates	γ test, Bonferroni at $\alpha = 10^{-2}$; no failed replication; ESS gates	nightly / weekly	22
A	analytic twins and exact oracles A1–A7	$\lvert z\rvert \leq \max(4, z_{\text{Bonf}})$ (A2–A5); log ML to 10^{-10} (A1)	PR: A1, A2, A5 ($n = 3$), A6, a small A7; nightly: A3, A4, the rest / the nightly part weekly	–
P	parity with independent implementations	quantile $\lvert z\rvert \leq \max(4, z_{1-0.01/(2m)})$; log ML rtol 10^{-8}	short PR, long nightly / short PR, long weekly	15[b]
M	18 injected mutants (Table 4)	caught iff the tier fails on the mutant and passes on the control	weekly / weekly	37[c]

[a]Composition-level tests in `tests/sbc/test_block_tier.py`. [b]Marker `parity_long`; `tests/parity` has 364 IDs in all. [c]Mutant × tier checks; the registry test runs on every PR.

$\gamma_{\text{GS}} = 0.0693$ and about 30% expected extra effort under the null. Holm's procedure (Holm, 1979) only lists the failing functions.

8.3 Tiers, cadence and suite size

Table 3 lists the seven tiers. The continuous-integration job named "nightly" runs weekly, so composition-level Tier B, G-full, SBC, the large analytic twins, long parity runs and the mutation suite run weekly; Tier U, G-small and the kernel-level Tier B run on every pull request (PR), on Linux for Python 3.12–3.14. Every `RuntimeWarning` is an error, as is any silent float32 truncation in the MCMC tests. At commit 62d15f6 the suite has 4,964 parametrised test IDs: 4,520 in the PR suite, 441 weekly and 3 benchmarks. [**TODO:** [REL] Refresh the counts here and in Table 3 from `docs/paper/data/release/collect.txt` on the release commit.]

8.4 Tier U: deterministic identities

For a block b with implemented conditional $q_b(\theta_b \mid \theta_{-b}, y)$ and two states x_0, x_1 that differ only in θ_b, the identity (Grosse and Duvenaud, 2014)

$$\log q_b(x_1) - \log q_b(x_0) = \log p(x_1, y) - \log p(x_0, y) \tag{21}$$

is checked against the independent log joint density. For a Metropolis–Hastings step, the computed log acceptance ratio before truncation must equal $\log \pi(x') + \log q(x \mid x') - \log \pi(x) - \log q(x' \mid x)$, including the mixture-correction weights (11), the combined accept/reject of the σ^2 step and the ASIS Jacobians. Both are asserted at $|\text{LHS} - \text{RHS}|/(1 + |\text{RHS}|) \leq 10^{-8}$. Dense oracles with injected noise check the linear algebra, at 10^{-10} to 10^{-13}. One tolerance lesson generalises: two exact samplers of the same Gaussian agree only to $O(\epsilon_{\text{mach}} \operatorname{cond}(K))$, with $\operatorname{cond}(K)$ the condition number of the Jacobi-scaled precision, which reaches 10^6 for Minnesota priors on level data. Path-against-path comparisons therefore use the envelope $2 \times 10^{-12} + 5\epsilon_{\text{mach}} \operatorname{cond}(K)(1 + t)$ for means (with t the largest posterior t-ratio) and $2 \times 10^{-12} + 5\epsilon_{\text{mach}} \operatorname{cond}(K)$ for covariances. Infrastructure checks cover float64 on every array, a JAX-free package import, typed keys with JAX's key-reuse checker, the invariances of Section 4.6 and the failure path.

8.5 Tier G: Geweke tests

Following Geweke (2004), the marginal-conditional simulator draws M_1 independent pairs $\theta \sim p(\theta)$, $y \sim p(y \mid \theta)$; the successive-conditional simulator runs C independent chains of M_c steps ($y \sim p(y \mid \theta)$, then one production sweep), each started at a prior draw with no burn-in. With $\bar{g}_{\text{MC}}$ and $\hat{\sigma}_g^2$ the marginal-conditional mean and variance of a test function g, and $\bar{g}_{\text{SC}}$ and s_C^2 the mean and variance of the C chain means,

$$z_g = \frac{\bar{g}_{\text{MC}} - \bar{g}_{\text{SC}}}{\sqrt{\hat{\sigma}_g^2/M_1 + s_C^2/C}}, \qquad p_g = 2\Pr\big(t_{C-1} > |z_g|\big), \tag{22}$$

and the test fails iff $\min_g p_g \leq \alpha/G$ over the G functions. Batch means within chains (Flegal and Jones, 2010) give a non-gating cross-check of the chains' long-run variance. A chain started at $\theta^{(0)} \sim p(\theta)$ is stationary from its first sweep under the null, so the size needs only that the sweep leave the posterior invariant; ergodicity affects power, and a burn-in would only lower it. [**TODO:** TODO 15: verify before attributing the opposite concern (Geweke z-scores need converged chains) to Talts et al. (2018); until then cite no one for it.] The t_{C-1} critical values matter at Bonferroni-sized tails: at $G = 100$ and $\alpha = 10^{-3}$ the normal approximation has actual size 1.64×10^{-3} for $C = 200$. Test priors follow the rule $r = \text{Var}_{\text{prior}}/\text{Var}_{\text{post}} \leq 5$, derived in the scalar conjugate model, where a successive-conditional sweep is an AR(1) step with integrated autocorrelation time $2r - 1$ (Appendix B); it is unverified for ϕ, σ, p and ν. Test functions must have finite variance: $\text{E}[y^2] = \infty$ under the SV test prior, so data functions are log-scale or bounded; $\log \lambda$ replaces λ^2, whose variance is infinite under an inverse-gamma prior with shape at most 4; and the Constant test prior is $\text{IG}(5, 4)$. The exact log-likelihood (2) is among the test functions.

G-small uses $C = 1000$, $M_c = 100$, $M_1 = 2 \times 10^5$, $\alpha = 10^{-3}$, and G-full $C = 4000$, $M_c = 500$, $M_1 = 10^6$, $\alpha = 10^{-2}$, at $n \leq 3$. Rows with the approximate (`exact=False`) SV sampler are recorded but do not gate. All 15 PR compositions pass G-small at commit 62d15f6 (Table D.1), with largest final-stage $|z|$ 3.03 against critical values of 4.10–4.50. In row 201 (structural Constant with dummy rows) stage 1 gave $q = 0.0012$ ($|z| = 4.21$) and the fresh, four times larger experiment passed with $q = 1.0$, deterministically on one machine for fixed code and seed (not a reproduction across machines). The smallest relative numerical efficiency per composition is 0.008–0.129, which lowers power, not size; in the four compositions examined it falls on intercepts, a data function and levels of h, not on ϕ or σ. G-full runs weekly. [**TODO:** [REL] Report G-full and the `exact=False` rows only from a committed `geweke_full.jsonl`; do not write that G-full passes on every row before then.] Exploring the whole prior, the Geweke simulator also reaches states far outside those that ordinary data produce; this is how a float64 weakness of a coefficient draw was found (Section 9.2).

8.6 Tier S: simulation-based calibration

SBC (Cook et al., 2006; Gelman, 2017; Talts et al., 2018; Modrák et al., 2025) checks that the rank of a prior draw $\tilde{\theta}$ among posterior draws given data $\tilde{y}$ simulated from it is uniform; a variant conditional on observed data (Säilynoja et al., 2025) is related work. Per replication, a pilot chain started at an independent prior draw gives IAT_i, the largest Geyer initial-monotone estimate (Geyer, 1992) over the quantities and their tail indicators; a fresh chain then keeps L draws every $t_i = \min(\lceil 2\,\mathrm{IAT}_i \rceil, t_{\max})$ sweeps. As t_i depends only on $\tilde{y}$ and pilot noise, ranks stay uniform. Uniformity is tested with the statistic of Säilynoja et al. (2022): for N ranks in $\{0, \ldots, L\}$, $z_i = i/(L+1)$ and R_i the number of ranks below i,

$$\gamma_{\mathrm{S}} = 2 \min_{1 \le i \le L} \min\bigl\{F_{\mathrm{Bin}}(R_i; N, z_i),\ 1 - F_{\mathrm{Bin}}(R_i - 1; N, z_i)\bigr\}, \tag{23}$$

whose exact null distribution follows from the forward recursion of Säilynoja et al. (2022) over the binomial increments of R_i; our implementation accumulates the mass that leaves the envelope at each step, so that far-tail p-values keep relative precision.

With $L = 99$ kept draws a per-replication effective sample size (ESS) of $0.8L$ cannot be certified, even for independent draws, so that ESS is predicted from the pilot IAT. A second gate, for a pilot that underestimates the IAT, requires the kept-draw ESS pooled over replications (from ECDF indicators, with Kish's effective number of replications; Kish, 1965) to reach $0.8L$. [**TODO:** TODO 25: the iid shortfall share (about 76%) and the pooled minimum ($0.836L$) are docstring-only; commit a test before quoting either.] The thinning cap is $t_{\max} = 200$; in runs recorded in the test module, a cap of 50 made the ESS rule fail on slowly mixing datasets of a t row and an outlier-t row while the γ test was clean. Besides the parameters, the ranked quantities include h at three dates, the total and per-equation log-likelihood, $\log p(\theta)$, and h_{T+1} and y_{T+1} drawn through the production forecast core. Such data-dependent quantities are needed: a "posterior" equal to the prior passes SBC on every data-free quantity (Modrák et al., 2025). [**TODO:** TODO 15: check the theorem number of this result in the journal version of Modrák et al. (2025) before citing it by number.] The configuration is $N = 1000$, $L = 99$, $\alpha = 10^{-2}$, for 22 compositions at $n \le 3$. Apart from the mutation suite's S check (Table 4) and the runs recorded in the test module, the only committed SBC result is for row 205 (structural `iw_equivalent` with dummy rows): minimum p 0.018. [**TODO:** [REL] Report the SBC rows from a committed `sbc.jsonl` (Table D.2).]

8.7 Tier B: block-level tests

Blocks that draw exactly from their conditional get block-conditional SBC ($N = 200$, $L = 99$): L independent draws of the block given $\tilde{\theta}_{-b}$, ranking its coordinates and the exact log-likelihood. Metropolis–Hastings, ASIS and Gibbs-scan blocks get one-block invariance: $N = 1000$ joint draws, $L = 20$ applications of the live block with $\tilde{y}$ fixed, and a two-sample Kolmogorov–Smirnov test per statistic against independent joint draws. Both use Bonferroni at $\alpha = 10^{-3}$ inside the sequential wrapper.

8.8 Tier A: analytic twins and exact oracles

Analytic twins and exact oracles compare the MCMC engine with closed forms, quadrature or exact draws, up to realistic sizes (Table D.3). A1 checks that the structural NIG log marginal likelihood of Chan (2022) equals the analytic engine's to 10^{-10}, with and without SOC/SUR rows. A2 and A3 compare MCMC structural Mode II × `Constant("iw")` with the analytic NIW posterior at $n = 3$ and at $n = 20$, $p = 4$, $T = 300$, a size at which Geweke and SBC tests would be too

expensive. A4 checks Chan's asymmetric conjugate prior against its closed form, A5 Mode I × Constant against quadrature, A6 the exact-h, ASIS and step-order kernels on toy posteriors, and A7 the reduced-form Constant sampler against a Gibbs sampler whose coefficient step is an exact system-wide draw. Each exact quantile x_q is tested in CDF form,

$$z = \frac{\hat{F}_{\mathrm{MCMC}}(x_q) - q}{\sqrt{\mathrm{MCSE}^2(\hat{F}) + q(1-q)/N_{\mathrm{ref}}}}, \tag{24}$$

with the MCSE from the indicator's own ESS and N_{ref} exact reference draws; it passes iff $|z| \leq \max(4, z_{\mathrm{Bonf}})$ at family level 10^{-2}, and it avoids the density estimate that made the order-statistic quantile MCSE heavy-tailed at $n = 20$. Negative controls show that the comparisons have power: A2 rejects a posterior with one extra degree of freedom, A5 an inverse-gamma shape off by one half, and A6 and A7 deliberately wrong kernels (the uncorrected non-centred move, the step order of Primiceri, 2005, the CCM-2019 step). Apart from the A6 toys, the twins cover constant volatility only.

8.9 Tier P: parity with independent implementations

Parity compares the 5, 25, 50, 75 and 95% quantiles of m quantities through

$$z = \frac{\hat{q}_{\mathrm{ours}} - \hat{q}_{\mathrm{ref}}}{\sqrt{\mathrm{mcse}^2_{\mathrm{ours}} + \mathrm{mcse}^2_{\mathrm{ref}}}}, \qquad \text{pass iff } |z| \leq \max\bigl(4, z_{1-0.01/(2m)}\bigr). \tag{25}$$

The analytic engine is compared with R BVAR 1.0.5 (Kuschnig and Vashold, 2021) and the Bank of England's Python bvar 0.3.5 (Labonne et al., 2026). The tests assert log marginal likelihoods to relative 10^{-8}; the recorded agreement is below 10^{-11} (BVAR, fixtures with exact marginal likelihoods) and at most 1.5×10^{-11} (bvar), with posterior moments to about 10^{-9}, empirical-Bayes modes to 10^{-3} and sampled quantiles within Monte Carlo error. On the MCMC side, stochvol 3.2.1 (Kastner, 2016; Hosszejni and Kastner, 2021) is compared with `ZeroMean` × `SV` at $n = 1$, main and default-prior settings on four datasets: largest $|z|$ 1.63–2.59 in the PR tier and 1.64–3.57 in long runs, against a bound of 4.00. Our standardised-t SV sampler is checked against an exact importance-sampling posterior on a four-observation problem (largest $|z|$ 1.83); a problem this small has an exact posterior, so the check needs no second sampler as its oracle. The planned reduced-form reference, bayesianVARs (Gruber et al., 2026), could not be built in our test environment and was not run. Its substitute, an R sampler that draws the reduced-form coefficients jointly from their exact system-wide conditional, gives largest $|z|$ of 2.76, 3.45 and 3.56 on three fixtures, against bounds of 4.05–4.51. The reduced-form parity criterion is met against this substitute, not against bayesianVARs. Our parity tier has no external VAR-level oracle for structural Mode I × SV, the default MCMC composition, nor an external oracle for exact-mode SV beyond reweighted single-series runs; these rest on Tiers U, B, G and S (details in Table D.4).

8.10 Coverage of the compositions the replication scores

The replication of Section 11 scores two compositions that are not registered validation rows. Its primary arm, reduced-form Minnesota × SV(rw), and the variant reduced form × SV(rw, t) have no Geweke or SBC row: at the commits cited in this paper the registry carries the random walk only in the structural row 14, and its reduced-form SV rows (8, 9, 10 and 13) use AR(1) log-volatilities. Neither is among the 12 MCMC compositions of the contract tests. Their correctness

currently rests on their validated parts: the CCCM step (the reduced-form rows 7–11, 13 and 203), the random-walk volatility block (row 14), the Student-t component (rows 4 and 10, with the standardised scale and ν on a grid; the replication's t variant fixes $\nu = 10$ and uses the unit scale, which only Tier U checks of the transitory block's closed forms and conditional draws cover), and the MCMC tests of the path-integrated estimator on reduced-form CCCM fits. The replication's baseline, reduced form × `Constant("iw_equivalent")`, is row 11, and its AR-SV(0.9) arm runs the code path of row 8 with a different prior on ϕ. [**TODO:** TODO 3: row 18, `reduced_sv_rw` (reduced form × SV(rw)), and a `reduced-SV(rw)` contract fit were committed in 7f410dc, after the acceptance commit c823d20; the SV(rw, t) variant is still not covered. Its results so far are recorded only in the commit message (under load); state them here only from the release records (TODO 7). If the release commit includes it, rewrite this paragraph.]

8.11 Tier M: the mutation suite

The 18 mutants M1–M18 (Table 4) are injected only through production seams: an overridable kernel internal, the volatility slot, the sampler's prior arrays, the simulator or the configuration. A tier catches a mutant only if it fails on the mutant and passes on the unmutated control, with common seeds; a per-PR test requires a confirmed catching tier for every mutant, and an uncaught mutant blocks the release. Tier U catches 14 mutants and Tier G all 18 (M7 with a coarse mixture table, M11 and M12 under the auxiliary-mixture sampler); Tier B catches the four that have a B check and Tier S the one with an S check, which makes 37 checks. At commit 62d15f6 all 37 catch their mutant with a passing control, in 177–182 s on an Apple M5 Max with a cold compilation cache. Recorded runs at earlier commits, M1–M16 and then M17–M18 on x86-64 Linux, also caught every mutant; statistics agree across machines to two or three significant figures, not bitwise (M2: 702 / 2.48 in the committed evidence, labelled M1 Pro; 705 / 2.48 on the M5 Max; 705 / 2.47 on x86-64). [**TODO:** TODO 8: fix the stale count (33/33) and the machine labels in the `mutants.py` evidence header and NOTES B.8 before the table is regenerated.] Figure D.1 shows each statistic against its threshold. Mutation testing is established (DeMillo et al., 1978; Jia and Harman, 2011), including for probabilistic programming systems (Dutta et al., 2018); we add curated algorithm-level mutants, some taken from published errors, and a release gate built on them. [**TODO:** TODO 11: prior-art search on mutation testing of MCMC samplers before any "to our knowledge" wording.]

The suite taught five lessons. *(1) Identity tests and joint tests are both needed.* The four mutants without a U check, M2, M3, M5 and M16, are the non-local errors: a step order (M2, the indicator placement of the original algorithm of Primiceri (2005), corrected by Del Negro and Primiceri (2015)), a simulator that draws fresh latents (M3), a prior mismatch between sampler and model (M5) and a model approximation (M16). Every conditional-formula error in the suite (M1, M4, M6–M15, M17, M18) is caught by a deterministic identity or closed form at one joint draw. *(2) Exactness changes what counts as a bug.* Exact-mode SV is exact for any mixture table and indicator law, because the correction (11) uses the same table; M11 and M12 are bugs only of the auxiliary-mixture sampler. The uncorrected move M7 is invisible with the 10-component table at our test sizes and caught once a one-component table is injected in mutant and control alike. The general device: degrade an approximation on purpose to test its exact correction. *(3) The log-likelihood is a test quantity.* Block SBC catches M1, the CCM-2019 step, through the rank of the exact log-likelihood, while one-block invariance of the whole CCCM scan did not in a recorded run that we have not repeated ($q = 0.29$); this is consistent with Modrák et al. (2025) on how test quantities shape sensitivity. *(4) Moves must preserve the measure.* M6 takes $|\sigma|$ without flipping $\tilde{h}$, and both U and G catch it; only the joint fold $(\sigma, \tilde{h}) \mapsto (-\sigma, -\tilde{h})$ leaves the non-centred posterior invariant (Frühwirth-Schnatter and Wagner, 2010). *(5) Wrong dummy-row weights are loud:* $|z|$ 23.1 for M17 and 99.7 for M18, against controls of 1.94 (dense eigen path) and 2.61.

Table 4: Mutation suite: 18 injected sampler bugs with injection seam, composition, confirmed catching tiers and statistics (mutant / control). All 37 mutant × tier checks catch their mutant while the control passes. U: error of the identity or closed form at one joint draw (tolerance 10^{-8}; 10^{-12} for M8 and M11, 10^{-4} for M12, 10^{-10} for M15). B and S: Bonferroni-adjusted q or minimum p. G: largest $|z|$ at G-small (critical values 4.10–4.40). Values reproduced on an Apple M5 Max at commit 62d15f6; they equal the committed evidence strings except M2 (702 / 2.48 in the evidence, labelled M1 Pro).

ID	Injected error	Seam / composition	Caught by	Mutant / control
M1	CCM-2019 triangular step instead of the corrected step	`cccm_core` / reduced Constant	U B G	U 1.70 / 1.6e−14; B p 2.8e−11 / 0.10; G 20.8 / 3.03
M2	stale mixture indicators (carried across sweeps, drawn after h)	vol slot / structural SV	G	G 705 / 2.48
M3	simulator draws fresh latents	simulator / structural SV	G	G 23.5 / 2.59
M4	τ conditional with shape $\nu/2$, not $(\nu+1)/2$	`tau_conditional` / structural SV-t	U G	U 0.163 / 5e−15; G 281 / 2.12
M5	$B_\sigma \times 2$ in the sampler only	sampler prior / structural SV	G	G 50.3 / 2.59
M6	non-centred $\lvert\sigma\rvert$ without flipping $\tilde{h}$	`nc_fold` / univariate SV	U G	U 1.94 / 0; G 7.4 / 2.34
M7	uncorrected non-centred move in exact mode	`nc_accept` with `COARSE1` / univariate SV	U G	U 2.87 / 0; G 77.6 / 2.28
M8	centring (4) dropped ($\gamma_i = \delta_i$)	`structural_gamma` / structural Constant	U G S	U 0.585 / 0; G 101 / 2.77; S q 1.6e−13 / pass[a]
M9	outlier pmf without $1/k$	`outlier_logits` / structural SV-outliers	U G	U 0.699 / 1e−15; G 1474 / 2.51
M10	$b_p + T$ instead of $b_p + T - N_1$ in the p update	`p_conditional` / structural SV-outliers	U G	U 0.120 / 1e−14; G 30.1 / 2.51
M11	indicator pmf without σ_j^{-1}	`indicator_logits` / auxiliary-mixture SV	U B G	U 0.112 / 6e−16; B q 4.9e−19 / 0.93; G 86.9 / 2.18
M12	mixture means shifted by -1.2704 twice	`mixture_table` / auxiliary-mixture SV	U B G	U 1.2704 / 8.3e−5; B q 2.5e−162 / 0.93; G 444 / 2.18
M13	IG shape with T_{pad} instead of T_{eff}	`constant_conditional` / structural Constant	U G	U 1.64 / 3e−15; G 429 / 2.77
M14	a-row regression on $+u_{<j}$ instead of $-u_{<j}$	`contemp_conditional` / reduced Constant	U G	U 14.9 / 9e−15; G 325 / 3.03
M15	ASIS map without the $\tilde{h}$ rescale	`nc_step` / univariate SV	U B G	U 0.457 / 6e−17; B q 1.8e−14 / 1.0; G 50.5 / 2.34
M16	offset $c = 0.1$ at $\lambda \approx 0.01$, approximate sampler	configuration / univariate SV	G	G 264 / 1.99
M17	Mode I SOC/SUR rows weighted by the current $1/\lambda_i$, not the fixed $1/\bar{\sigma}_i^2$	`linear_conditional` / structural Constant + dummies	U G	U 0.060 / 2.4e−15; G 23.1 / 1.94[b]
M18	SOC/SUR rows dropped from the CCCM II step	`cccm_core` / reduced Constant + dummies	U G	U 0.405 / 2.9e−14; G 99.7 / 2.61

[a] The control passes at stage 3 ($q = 0.0069, 0.065, 0.95$). [b] Control on the dense eigen path, passing at stage 2 after a stage-1 q of 0.0012; 2.44 on the `cholesky_const` path.

8.12 Runtime convergence diagnostics and the gate

The convergence flag that the gate of Section 6.8 reads is computed for every fit, with split $\hat{R}$ taken as the larger of its bulk and folded versions. At $n = 20$ the primary sets hold 41 (Constant), 101 (SV), 61 (random-walk SV) and 101 (SV with outliers and t) quantities; secondary sets never gate. The estimators follow Geyer (1992) and Vehtari et al. (2021); the NumPy implementation is tested against hard-coded ArviZ values (Kumar et al., 2019) at rtol 10^{-10} and against AR(1) chains with known autocorrelation times. The gate is strict for SV: a pilot study predicted that SV fits at the default budget would routinely fail through ϕ and σ, the production benchmark ($4 \times 10{,}000$ draws) has `converged = False` in all six runs, and in the first full Clark–Ravazzolo run (four chains of 5000 draws) the SV arms converged at 37–68% of the main-stage origins and SV(rw) at 57–82% across orderings (the acceptance run is in Section 11).

8.13 Cost

On one Apple M5 Max with a cold compilation cache and other jobs running, G-small for all PR compositions takes about 95–100 s, the mutation suite 177–182 s and the PR fast suite about 1,000 s. These are wall-clock times under load, not budgets.

9 Numerical robustness: what the validation and replication runs found

9.1 The pattern

Three failures surfaced in the Phase 2 validation and replication runs, none from a user report: two float64 failures, F1 and F2, at states that are legitimate under the model, found by a Geweke test with a heavy-tailed test prior and by a replication of 24 orderings at 106 origins, and F3, a bug in the failure-reporting path. In F1 and F2 the information was lost when the matrix was assembled, not when it was factored, so both fixes factor the data rather than the assembled matrix. Because failures are values and are never jittered (Section 4.4), both were reported rather than masked; Table E.1 summarises F1–F3.

9.2 F1: a Geweke chain in the tail of the test prior

The weekly G-full Geweke test (Geweke, 2004) of `reduced_sv` failed on chain 3214 of 4000 at sweep 194 with status 1003 (block `EQ_CONTEMP`, equation 2), with equation 0 at log-variances $h \approx 31$–37 and $|y| \approx 2 \times 10^8$. The contemporaneous precision of equation 2, $K_a = V_a^{-1} + Z^\top W Z$ (prior covariance V_a, regressors Z, inverse variances W), had exact eigenvalues 14.8 and 1.9×10^{17}; forming $Z^\top W Z$ lost the small one and the Cholesky pivot was zero, although the conditional is benign, with standard deviations 0.20 and 0.16. The fix draws $a = R^{-1}(Q^\top \tilde{r} + z)$ from a Householder QR factorisation $[V_a^{-1/2}; W^{1/2} Z] = QR$ with $\mathrm{diag}(R) > 0$, for the whitened right-hand side $\tilde{r}$ and injected noise z: R is the upper Cholesky factor of K_a, so the noise map is unchanged, but K_a is never formed; the factorisation works with the stacked matrix, whose condition number $\mathrm{cond}(K_a)^{1/2}$ is never squared. Tests cover agreement with the Cholesky draw to rounding, a synthetic extreme-scale state with $\mathrm{cond}(K_a) > 10^{15}$ checked against a 60-digit exact conditional, and a replay of chain 3214 in which the production step now succeeds. Narrowing the test prior was rejected, because it would have hidden a real float64 weakness. [**TODO:** TODO 23: re-run the chain-3214 replay on Linux.] QR

in place of the normal equations is textbook (Higham, 2002; Björck, 1996); only the application is ours.

9.3 F2: the random-walk SV draw at $\sigma \approx 5 \times 10^{-9}$

The first full Clark–Ravazzolo attempt (code at 8a622c5) aborted with status 2001 (block `EQ_VOL`, equation 0) in an SV(rw) reduced-form fit at origin 1987Q2, burn-in sweep 318, with GDP's volatility scale at $\sigma = 4.98 \times 10^{-9}$. The state is valid: the non-centred prior $\sigma \sim N(0, B_\sigma)$, equivalently $\sigma^2 \sim B_\sigma \chi^2_1$, has positive density at $\sigma = 0$, and the same sweep's non-centred move would have returned σ to 0.031. In the scaled AWOL system $\tilde{Q} = \sigma^2 Q = \mathrm{Lap}_w + \mathrm{diag}(\delta)$, the terms $\delta_t = \sigma^2 / v_{s_t} \approx 10^{-17}$ (mixture-component variances v_{s_t}, notation of Section 5.5) fell below the floating-point spacing at the Laplacian's diagonal entries (2), so the assembled matrix was the singular path Laplacian and the level pivot, $\approx \sigma^2(1/V_0 + \sum_t 1/v_{s_t}) \approx 10^{-15}$, came out zero. The excess-pivot recursion of Section 5.5 factors the same matrix without assembling it, with non-negative terms only, so nothing cancels. At a synthetic test state with $\sigma = 5 \times 10^{-9}$ and 88 periods, it reproduces the smallest pivot, $\log|\tilde{Q}|$ and h of a 60-digit reference to relative errors of 10^{-12} or less on an Apple M5 Max. At the same test state, not the production fit, the assembled path gives a pivot of 2.22×10^{-16} (exact 5.308×10^{-15}) and $h_0 = -73.90$ (exact -3.024); Table E.1 and Figure E.1 recompute both paths. [**TODO:** TODO 23: re-run the tiny-sigma tests on Linux and commit a test asserting the old-path values; until then nothing is said about whether pre-fix runs were affected.] The random-walk integrated likelihood had the same flaw at $\sigma = 5 \times 10^{-9}$, 17534.8 instead of -1650.8, and now shares the factorisation. We read the recursion as an instance of subtraction-free elimination for diagonally dominant M-matrices (Grassmann et al., 1985; Alfa et al., 2002; Ye, 2008) inside a Gaussian Markov random field sampler (Rue, 2001); we claim the application, not the algorithm. [**TODO:** TODO 11: prior-art check of the recursion against GTH, Alfa–Xue–Ye, Ye (2008) and GMRF samplers.] The fix removes the crash, not the slow mixing near $\sigma = 0$ (Sections 11.5 and 12).

9.4 F3: the replay of F2 crashed

The deterministic replay of F2 itself raised `TracerArrayConversionError`, because it ran the production CCCM step on host arrays, so F2's diagnostics were lost. Commit 93f2bcd fixes this and tests the replay with the production program instead of toy kernels.

9.5 SOC/SUR dummy rows on the MCMC engine

In Mode II the dummy rows are weighted by $1/\lambda_i$, so the λ_i step must count them; a hand-injected omission was caught by the analytic twin (Section 8.8) but not by the dense oracles of the coefficient step. [**TODO:** TODO 24: make the injection reproducible before quoting how many quantiles it moves.] For exact Geweke and SBC references, $\lambda_i \mid d$ is inverse gamma under Mode II, and the coefficients are then conditioned on the rows by Matheron's rule (Journel and Huijbregts, 1978; Chilès and Delfiner, 2012; Wilson et al., 2020). Under `iw_equivalent`, $p(d \mid \lambda) = N(r; 0, S(\lambda))$ with $S(\lambda) = S_0 + \lambda U U^\top$ is not of that form, but the base prior $\mathrm{IG}(c, C)$ is an exact rejection envelope: the acceptance probability $g(\lambda) = (|S_0|/|S(\lambda)|)^{1/2} \exp\{-\frac{1}{2}(q(\lambda) - q_\infty)\}$, with $q(\lambda) = r^\top S(\lambda)^{-1} r$ and q_∞ its limit as $\lambda \to \infty$, is at most one because $S(\lambda) \succeq S_0$ grows with λ (Appendix B). The λ draws pass a Kolmogorov–Smirnov test against quadrature, and row 205 passes Geweke (maximum $|z|$ 2.24 in G-small, 2.76 in G-full) and SBC (minimum p 0.018). The $O(q^2)$ eigen fast path survives the dense prior precision that the rows induce.

9.6 Notes for other implementations

The following generic checks describe no particular package. (i) Implementations that form a weighted Gram for a Gaussian conditional draw should check regressors whose scales differ by many orders of magnitude (F1). (ii) Implementations that solve the random-walk SV system in scaled form should check $\sigma \to 0$ (F2). (iii) Implementations of a non-centred sign fold should check that it flips $(\sigma, \tilde{h})$ jointly, the map under which the posterior is invariant (Frühwirth-Schnatter and Wagner, 2010), not $\sigma \mapsto |\sigma|$ alone (mutant M6, Section 8.11). (iv) Implementations that score multi-step forecasts conditionally on simulated paths should inspect the weight ESS for tail outcomes (Section 6.4).

9.7 Platform caveats

On the macOS development machine (NumPy and SciPy on Accelerate), `slogdet` and `det` warned of division by zero on well-conditioned positive-definite tridiagonals, so the test oracle now uses the Cholesky factor. A Phase 1 empirical-Bayes test of the analytic engine has also repeatedly crashed the interpreter there, for unknown reasons. We attribute neither to a particular library; Linux CI is the reference platform. [**TODO:** Dossier 05 Q9: run the n = 50 EB test on Linux and record the result.]

10 Performance

The performance target, BENCH-P2, is one realistic SV workload: structural Linear × SV with $n = 20$, $p = 4$ and $T = 300$ on a simulated stationary VAR(4) with SV, sampled with four chains of 1000 burn-in and 10,000 kept sweeps, fixed hyperparameters and one CPU device, with a target of 120 s. The measured quantity is the end-to-end wall time of `BVAR.fit()`, including compilation of the JAX program (Bradbury et al., 2018), storage and diagnostics; each fit runs in a fresh process, with an empty (cold) or a populated (warm) persistent compilation cache, three times each.

The only reference measurement of the production engine is the committed baseline, taken at commit deb3929 on an Apple M5 Max with desktop applications open, at a one-minute load average of 2.4–6.1 before the six runs. This deviates from the protocol, which asks for an idle machine with at least eight performance cores; the record lists six. The median wall time is 52.3 s cold and 49.6 s warm, of which compilation takes 3.46 s cold and 1.00 s when the executables are loaded from the cache (medians over the three cold and the three warm runs). One sweep of all four chains takes 3.90 ms, the minimum over three blocks of 1000 sweeps of the compiled burn-in executable, and peak resident memory is 3,043 MiB (medians over all six runs). The target is met, but the numbers belong to this machine, load and commit and are not portable. [**TODO:** TODO 9: re-measure BENCH-P2 on an idle machine on the release commit, and add an idle x86 run.]

Mixing is the binding constraint, not speed. The slowest quantities are the SV scales σ_i and persistences ϕ_i, with minimum bulk ESS per second of 10.2 and 11.2 over the warm-run wall time, against 403 for the own first-lag coefficients. Every run reports `converged = False`. The runs share a seed and give identical draws, so this is one outcome seen six times; according to the commit message, 4 of the 101 primary quantities miss the R-hat threshold of 1.01 (worst 1.016, a σ_i), while every group's minimum bulk ESS exceeds 400. Even at five times the default number of draws, this workload thus fails the convergence rule of Section 6.8; Section 12 discusses the budget.

Several configurations have no production measurement: the stretch target of 60 s on four CPU devices, the reduced-form CCCM sampler, SV with Student-t errors and with outliers plus t, and an idle x86 machine (the only x86 record is provisional and was taken under heavy load). Timings of design stand-ins and pilot prototypes, which ran different code on another machine, are

not quoted. Table E.2 gives the full benchmark record and Table E.3 the kernel microbenchmarks. [**TODO:** Dossier 03 §13.10: the baseline measured deb3929, and later commits change engine and component sources. A reading of the diffs suggests that none is on the BENCH-P2 path, but that is an argument, not a measurement; re-measuring on the release commit (TODO 9) settles it.]

11 Illustration: a pre-registered replication of Clark and Ravazzolo (2015)

11.1 Role and target

The replication is one of the project's Phase 2 exit criteria, and it is qualitative: the target is the ranking reported by Clark and Ravazzolo (2015), that VARs with stochastic volatility beat homoskedastic VARs in density forecasts, not their numbers cell by cell. It exercises the whole stack on real data: a pinned, checksummed download, the composition API, the look-ahead-free backtest, the scores, `compare()` with its convergence gate, and `ordering_sensitivity`. It illustrates the workflow; it does not test the econometric claim beyond these data and these models.

We compare with the numbers of the working paper, Clark and Ravazzolo (2012, Tables 1–3, United States, VAR block), which the script hard-codes for display; the tables of the published article were not accessed. [**TODO:** TODO 22: obtain the JAE tables and re-check, or keep this sentence; resolve N08's "every number re-read" (40) against the 56 numbers quoted.] Their VAR(4) has a Minnesota prior without cross-variable shrinkage (tightness 0.2, $1/l^2$ lag decay, zero prior means) and a normal-diffuse error covariance (Kadiyala and Karlsson, 1997). Their VAR-SV has random-walk log-volatilities, sampled with the single-move step of Jacquier et al. (1994); the variants we use are AR-SV with a nearly fixed autoregressive coefficient and SV with Student-t errors with fixed degrees of freedom d (Jacquier et al., 2004). They do not state the recursive ordering, so we run all 24 orderings, with the table order GDP, INFL, UR, TBILL as the identity ordering.

11.2 Data and models

FRED-QD and FRED-MD, vintage 2026-09 (McCracken and Ng, 2021, 2016), are downloaded on demand and checked against their SHA-256 checksums. GDP growth and inflation are $100\,\Delta \log$ of `GDPC1` and `GDPCTPI` as non-annualised quarterly rates (FRED-QD has no GDP deflator); `UNRATE` and `TB3MS` enter in levels. Estimation starts in 1965Q1, as in Clark (2011), because FRED-QD starts in 1959. The forecasts are pseudo real time on the final vintage, not on real-time vintages (Croushore and Stark, 2001). Table F.2 lists the series and transformations, and Appendix F every deviation from the original design. Two properties are asserted at run time. The quarterly `UNRATE` and `TB3MS` of FRED-QD must be within-quarter means of the monthly FRED-MD values (to 0.01); the largest difference was 3.3×10^{-5} in the first full run and is [ACC: `data_checks.aggregation.UNRATE.max_abs_diff`] and [ACC: `data_checks.aggregation.TB3MS.max_abs_diff`] in the acceptance run. And there is no look-ahead: at 1985Q1, 2007Q4 and 2011Q2 the resolved prior, its hyperparameters and the padded data are bitwise identical when the vintage itself is cut at the origin ([ACC: `data_checks.in_window.identical`]). This check runs for the primary arm only.

Every arm shares the mean block `Linear(prior=Minnesota(form='reduced', hierarchy='fixed', lambda_=0.2, alpha=2, cross=1, mean=0))`, $p = 4$: Clark and Ravazzolo's prior on the reduced form, sampled with the corrected triangular algorithm (Carriero et al., 2022). The volatility arms (Table F.3) are: the baseline `Constant('iw_equivalent')`, the prior of Chan (2022) under which Σ is inverse-Wishart and the reduced form is order invariant, in place of the normal-diffuse VAR; the primary arm SV(rw), Clark and Ravazzolo's VAR-SV; the library's AR(1) default

SV; SV(rw,t10), with ν fixed at their $d = 10$, so that, by construction, the library's grid-bound warning on ν fires at every origin; AR-SV(0.9), whose $(\phi + 1)/2 \sim \text{Beta}(71.25, 3.75)$ matches the mean and standard deviation of their $a_1 \sim N(0.9, 0.05^2)$; and a structural Mode I pair, `Constant('ig')` against SV(rw), as a robustness check. Apart from the ν prior of SV(rw,t10) and the ϕ prior of AR-SV(0.9), the SV priors are the library defaults (Section 3.4). Clark and Ravazzolo's IG$(1/2, 0.0175)$ prior on the random-walk innovation variance is not adopted, and no prior was tuned.

11.3 Evaluation design

Forecasts come from an expanding window at the 106 origins 1985Q1–2011Q2, of which the 92 up to 2007Q4 form the pre-crisis sample, for $h = 1, 2, 4, 8$ quarters ahead. Each origin is a separate MCMC fit with its own fit seed. Under an expanding window the Diebold–Mariano test with the Harvey–Leybourne–Newbold correction (Diebold and Mariano, 1995; Harvey et al., 1997) is not formally valid (Clark and McCracken, 2001; Giacomini and White, 2006), so its p-values are descriptive, and a "better" verdict can only come through the model confidence set (MCS; Hansen et al., 2011).

The primary score is the Rao–Blackwellised (RB) log predictive density of Section 6.3: path-conditioned at $h > 1$ in the first full run, path-integrated in the acceptance run; the two coincide at $h = 1$. The acceptance run also reports the path-conditioned score on the same draws, as a secondary number that never enters a rule. For parity we report Clark and Ravazzolo's Gaussian-approximation score (Adolfson et al., 2007), computed from the same draws' mean and covariance. It is $-\frac{1}{2}$ times the Dawid–Sebastiani score up to a constant (Dawid and Sebastiani, 1999), proper but not strictly proper (Gneiting and Raftery, 2007; Krüger et al., 2021), and we flag the cells in which its sign differs from that of the RB score. CRPS and RMSE ratios complete the set. ΔLPS is the mean per-origin log-score difference against `Constant`, in nats per quarter. Verdicts are reported with the convergence gate on, as `compare()` returns them, and off.

An origin is *RB-degenerate* at horizon h when the delta-method MCSE of the joint RB score of either arm is at least 0.9: the weights then sit on about one draw, and the MCSE saturates near one nat however far off the estimate is. [**TODO:** TODO 14: cite the MCSE weight bound of Appendix B.4 here only after it is derived and unit-tested.] The flag is descriptive: flagged origins are kept, because dropping exactly the tail origins would bias the comparison. The ordering stage runs `ordering_sensitivity(mode='backtest', design='all')` with one noise replicate for SV(rw) and `Constant`, then a `compare()` verdict per ordering, each ordering with its own backtest seed. `Constant('iw_equivalent')` is order invariant, so its spread across orderings is Monte Carlo noise, which makes it a negative control.

11.4 Criteria

The replication plan states six criteria in words, and the script turns them into rules; both are in Table 5, the rules verbatim. The operationalisations are the project's own: "about 1" became ≤ 1.02, and "systematically" became at least 3 of 4 joint horizons or 8 of 16 marginal cells. Criterion 4's rule compares $h = 8$ with $h = 1$ only, whereas the wording says the gain "shrinks or turns negative"; the rule was not changed after the first run. Only criterion 6 uses the convergence gate, and it uses $h = 1$ scores only, which do not depend on the estimator.

11.5 The first full run (history)

The numbers of this subsection are history, not results. The first full run used 4×5000 kept draws (burn-in 1000), 1000 forecast draws and the path-conditioned LPD; an earlier attempt had aborted

Table 5: The six acceptance criteria: the qualitative wording of the replication plan, the rule as coded (verbatim from `criteria()`), and the outcome in the first full run (report at 551f181, path-conditioned LPD, 4×5000 draws) and in the pre-registered acceptance run (path-integrated LPD, $4 \times 20{,}000$ draws). The first-run column is history.

#	Wording (plan)	Rule as coded	First run	Acceptance
1	SV beats Constant in average LPD at $h = 1, 2$ for all four variables in both samples; the joint $h = 1$ gain is clearly positive	ΔLPS(SV(rw) $-$ Constant) > 0 at $h = 1, 2$ for all four variables in the full and pre-crisis samples, and the joint ΔLPD at $h = 1 > 0$ with one-sided DM/HLN $p < 0.10$ in both samples	met	[ACC: `criteria[id=1].met`]
2	CRPS ratio < 1 at $h \leq 2$ for all four variables	CRPS ratio SV(rw)/Constant < 1 at $h = 1, 2$ for all four variables (full sample)	met	[ACC: `criteria[id=2].met`]
3	RMSE ratios $\leq$ about 1 at short horizons	RMSE ratio SV(rw)/Constant ≤ 1.02 ('about 1') at $h = 1, 2$ for all four variables (full sample); CR's gains for GDP and the bill are 5–10%	met	[ACC: `criteria[id=3].met`]
4	Full sample: the $h = 8$ gain shrinks or turns negative for UR and the bill; pre-crisis it stays positive for GDP and inflation	full sample: ΔLPS(h=8) $<$ ΔLPS(h=1) for UR and TBILL; pre-crisis: ΔLPS(h=8) > 0 for GDP and INFL	not met	[ACC: `criteria[id=4].met`]
5	Neither SV-t nor AR-SV systematically beats RW or AR SV	neither SV(rw,t10) nor AR-SV(0.9) is better than both SV(rw) and SV (one-sided DM/HLN $p < 0.10$, positive ΔLPD) at $\geq$ 3 of 4 joint horizons or $\geq$ 8 of 16 marginal cells (full sample)	met	[ACC: `criteria[id=5].met`]
6	`compare()` says "better" (DM one-sided at 10% for at least GDP and the bill at $h = 1$) in all 24 orderings, or the report states which fail	in every one of the 24 orderings: `compare()` verdict SV(rw) vs Constant at $h = 1$ is 'better', and the one-sided DM/HLN p-value of the marginal log score is < 0.10 for GDP and TBILL at $h = 1$	not met	[ACC: `criteria[id=6].met`]

on the float64 failure of Section 9.3 (run history in Table F.1). It met criteria 1, 2, 3 and 5 and missed 4 and 6.

Criterion 4 failed on one cell. In the full sample the unemployment gain at $h = 8$, $+1.954$, exceeded the one at $h = 1$, $+0.159$. At three 2007–08 origins the per-origin difference was $+41.6$, $+41.4$ and $+40.4$ nats, the median difference was $+0.099$, and 12 origins were RB-degenerate at $h = 8$; the Gaussian approximation gave -0.499. Across arms and samples the RB and Gaussian ΔLPS differed in sign in 28 marginal and 8 joint cells, all at $h \geq 2$.

Criterion 6 failed on the convergence gate alone. SV(rw) was "better" at $h = 1$ in 0 of 24 orderings with the gate on and in 24 of 24 with it off. It converged at 57–82% of origins across orderings (`Constant`: 100%). In the report's example warning for each SV arm (main stage), a fit missed on one σ quantity out of 13 primary quantities (random-walk arms) or 21 (AR arms), with $\hat{R}$ 1.012–1.020 and bulk ESS 537–743. With the gate off, all four SV arms were "better" than `Constant` at $h = 1$ in the full sample through the MCS route (total ΔLPD $+85.7$ to $+87.5$ nats, MCSEs 1.00–1.06). Inflation gains were 1.6 to 3.2 times Clark and Ravazzolo's (our RB score

against their Gaussian one); final-vintage `GDPCTPI` and the `iw_equivalent` baseline are candidate causes that we did not test.

11.6 Diagnosis and the pre-registration

(a) The budget. SV(rw) was refitted at the main-stage fit seeds under several MCMC budgets, with fit-level diagnostics only and no forecasts or scores. In the identity ordering 72 of 106 origins converged at 4×5000, 79 with a burn-in of 5000, 92 with 8 chains, and 106 at $4 \times 20{,}000$ (largest $\hat{R}$ 1.0061, smallest bulk ESS 1763); in the two worst orderings the counts went from 71 and 63 to 106. All $520 + 424$ fits finished without error, and every fit that failed at the base budget failed on a σ quantity with $\hat{R} < 1.03$, which the pre-registration reads as $\hat{R}$ sampling noise at a bulk ESS near 600. [**TODO:** TODO 4: commit the diagnostic script and output to the main repository (source of Table F.7).] The first run holds a seed-to-seed check: its main stage and ordering 0 of its ordering stage fit the same models in the same ordering under different seeds, and SV(rw) converged at 72 against 69 origins (joint $h = 1$ ΔLPD $+0.809$ against $+0.838$). Near the threshold the converged count moves with the seed, so budgets are best sized with fit-only diagnostics before any score exists.

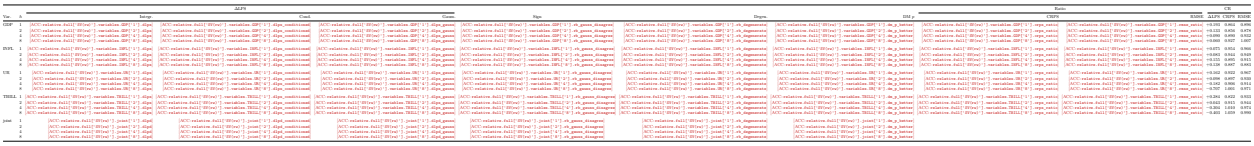
(b) The estimator. The criterion-4 cell was driven by the tail failure of the path-conditioned estimator (Section 6.4, Figure 2); the acceptance run uses the path-integrated estimator.

(c) Forecast draws. The path-integrated estimator is still an average over draws, and at crisis origins its weights concentrate. In the review of the estimator at the acceptance MCMC budget (Section 6.4), Constant's estimate at 2008Q2, unemployment $h = 8$, had a standard deviation of 0.75–1.06 nats across thinning offsets with 1000 forecast draws and 0.09–0.19 with 8000; the MCMC error across fit seeds, about 1.3 nats, does not shrink with more forecast draws.

What was registered. The pre-registration, committed in c823d20, changes exactly three settings: $4 \times 20{,}000$ kept draws (burn-in 1000) for every MCMC arm in every stage, the path-integrated LPD as the primary score, and 8000 forecast draws per origin. Data, models, priors, origins, horizons, seeds, the ordering stage, the coded criteria, the 95% gate and the descriptive RB-degenerate flag are unchanged. It is explicit about its limits: "These changes are motivated by that failure analysis, so the run is not blind to the previous results. What is pre-registered is that the settings below were fixed before any score of the new run existed, and that the criteria are evaluated on them as coded, whatever the outcome." It states in advance that criterion 4 may still fail on unemployment (in the review of the quick run, the estimate with corrected tail origins was about $+0.23$ at $h = 8$ against $+0.157$ at $h = 1$), that criterion 6 is expected to pass if the gate is met, and that no further run with other settings is made after the scores are seen, except to fix a bug.

Provenance. The run was launched from a detached worktree at c823d20 with a clean tree, and therefore after the pre-registration had been committed. [**TODO:** TODO 2: commit the acceptance JSON and report; archive the run log with its UTC start time.] The report labels itself the acceptance run only when the registered settings were used, every stage ran and the tracked files were unchanged (`acceptance_gaps()`); the fields that decide this are listed in Appendix F. Here the configuration is [ACC: `config.name`], the commit [ACC: `versions.git_commit`], and modified tracked files [ACC: `versions.git_dirty`].

Table 6: SV(rw) against `Constant`, full sample (106 origins), acceptance run. ΔLPS: mean per-origin log-score difference in nats per quarter, path-integrated RB estimator (primary), path-conditioned RB estimator on the same draws (secondary, never in a rule) and Gaussian approximation (parity metric). "Sign": RB and Gaussian ΔLPS differ in sign. "Degen.": RB-degenerate origins (joint MCSE ≥ 0.9 for either arm; the count is per horizon and the same for all variables). DM p: one-sided DM/HLN p-value for "SV(rw) is better", descriptive under the expanding window. CR: Clark and Ravazzolo's VAR-SV against VAR, working-paper Tables 1–3, Gaussian-approximation log score.



11.7 Results of the acceptance run

Reading rules. Which numbers may move between the two runs was written down before the acceptance numbers existed. [**TODO:** Commit these reading rules before the acceptance JSON is committed, so that their timing is checkable, or drop the clause "before the acceptance numbers existed".]

A. *Estimator:* every RB ΔLPS and ΔLPD at $h \geq 2$, the DM p-values at $h \geq 2$, the RB/Gaussian sign disagreements, the criterion-4 cells, the variant comparisons and the ordering spread at $h \geq 2$.

B. *Monte Carlo noise only:* all $h = 1$ LPD numbers, CRPS and RMSE ratios, Gaussian scores and $h = 1$ DM p-values; between the quick (4×2000) and the first full run, the full-sample $h = 1$ ΔLPS of SV(rw) moved by at most 0.003 per marginal cell (0.002 joint), and over all arms and both samples by at most 0.0044 per marginal cell (0.0122 joint).

C. *Budget:* converged shares, gate-on verdicts, criterion 6's gate-on count, the MC standard errors of the totals and the run time.

D. *Unchanged by design:* data checks, models, priors, origins, rules and fit seeds.

Criteria and headline. Table 5 gives each criterion's outcome; the details of criteria 4 and 6, including the secondary numbers that do not enter the rules, are [ACC: `criteria[id=4].detail`] and [ACC: `criteria[id=6].detail`]. Table 6 is the headline comparison. The joint $h = 1$ gain is [ACC: `relative.full['SV(rw)'].joint['1'].dlpd`] nats per quarter (descriptive one-sided DM p [ACC: `relative.full['SV(rw)'].joint['1'].dm_p_better`]). The unemployment cell at $h = 8$ is [ACC: `relative.full['SV(rw)'].variables.UR['8'].dlps`] path-integrated, [ACC: `relative.full['SV(rw)'].variables.UR['8'].dlps_conditioned`] path-conditioned and [ACC: `relative.full['SV(rw)'].variables.UR['8'].dlps_gauss`] under the Gaussian approximation, with [ACC: `relative.full['SV(rw)'].joint['8'].rb_degenerate`] RB-degenerate origins. Across arms and samples the RB and Gaussian ΔLPS differ in sign in [ACC: `count(relative[*][*].variables[*][*].rb_gauss_disagree = true)`] marginal and [ACC: `count(relative[*][*].joint[*].rb_gauss_disagree = true)`] joint cells.

Pre-crisis sample and the other arms. Criterion 4 also needs the pre-crisis gains of GDP and inflation at $h = 8$ to stay positive (Table F.4). The GDP cell was +0.108 (path-conditioned) and −0.004 (Gaussian) in the first run and is [ACC: `relative.pre_crisis['SV(rw)'].variables.GDP['8'].dlps`] (path-integrated), [ACC: `relative.pre_crisis['SV(rw)'].variables.GDP['8'].dlps_conditioned`]

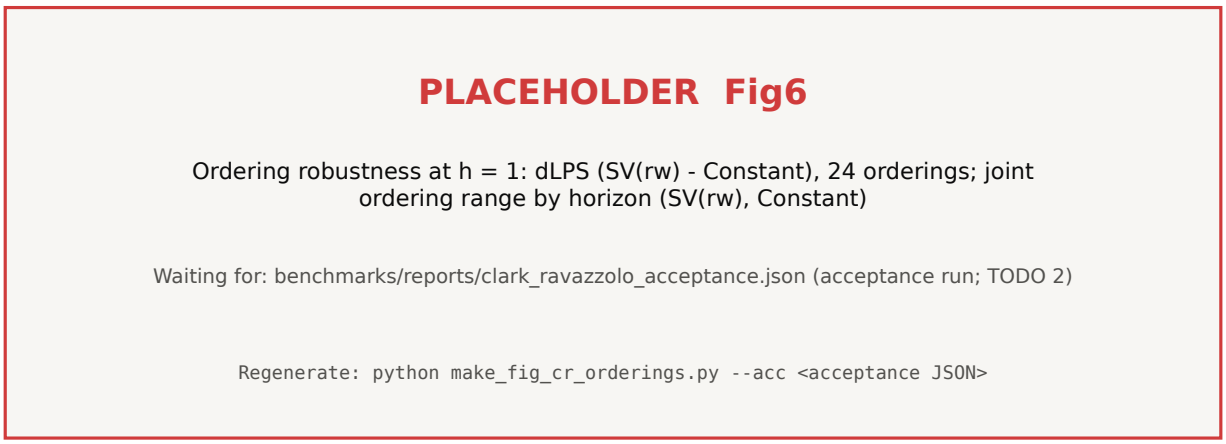


Figure 4: Ordering robustness, acceptance run. Left: ΔLPS (SV(rw) − `Constant`) at $h = 1$ per variable and joint, one point per ordering (24), with the identity ordering marked; each column gives the spread across orderings and SV(rw)'s signal-to-noise ratio, the quantity the materiality flag compares with 3. Right: joint ordering range by horizon for SV(rw) and for the order-invariant `Constant` arm (negative control), with flagged cells marked. At $h \geq 2$ the scores are path-integrated.

(path-conditioned) and [ACC: `relative.pre_crisis['SV(rw)'].variables.GDP['8'].dlps_gauss`] (Gaussian) here; inflation is [ACC: `relative.pre_crisis['SV(rw)'].variables.INFL['8'].dlps`]. The budget diagnostic covered SV(rw) only, in three orderings at the main-stage seeds, so the other SV arms may still miss the gate and stay "inconclusive" (Table F.5). Over the full sample the converged shares are [ACC: `converged.full['SV(rw)']`] for SV(rw), [ACC: `converged.full.SV`] for SV, [ACC: `converged.full['SV(rw,t10)']`] for SV(rw,t10), [ACC: `converged.full['AR-SV(0.9)']`] for AR-SV(0.9) and [ACC: `converged.full.Constant`] for `Constant`. At $h = 1$ the SV(rw) verdict is [ACC: `verdicts.full[model='SV(rw)',horizon=1,convergence_gate=true].verdict`] with the gate on and [ACC: `verdicts.full[model='SV(rw)',horizon=1,convergence_gate=false].verdict`] with it off. [**TODO:** After the JSON is in: state the route of every "better" verdict from `verdicts.full[*].reasons` and `model_in_mcs/reference_in_mcs`; under the expanding window it can only be the MCS.]

Ordering robustness. Figure 4 shows the $h = 1$ gain in each of the 24 orderings. In the first run the joint $h = 1$ range across orderings was 0.083 nats against a gain of 0.809, and the per-variable gains were 2.2–7.8 times their ranges. In the acceptance run the joint $h = 1$ range is [ACC: `orderings.results['SV(rw)'].joint_lpd_range['1']`] nats, the joint gain runs from [ACC: `min(orderings.per_ordering[*].dlpd_h1)`] to [ACC: `max(orderings.per_ordering[*].dlpd_h1)`], SV(rw) converged at [ACC: `min(orderings.per_ordering[*].converged_sv)`] to [ACC: `max(orderings.per_ordering[*].converged_sv)`] of origins, and the verdict is "better" in [ACC: `count(orderings.per_ordering[*].verdict_h1 = 'better')`] of 24 orderings with the gate on and [ACC: `count(orderings.per_ordering[*].verdict_h1_ungated = 'better')`] with it off. In the first run the order-invariant `Constant` arm was flagged "material" at joint $h = 2$ and unemployment $h = 8$, both path-conditioned cells with a one-replicate noise floor. Here the flag is [ACC: `orderings.results.Constant.material`] ([ACC: `orderings.results.Constant.summary[material=true]`]); because the arm is order invariant in theory, we read a flag on it as a limitation of the noise floor (Section 12), not as evidence of order dependence. Figure F.2 compares the converged shares per ordering of the two runs. [**TODO:** TODO 5: add the per-origin export (`docs/paper/data/cr_acceptance_per_origin.csv`) for Figures F.1 and F.3, or drop both figures.]

Interpretation. The pre-registration fixed how a failure of criterion 4 is reported: as a difference from Clark and Ravazzolo, attributed with evidence where there is any to the final-vintage data, the `iw_equivalent` baseline or the default SV priors, with no setting tuned in response. Reading rule B is the check on the $h = 1$ numbers: they should differ from the first run's by Monte Carlo noise only. The run-level warnings ([ACC: `warnings`]) are listed in Appendix F. [**TODO:** After the acceptance JSON is committed: two or three sentences of interpretation from the numbers above, within these branches only.]

11.8 Caveats

The illustration is limited by the following: final-vintage instead of real-time data; the working paper's numbers instead of the article's; Chan's `iw_equivalent` baseline instead of the normal-diffuse VAR, and the library's SV priors instead of Clark and Ravazzolo's; descriptive significance, with verdicts that come through the MCS, whose validity in this setting is open; operational thresholds of our own choosing; a pre-registration that is not blind; one backtest seed in the main stage; and a change of the primary estimator between the runs at $h > 1$, which is why every LPD number above names its estimator. Finally, the primary arm, reduced form × SV(rw), and its t variant are validated only through their parts: at the commit of the run they have no registered Geweke or SBC row and no contract-test fit (Section 8.10). [**TODO:** TODO 3: row 18, `reduced_sv_rw` (reduced form × SV(rw)), and a `reduced-SV(rw)` contract fit were committed in 7f410dc, after the acceptance commit c823d20; the SV(rw, t) variant is still not covered (SBC needs a per-row $t_{\max}$ of 400). The caveat stays true of the run's commit; once the release records exist, add that the arm has since been validated as a row of its own, here and in Sections 8.10 and 12.]

12 Limitations

Scope. The package implements Phases 0–2 of its roadmap. The mean axis is linear and the prior axis is the Minnesota family (conjugate, structural and reduced form, with dummy-observation priors and the GLP hierarchy); the volatility axis is the one that is populated. Global-local priors, common stochastic volatility and conditional forecasts (Phase 3), BART means, generalised impulse responses and approximate leave-future-out cross-validation (Phase 4; Bürkner et al., 2020), time-varying parameters, threshold and Markov switching, factor and order-invariant SV (Phase 5), and identification beyond the recursive scheme (Phase 6) are roadmap items, not features; for the methods see, for example, Huber and Rossini (2022), Clark et al. (2023), Gruber and Kastner (2025), Kastner (2019), Chan et al. (2024) and Bertsche and Braun (2022). That later components plug in without editing the sweep, runner or forecast modules is a design intention that no component has yet tested. The data layer rejects missing values inside the sample, has no mixed-frequency support, and ships no real-time vintages; it only provides the hook for them.

The verdict engine. The verdict rule is implemented and unit-tested, but its statistical properties are not established. The size and power study on simulated data-generating processes that the specification calls for has not been implemented, so we claim no false-positive rate for "better". What exists are tests of the Diebold–Mariano size and power, of MCS behaviour and of the verdict logic. The MCSE of a total ΔLPD adds the two models' per-origin MCSEs in quadrature, although both models' forecasts at an origin draw on random streams derived from the same backtest seed and origin; whether the factor of three is conservative is unknown. Under expanding windows a "better" verdict can only come through the MCS, whose validity in that setting is open (cf. Hansen et al., 2011). Verdicts are per horizon, with no rule that combines horizons. The Diebold–Mariano

long-run variance uses a rectangular kernel with $h-1$ autocovariances and falls back to Bartlett weights (Newey and West, 1987) only if that estimate is not positive, whereas the specification states a Newey–West variance. [**TODO:** TODO 10: implement the verdict-engine DGP size and power study, or keep it as future work here.]

The convergence gate. SV fits routinely miss the $\hat{R} < 1.01$ and bulk ESS > 400 targets. The prototype pilot predicted this, through ϕ and σ, at the default budget of 4×2000 draws, at which no SV arm of the quick Clark–Ravazzolo run converged at any origin. It persists at larger budgets: all six runs of the performance benchmark ($4 \times 10{,}000$ draws) report `converged = False`, and in the first full Clark–Ravazzolo run (4×5000) SV(rw) converged at 57–82% of origins across orderings. Near the threshold the converged count moves with the seed: 72 against 69 of 106 origins for the same models in the identity ordering, and 71 and 63 at the main-stage seeds against 63 and 60 at the ordering-stage seeds in the two worst orderings. The gate is strict by design, and the price is that users need larger budgets than the defaults: the acceptance run used $4 \times 20{,}000$ draws, and SV(rw) converged there at [ACC: `converged.full['SV(rw)']`] of origins. The default `MCMCSettings` (2000 draws) are unchanged. [**TODO:** TODO 33: decide the convergence-gate policy for SV at the default budget.]

Log scores. The path-conditioned estimator remains the default at $h > 1$. The path-integrated estimator is opt-in and covers MCMC forecasts of linear or zero means only; analytic, benchmark and pooled forecasts raise at $h > 1$. It removes the path-conditioning error but not the Monte Carlo error: it is still an average over the forecast's parameter and volatility-path draws, whose weights concentrate in the tails. In the review of the estimator at a crisis origin, SV(rw)'s 1000-draw estimate varied across forecast seeds by more than its delta-method MCSE, and the values with 80,000 forecast draws still differed by about 1.3 nats per arm across fit seeds, an MCMC component that more forecast draws do not remove. All scores are on the transformed (model) scale. The Monte Carlo noise of the subsampled energy and variogram scores is not reported, and there is no simulation-based calibration of y_{T+h} for $h > 1$.

The ordering diagnostic. The noise floor of `ordering_sensitivity` in the replication rests on one replicate (`noise_reps=1`), and in backtest mode it has no MCSE term. In the first full run it flagged the order-invariant `Constant` arm as material, at two $h > 1$ cells. In the acceptance run the flag is [ACC: `orderings.results.Constant.material`]. [**TODO:** TODO 26: decide `noise_reps` for future runs.]

Validation evidence. Geweke tests and SBC run at $n \leq 3$; the realistic size ($n = 20$, $p = 4$, $T = 300$) is covered only by the analytic twins, and only for constant-volatility compositions. Our parity tier has no external VAR-level oracle for structural Mode I $\times$ SV. At the commit of the acceptance run, the replication's primary arm, reduced form $\times$ SV(rw), and its Student-t variant have no registered Geweke or SBC row and are not among the 12 MCMC compositions of the contract tests; their correctness rests on their validated parts (Section 8.10). The composition-level G-full, SBC and Tier B runs and the weekly tier have no committed run record yet. The test-prior design rule $r \leq 5$ is a derivation for the scalar conjugate model, unverified for ϕ, σ, p and ν, and low relative numerical efficiency of some Geweke functions lowers power. The power of the suite is measured only against 18 curated mutants. Whether the 10-component mixture approximation is visible to G-full with unknown SV parameters is undetermined. The reduced-form parity check against bayesianVARs was replaced by an in-house oracle because the package could not be built

in the test environment, and all third-party parity holds for pinned versions (stochvol 3.2.1, BVAR 1.0.5, BoE bvar 0.3.5).

Numerics, performance and platforms. The formed-Gram draws that were not rewritten, among them the structural coefficient step and the CCCM II step, still accept any positive pivot, with no relative floor; whether reachable states expose them is unknown. [**TODO:** TODO 23: decide on a relative pivot floor; commit tests of the old-path failures.] The excess-pivot recursion removed the crash of the random-walk volatility draw near $\sigma = 0$, not the slow mixing there. The performance evidence comes from one Apple M5 Max that was not idle; there is no idle x86 measurement and no multi-device timing of the production engine. Bitwise reproducibility holds only within one platform and version set: invariance to chunking is tested on runner toys, and the toy chain-prefix test is bitwise on macOS and agrees only to rounding on x86 Linux. The macOS caveats of Section 9.7 apply; Linux CI is the reference platform.

Release state. The package is at version 0.0.1, classified as pre-alpha. [**TODO:** TODO 19: update the README status banner and quick start; release on PyPI with a version tag.]

13 Conclusion

A model comparison is only as credible as its weakest link: the sampler, convergence at every origin, the Monte Carlo error of the score, the variable ordering, the validity of the test, and choices made after the scores were seen. rai-bvar makes each of these links checkable, and the preceding sections show the evidence for each. The econometrics is not new; the contribution is the integration, the contracts between the components, and the evidence, with its measured power and its stated gaps, that the samplers target the intended posteriors. For an applied forecaster this means that a BVAR is composed from a mean, a volatility and a prior component, fitted by whichever engine the composition requires, scored through one predictive contract, and compared by a verdict engine that credits a gain only when the two models differ along exactly one axis, so that a volatility gain is never credited to the mean or the prior, and that withholds a verdict which the Monte Carlo error or the convergence of the fits cannot support. The validation code behind that evidence ships with the library. In the pre-registered replication of Clark and Ravazzolo (2015), the outcome of the acceptance run, criterion by criterion, is [ACC: `criteria[*].met`]. [**TODO:** One clause stating which criteria the acceptance run met and, if any failed, which, from `criteria[*].met`; no further outcome-dependent claim.]

A checklist. The lessons of building and validating the package, for forecasters and for implementers of the same algorithms, are these.

1. Write every kernel as a pure function of injected noise, and expose each exact draw's conditional and each Metropolis–Hastings step's log acceptance ratio (Section 4.4).

2. Write the reference prior samplers, simulators and joint densities independently of the kernels, and enforce the separation with an import test (Section 8.2).

3. Check an identity at a joint draw for every conditional and every Metropolis–Hastings step; run prior-started Geweke chains and SBC with data-dependent test quantities, including the exact log-likelihood (Section 8).

4. Measure the power of the suite with injected bugs, among them errors of published samplers, and degrade an approximation on purpose to test its exact correction (Section 8.11).

5. Treat numerical failures as values: never add jitter, and fix the kernel rather than the test prior (Section 9).

6. Check convergence at every origin and gate verdicts on it, and size the MCMC budget with fit-only diagnostics before any score exists (Sections 6.8 and 11.6).

7. Do not read the delta-method MCSE of a Rao–Blackwellised log score as a bound on its tail error; inspect the effective number of weights (Kish, 1965), and at $h > 1$ integrate the simulated path out and use enough forecast draws (Section 6.4).

8. Report RB and Gaussian-approximation scores side by side, the ordering spread next to the gain, and an order-invariant negative control (Sections 11.3 and 11.7).

9. Treat Diebold–Mariano p-values under expanding windows as descriptive, and state which route produced each verdict (Section 6.6).

10. Pre-register settings as code, say how blind the pre-registration is, and make the absence of look-ahead a run-time check (Sections 11.2 and 11.6).

Roadmap and availability. The roadmap continues by phase: global-local priors, common stochastic volatility and conditional forecasts; non-linear means; time-varying parameters, regime switching and factor or order-invariant volatility; and identification beyond the recursive scheme. We make no promise about results. The next priority is the size and power study of the verdict engine on simulated data (Section 12), followed by the Phase 3 prior axis, so that a two-axis ablation can be shown. rai-bvar is licensed under Apache-2.0. [**TODO:** TODO 20: repository URL, release tag and commit hash, Zenodo DOI for the release and the replication outputs, and the single command that rebuilds every table and figure (the `Makefile` in `docs/paper`).]

A Notation and conventions

The paper uses mathematical symbols where the code uses field names, and in two places the specification and the code use different symbols for the same object. Table A.1 fixes the convention used throughout and gives the code name of each symbol. Two choices matter for reading the code. First, the Minnesota overall tightness is written κ, although its field is `Minnesota.lambda_`, because $\lambda_{i,t}$ denotes the structural variance of equation i at time t. Second, the text numbers equations $i = 1, \dots, n$ in the recursive order, while the code indexes them from 0; where the position enters a prior formula as a number it is written ι ($\iota = i$ in the text's numbering, the code index plus one), so that, for example, the Constant prior of equation i is $\lambda_i \sim \mathrm{IG}(1 + \iota/2, \psi_i/2)$; the derivations of Appendix B mostly index equations by ι. The letter h is overloaded in the usual way: with subscripts, $h_{i,t} = \log \lambda_{i,t}^{\mathrm{pers}}$ is a log-volatility; without, $h \in \{1, 2, 4, 8\}$ is a forecast horizon.

All floating-point work is in float64, densities are evaluated in log space, and log scores are in nats. Data are stored as raw levels with FRED transformation codes (McCracken and Ng, 2016); each code has a forward map and an inverse that is applied to whole simulated paths, never to quantiles or means, because nonlinear transforms do not commute with expectations. Table A.2 lists them.

B Derivations

Each derivation below states the committed test that checks it, or says that none exists. None of the results is claimed as new except where a subsection says otherwise; the sources are credited in each.

B.1 The structural form of the normal–inverse-Wishart prior

Let $\Sigma \sim \mathrm{IW}(\Psi, d)$ with $\Psi = \mathrm{diag}(\psi)$, and $\mathrm{vec}(B) \mid \Sigma \sim \mathrm{N}(\mathrm{vec}\, M, \Sigma \otimes \Omega)$, the conjugate Minnesota prior of the analytic engine. Write $\Sigma^{-1} = A' D^{-1} A$ with A unit lower triangular and $D = \mathrm{diag}(\sigma_1^2, \dots, \sigma_n^2)$, and set $\Gamma = AB'$, so that $A\Sigma A' = D$.

Proposition B.1. *Independently over $\iota = 1, \dots, n$,*

$$\sigma_\iota^2 \sim \mathrm{IG}\Big(\frac{d - n + \iota}{2}, \frac{\psi_\iota}{2}\Big), \qquad a_\iota \mid \sigma_\iota^2 \sim \mathrm{N}\big(0, \sigma_\iota^2\, \mathrm{diag}(\psi_1^{-1}, \dots, \psi_{\iota-1}^{-1})\big), \qquad \gamma_\iota \mid a_\iota, \sigma_\iota^2 \sim \mathrm{N}(M\tilde{a}_\iota, \sigma_\iota^2 \Omega). \tag{B.1}$$

Proof sketch. The inverse-Wishart partition theorem (Gupta and Nagar, 2000) gives the first two statements when applied twice. The leading $\iota \times \iota$ block is $\Sigma_{1:\iota} \sim \mathrm{IW}(\Psi_{1:\iota}, d - n + \iota)$. Partition that block as $(\iota - 1, 1)$: the Schur complement $\sigma_\iota^2 = \Sigma_{\iota\iota \cdot <\iota}$ is $\mathrm{IW}_1(\psi_\iota, d - n + \iota)$, that is, the inverse gamma above, and the regression coefficient $a_\iota = -\Sigma_{<\iota}^{-1} \Sigma_{<\iota,\iota}$ is conditionally $\mathrm{N}(0, \sigma_\iota^2 \Psi_{<\iota}^{-1})$; the sign is irrelevant for a zero-mean law. The theorem also makes $(a_\iota, \sigma_\iota^2)$ independent of $\Sigma_{1:\iota-1}$, of which every earlier (a_j, σ_j^2) is a function, which gives independence across equations. For the third statement, $\gamma_\iota = B\tilde{a}_\iota$, so $\mathrm{E}[\gamma_\iota \mid A, D] = M\tilde{a}_\iota$ and $\mathrm{Cov}(B\tilde{a}_\iota, B\tilde{a}_j \mid \Sigma) = (\tilde{a}_\iota' \Sigma \tilde{a}_j)\, \Omega = \delta_{\iota j} \sigma_\iota^2 \Omega$ because $A\Sigma A' = D$. Finally, $(B, \Sigma) \leftrightarrow (A, D, \Gamma)$ is a bijection and the likelihoods coincide because $|A| = 1$, so the posteriors coincide as well. ∎

The direction "inverse-Wishart implies the structural normal–inverse-gamma form" is Corollary 1 of Chan (2022), and his Proposition 1 is the converse; the conditional centring $\mathrm{E}[\gamma_\iota \mid a_\iota] = M\tilde{a}_\iota$ has a precedent in the structural priors of Sims and Zha (2006), who attribute it to earlier work of theirs. [**TODO:** TODO 15: the 1998 paper itself was not obtained; check the precedent against it,

Table A.1: Notation. "Code" gives the field or variable name in rai-bvar; defaults are those of the committed specification classes.

Symbol	Meaning	Code (default)
n, p, n_d	variables, lags, deterministic terms ($n_d \in \{0,1,2\}$)	`lags`, `deterministic` (`"n"`, `"c"`, `"ct"`)
$k = n_d + np$	regressors $x_t = (d_t', y_{t-1}', \ldots, y_{t-p}')'$	
T (T_{eff}), T_{pad}	effective observations; padded length (bucket of 32 rows)	`T_eff`, `T_pad`
$q = k + n - 1$	coefficient slots of a structural equation ($k+i$ active in equation i)	
A, a_{ij} $(j < i)$	unit lower-triangular contemporaneous matrix	`eq.struct.a`
Γ, γ_i; $B' = A^{-1}\Gamma$; Π	structural, reduced-form ($k \times n$) coefficients	`eq.mean`, `glob.mean.Pi`
M, m_i, $\tilde{a}_i$	reduced-form prior mean, its column i, $\tilde{a}_i = (a_{i1}, \ldots, a_{i,i-1}, 1, 0, \ldots, 0)'$	`mean` (from tcodes)
$\delta_i = \gamma_i - M_{<i} a_i$	δ-coordinates (unit Jacobian)	`eq.mean.delta`
κ, α	overall tightness, lag decay	`lambda_` (0.2), `alpha` (2.0)
κ_c, κ_a, κ_d	cross-lag tightness (scales the s.d.), contemporaneous scale, deterministic-term variance	`cross` (1.0), `contemp` (1.0), `var_const` (10^7)
ψ_j	AR(p) residual variance of variable j in the window	`psi="ar"`
ϑ	Minnesota hyperparameters that the hierarchy fixes, estimates by EB or samples (κ, α, ψ, SOC/-SUR weights)	`hierarchy` (`"eb"`)
$\Sigma \sim \text{IW}(\Psi, d)$	$\Psi = \text{diag}\,\psi$, $d = n + 2$, $\text{E}[\Sigma] = \Psi/(d - n - 1)$	`ConjugateProblem.d`
$\text{IG}(c, C)$	inverse gamma, shape c, scale C; density $\propto \lambda^{-c-1} e^{-C/\lambda}$	
$\lambda_{i,t}$, $\lambda_{i,t}^{\text{eff}}$	structural variance; effective variance $e^{h_{i,t}} o_{i,t}^2 \tau_{i,t}$	`log_lam`, `h`
μ_i, ϕ_i, σ_i, B_σ	SV level, persistence, volatility of volatility, $\sigma_i^2 \sim B_\sigma \chi_1^2$	`SV(...)` priors
s_t, (p_j, m_j, v_j), c_i	mixture indicator, mixture table, offset $c_i = 10^{-12}\psi_i$	`mixture` (OCSN-10)
$o_{i,t} \in \{1, \ldots, K\}$, p_i	outlier scale ($K = 20$) and outlier probability	`outlier_max`
$\tau_{i,t}$, ν_i	Student-t mixing scale, degrees of freedom $\nu \in \{3, \ldots, 40\}$	`tails="t"`, `nu_prior`
S, H, $L_{s,h}$, $m_{s,h}$	forecast draws, horizon, conditional Cholesky factor and mean	`cond_chol`, `cond_mean`
$\tilde{m}_{s,h}$, $M_{s,h}$	path-integrated mean and factor of draw s at horizon h (D-34)	from `lag_coefs`
C, M_c, M_1, G	Geweke: chains, sweeps per chain, iid draws, test functions	`GewekeConfig`
N, L	SBC replications and kept draws per replication	`SBCConfig`

or cite only Sims and Zha (2006).] Three consequences are used in the paper. (i) With $d = n + 2$, $\sigma_\iota^2 \sim \text{IG}(1 + \iota/2, \psi_\iota/2)$ and $\text{E}[\sigma_\iota^2] = \psi_\iota/\iota$; this is the Constant prior of every Mode. (ii) Mode II at $\kappa_c = \kappa_a = 1$ is exactly (B.1) in δ-coordinates, $\delta_\iota \mid \sigma_\iota^2 \sim \text{N}(m_\iota, \sigma_\iota^2 \Omega)$, which is what makes the MCMC twin of the conjugate model an end-to-end oracle. (iii) $\text{E}[B' \mid A] = A^{-1}\text{E}[\Gamma \mid A] = M'$ holds for every A if and only if $\text{E}[\Gamma \mid A] = AM'$, so D3 centring is the only linear centring that keeps the reduced-form prior mean at M conditionally on A. The equivalence is tested in two ways. The structural normal–inverse-gamma log marginal likelihood equals the Phase 1 NIW value to 10^{-10} in five cases (constant only; SOC and SUR with own-lag means $(1, 0.5, 0, 0.9)$; trend and SUR; no deterministic terms with `soc_scope="all"`; $n = 1$) and under perturbed hyperparame-

Table A.2: Transformation codes, their pathwise inverses and the own-first-lag prior mean they imply. x is the raw series, y the transformed one, T the last observed period; sums and products run over $j = 1, \dots, h$.

tcode	y_t	Pathwise inverse for x_{T+h}	History	Own-lag mean
1	x_t	$x_{T+h} = y_{T+h}$	0	1
2	Δx_t	$x_{T+h} = x_T + \sum_j y_{T+j}$	1	0
3	$\Delta^2 x_t$	$\Delta x_{T+h} = \Delta x_T + \sum_j y_{T+j}$, then $x_{T+h} = x_T + \sum_j \Delta x_{T+j}$	2	0
4	$\log x_t$	$x_{T+h} = \exp(y_{T+h})$	0	1
5	$\Delta \log x_t$	$x_{T+h} = x_T \exp(\sum_j y_{T+j})$	1	0
6	$\Delta^2 \log x_t$	as code 3 on $\log x$, then exp	2	0
7	$\Delta(x_t/x_{t-1} - 1)$	$r_{T+h} = r_T + \sum_j y_{T+j}$ with $r_t = x_t/x_{t-1} - 1$, then $x_{T+h} = x_T \prod_j (1 + r_{T+j})$	2	0

ters. Kolmogorov–Smirnov tests on functionals of Σ and on coefficient marginals, from 20,000 prior draws, give a minimum p-value above 10^{-3}.

B.2 Mode I moment matching and the implied reduced-form prior

Mode I keeps the coefficients independent of λ_ι. With $\bar\sigma_\iota^2 = \psi_\iota/\iota$ it sets $a_{\iota j} \sim \mathrm{N}(0, \kappa_a \bar\sigma_\iota^2/\psi_j)$ and $\delta_\iota \sim \mathrm{N}(m_\iota, V_\iota)$, where V_ι has entries $\kappa^2/(\iota\, l^\alpha)$ on the own lag l, $\kappa^2\kappa_c^2\psi_\iota/(\iota\, l^\alpha \psi_m)$ on lag l of variable $m \neq \iota$ and $\kappa_d \psi_\iota/\iota$ on the deterministic terms; under Constant volatility $\lambda_\iota \sim \mathrm{IG}(1 + \iota/2, \psi_\iota/2)$, whose mean is $\bar\sigma_\iota^2$. At $\kappa_c = 1$, $V_\iota = \bar\sigma_\iota^2 \Omega$ with Ω the conjugate prior's Ω at $d = n + 2$. Write $u_t = A^{-1} e_t$ for the reduced-form residual and β_j for column j of B.

1. *Mean.* From $\Gamma = \Delta + (A - I)M'$, where Δ stacks the δ_ι,

$$B' = A^{-1}\Gamma = M' + A^{-1}(\Delta - M'), \qquad \mathrm{E}[B \mid A] = M \text{ for every } A. \tag{B.2}$$

2. *Covariance prior.* The rows a_ι have mean zero and are independent of $u_{<\iota}$, so $\mathrm{E}[\Sigma_{\iota\iota}] = \mathrm{E}[\lambda_\iota] + \sum_{j<\iota} \mathrm{E}[a_{\iota j}^2]\,\mathrm{E}[\Sigma_{jj}]$. At $\kappa_a = 1$ this is $\psi_\iota/\iota + \sum_{j<\iota}(\psi_\iota/(\iota\psi_j))\,\psi_j = \psi_\iota$ by induction, and the off-diagonal terms vanish by sign symmetry, so $\mathrm{E}[\Sigma] = \operatorname{diag}\psi$. The "naive" choice $a_{\iota j} \sim \mathrm{N}(0, \psi_\iota/\psi_j)$ with $\mathrm{E}[\lambda_\iota] = \psi_\iota$ instead gives $\mathrm{E}[\Sigma_{\iota\iota}] = 2^{\iota-1}\psi_\iota$, the growth pointed out by Chan et al. (2024).

3. *Marginal variances.* From (B.2), $\mathrm{Var}(\beta_j \mid A) = c_j(A)\,\Omega$ with $c_j(A) = \sum_\iota (A^{-1})_{j\iota}^2 \bar\sigma_\iota^2$ at $\kappa_c = 1$. Since $(A^{-1})_{j\iota} = \delta_{j\iota} - \sum_{l<j} a_{jl}(A^{-1})_{l\iota}$ with independent zero-mean rows, $c_j = \mathrm{E}[c_j(A)]$ satisfies $c_j = \bar\sigma_j^2 + \sum_{l<j}(\bar\sigma_j^2/\psi_l)\,c_l$, the recursion of item 2, so $c_j = \psi_j$. Because $\mathrm{E}[B \mid A]$ does not depend on A, $\mathrm{Var}(B_{(l,m),j}) = \kappa^2\psi_j/(l^\alpha\psi_m)$, the NIW marginal variance.

So, at $\kappa_c = \kappa_a = 1$, Mode I reproduces three moments of the NIW prior: the reduced-form mean (for every A), the marginal coefficient variances and $\mathrm{E}[\Sigma]$. It is not the NIW distribution.

Cross-lag leak. With $\kappa_c < 1$, $\kappa_a = 1$, $m_a = 0$ and $d = n + 2$, the same calculation with the own-lag entry separated out gives, for equation j and variable m (1-based),

$$\frac{\mathrm{Var}(B_{(l,m),j})}{\kappa^2\psi_j/(l^\alpha\psi_m)} = \kappa_c^2 + (1 - \kappa_c^2)\,s_{jm}, \qquad s_{jm} = \frac{\mathrm{E}[(A^{-1})_{jm}^2]\,\bar\sigma_m^2}{\psi_j} = \begin{cases} 1/j & m = j, \\ 1/(m(m+1)) & m < j, \\ 0 & m > j. \end{cases} \tag{B.3}$$

Here s_{jm} is the prior share of structural shock m in $\mathrm{Var}(u_j)$, so $\sum_m s_{jm} = 1$. The closed form follows from the share recursion $r_{j,m} = j^{-1}\sum_{l=m}^{j-1} r_{l,m}$ with $r_{m,m} = 1/m$, whose solution is $1/(m(m+1))$ for every $j > m$; it holds in Mode II as well, because $(A^{-1})_{j\iota}$ does not involve row ι of A. For example, at $\kappa_c = 0.5$ every later equation's coefficients on the lags of variable 1 get relative variance 0.625 instead of the intended 0.25. The implied reduced-form moments are reported by `fit.summary()`.

B.3 The exact mixture correction and the single ASIS accept/reject

Consider one equation with cleaned residuals $\tilde{e}_t$ (transitory scales removed), log-volatility path h and SV parameters $\theta = (\mu, \phi, \sigma)$. The sampler works with $y_t^* = \log(\tilde{e}_t^2 + c)$ and an auxiliary model $y_t^* = h_t + r_t$, where r_t has the mixture density $g(r) = \sum_j p_j \,\mathrm{N}(r; m_j, v_j)$ (Kim et al., 1998; Omori et al., 2007). Define the augmented target

$$\pi(\theta, h, s) = p(\theta, h \mid \tilde{e}) \prod_t \frac{p_{s_t}\,\mathrm{N}(y_t^* - h_t; m_{s_t}, v_{s_t})}{g(y_t^* - h_t)}, \tag{B.4}$$

whose marginal in (θ, h) is the exact posterior, because each factor of the product is a probability mass function in s_t. Drawing $s \mid h, \theta$ from that product is therefore an exact Gibbs step. Let $q(h \mid s, \theta) \propto p(h \mid \theta)\prod_t \mathrm{N}(y_t^* - h_t; m_{s_t}, v_{s_t})$ be the Gaussian conditional of the auxiliary model, which the precision sampler draws. Under (B.4), the conditional of h given (s, θ) is $q(h \mid s, \theta)\exp\{w(h)\}$ up to a constant, with

$$w(h) = \sum_t m_t \Big[\log \mathrm{N}(\tilde{e}_t; 0, e^{h_t}) - \log g(y_t^* - h_t)\Big], \qquad \log \mathrm{N}(\tilde{e}_t; 0, e^{h_t}) = -\tfrac{1}{2}\big(\log 2\pi + h_t + \tilde{e}_t^2 e^{-h_t}\big), \tag{B.5}$$

where m_t masks padded rows. An independence Metropolis–Hastings step with proposal q therefore accepts h' with probability $\min\{1, \exp(w(h') - w(h))\}$. This is Algorithm 3 of Del Negro and Primiceri (2015), with the precedent of Stroud et al. (2003). Because the same g appears in the indicator law and in w, the invariant law is exact for any mixture table and any offset $c \geq 0$; the table and the offset change only the acceptance rate and the mixing. This is why mutants that corrupt the table or the indicator law are not bugs in exact mode (Appendix D.6).

The non-centred step. The ancillarity–sufficiency interweaving step (Yu and Meng, 2011; Kastner and Frühwirth-Schnatter, 2014) holds $\tilde{h} = (h - \mu)/\sigma$ fixed and redraws (μ, σ) from the s-conditional Gaussian regression $y_t^* - m_{s_t} = \mu + \sigma\tilde{h}_t + \sqrt{v_{s_t}}\, z_t$ with a Gaussian prior on (μ, σ). That draw is an exact Gibbs step for the auxiliary augmented target $\tilde{\pi}$. The exact target is $\pi = \tilde{\pi} \cdot \exp\{w(h)\} \cdot \mathrm{const}$, and the reparametrisation Jacobian σ^{-T} appears in both and cancels, so using the $\tilde{\pi}$-conditional as a proposal under π gives the acceptance ratio $\exp\{w(\mu'+\sigma'\tilde{h}) - w(\mu+\sigma\tilde{h})\}$. If the non-centred step is itself a Metropolis–Hastings step under $\tilde{\pi}$, with ratio α_{aux}, a single accept/reject with

$$\log \alpha = \log \alpha_{\mathrm{aux}} + w(h_{\mathrm{new}}) - w(h_{\mathrm{old}}) \tag{B.6}$$

is used; two sequential tests would also be valid but define a different kernel. The value $w(h_{\mathrm{old}})$ must be that of the state after the h step of the same sweep; a stale value is a bug. The centred (γ, ϕ) and σ^2 steps need no correction: they do not move h, and the indicator law in (B.4) does not involve (μ, ϕ, σ). On acceptance, a negative σ' is folded by $(\sigma', \tilde{h}) \mapsto (-\sigma', -\tilde{h})$. The fold leaves $h = \mu + \sigma\tilde{h}$ unchanged and is a measure-preserving involution of the non-centred posterior, whose prior is symmetric under the joint sign change (Frühwirth-Schnatter and Wagner, 2010); folding σ alone is not. The combination (B.6) was derived in the repository's design notes and checked there on a toy posterior by quadrature; it is gated by the identity tests above, by the toy quadrature

twin (A6), by Geweke tests in exact mode, and by mutant M7 with a coarse mixture table. We do not claim it as new. [**TODO:** TODO 11: prior-art search for the ASIS + exact-correction ratio before any novelty wording.]

B.4 The path-integrated log predictive density, and a bound on its delta-method MCSE

Derivation. Fix a forecast draw s: reduced-form lag matrices $A_1, \dots, A_p$, deterministic terms, and a drawn path of effective volatilities, which determines the conditional Cholesky factors $L_k = \texttt{cond_chol}_{s,k}$ for $k = 1, \dots, h$. The simulated path satisfies $y_{T+k} = \text{cond_mean}_k + u_k$ with $u_k = L_k \varepsilon_k$, where cond_mean_k is the VAR mean evaluated on the simulated path up to $T + k - 1$. Let $\Phi_0 = I$ and $\Phi_j = \sum_{l=1}^{\min(j,p)} A_l \Phi_{j-l}$. By linearity, $y_{T+h} = \tilde{m}_h + \sum_{k=1}^{h} \Phi_{h-k} u_k$, where $\tilde{m}_h$ is the zero-shock path, and $\text{cond_mean}_h = y_{T+h} - u_h$. Hence

$$\begin{aligned} \tilde{m}_h &= \text{cond_mean}_h - \sum_{k<h} \Phi_{h-k} u_k, \\ y_{T+h} \mid \theta_s, \lambda_s \sim \mathrm{N}(\tilde{m}_h, M_h M_h'), &\qquad M_h = [\Phi_{h-1} L_1 \mid \Phi_{h-2} L_2 \mid \cdots \mid \Phi_0 L_h], \end{aligned} \tag{B.7}$$

because the u_k are independent $\mathrm{N}(0, L_k L_k')$ given the volatility path, and the deterministic terms cancel. For a subset $\mathcal{I}$ of variables, the lower factor of $(M_h M_h')_{\mathcal{I}\mathcal{I}}$ is R' from the QR decomposition $M_h[\mathcal{I}, :]' = QR$ with $\operatorname{diag}(R) > 0$, so $M_h M_h'$ is never formed and no additional random numbers are drawn. The estimator is $\operatorname{logsumexp}_s \log \mathrm{N}(y; \tilde{m}_{s,h}, M_{s,h} M_{s,h}') - \log S$ (equation (17) in the notation of Section 6.4). At $h = 1$, $M_1 = L_1$ and $\tilde{m}_1 = \text{cond_mean}_1$, so it coincides with the path-conditioned estimator, and the library runs the conditioned code there. It is exact given each draw's parameters and volatility path, and it is still a Monte Carlo average over draws. In a synthetic persistent VAR at $h = 8$ with an outcome 6 standard deviations out, the test's docstring records a path-conditioned estimate of -35.2 (MCSE 1.0) against the exact -22.3; the test asserts a gap of more than 5 nats and an MCSE above 0.5, and that the integrated estimator is exact.

A bound on the delta-method MCSE. The library reports the delta-method MCSE $\operatorname{sd}(w)/(\sqrt{S}\,\bar{w})$ with $w_s = \exp(\log p_s - \max_r \log p_r)$ and the sample standard deviation (`ddof=1`); for MCMC draws it multiplies this by $\sqrt{S/\mathrm{ESS}}$, with ESS the effective sample size of the weights over the (chain, draw) array.

Proposition B.4. *Let $w_1, \dots, w_S \geq 0$, not all zero, with mean $\bar{w}$, sample variance $s_w^2 = (S-1)^{-1} \sum_s (w_s - \bar{w})^2$ and Kish effective sample size $\mathrm{ESS}_K = (\sum_s w_s)^2 / \sum_s w_s^2$ (Kish, 1965). Then*

$$\frac{s_w^2}{S\,\bar{w}^2} = \frac{S/\mathrm{ESS}_K - 1}{S - 1} \leq 1, \tag{B.8}$$

with equality if and only if exactly one w_s is non-zero. With the population variance $\sigma_w^2 = S^{-1} \sum_s (w_s - \bar{w})^2$ in place of s_w^2, the bound is $\sigma_w/(\sqrt{S}\bar{w}) \leq \sqrt{(S-1)/S}$.

Proof. $(S-1)s_w^2 = \sum_s w_s^2 - S\bar{w}^2 = S^2\bar{w}^2/\mathrm{ESS}_K - S\bar{w}^2$; dividing by $S(S-1)\bar{w}^2$ gives the identity. For non-negative weights $\sum_s w_s^2 \leq (\sum_s w_s)^2$, that is $\mathrm{ESS}_K \geq 1$, with equality if and only if $\sum_{s \neq r} w_s w_r = 0$, that is, if at most one weight is non-zero. The population case follows from $\sigma_w^2 = (S-1)s_w^2/S$. ∎

Four consequences follow. (i) The iid delta-method MCSE of a Rao–Blackwellised log score cannot exceed 1 nat, however far off the estimate is; it approaches 1 as the weight concentrates on one draw, and reaches it in floating point when the other weights underflow. A reported MCSE near 1 at a tail origin is therefore the ceiling of the formula, not an estimate of the error. (ii) The

iid MCSE is a function of ESS_K alone; for large S it is approximately $(1/\mathrm{ESS}_K - 1/S)^{1/2}$, so the descriptive RB-degenerate threshold of 0.9 used in the replication corresponds, before the MCMC adjustment, to $\mathrm{ESS}_K \approx 1/0.81 \approx 1.2$. (iii) The factor $\sqrt{S/\mathrm{ESS}}$ applied to MCMC draws is not covered by the proposition and can raise the reported value above 1. (iv) An MCSE is a model of the error, not a certificate: at tail origins the weight ESS should be inspected directly. [**TODO:** TODO 14: confirm Proposition B.4 (including the ddof convention and the $\sqrt{S/\mathrm{ESS}}$ factor) and add a unit test before Section 6 and Section 11 rely on it.]

B.5 Dummy rows: the closed-form λ marginal and an exact rejection envelope

The SOC/SUR rows (y_d, F_d) of equation i (m rows) are pseudo-observations that update a base prior $\theta \sim \mathrm{N}(\mu_0, V_0)$ of the equation's coefficients; write $r = y_d - F_d\mu_0$.

Mode II. Here $V_0 = \lambda\tilde{V}_0$ and the rows have noise variance λ/w, so $d \mid \lambda \sim \mathrm{N}(F_d\mu_0, \lambda\tilde{S})$ with $\tilde{S} = F_d\tilde{V}_0F_d' + I/w$. With the base prior $\lambda \sim \mathrm{IG}(c, C)$,

$$\lambda \mid d \sim \mathrm{IG}\big(c + m/2,\ C + \tfrac{1}{2}r'\tilde{S}^{-1}r\big), \tag{B.9}$$

and $p(d)$ has a closed form. The Geweke reference therefore samples the prior with the rows exactly: draw $\lambda \mid d$, then condition θ on the rows by Matheron's rule, $\theta = \theta_0 + V_0F_d'(F_dV_0F_d' + RI)^{-1}(y_d - F_d\theta_0 - \varepsilon)$ with $\theta_0 \sim \mathrm{N}(\mu_0, V_0)$ and $\varepsilon \sim \mathrm{N}(0, RI)$, which is exact Gaussian conditioning (Journel and Huijbregts, 1978; Chilès and Delfiner, 2012; Wilson et al., 2020). The reference is written in this covariance form ($m \times m$ systems), while the kernels use the precision form ($q \times q$ Gram updates), so the two implementations differ in algebra, not only in authorship.

iw_equivalent. Only the a-block is λ-scaled and the rows have a fixed weight, so $d \mid \lambda \sim \mathrm{N}(r; 0, S(\lambda))$ with $S(\lambda) = S_0 + \lambda UU'$, $S_0 = F_\delta V_\delta F_\delta' + I/w$ and $U = F_a\tilde{V}_a^{1/2}$. This is not of inverse-gamma form. Let $q(\lambda) = r'S(\lambda)^{-1}r$ and $q_\infty = \min_u (r - Uu)'S_0^{-1}(r - Uu)$, the generalised least-squares residual.

Proposition B.5. *For every $\lambda > 0$,*

$$g(\lambda) = \Big(\frac{|S_0|}{|S(\lambda)|}\Big)^{1/2} \exp\big\{-\tfrac{1}{2}\big(q(\lambda) - q_\infty\big)\big\} \le 1, \tag{B.10}$$

so drawing $\lambda \sim \mathrm{IG}(c, C)$ and accepting with probability $g(\lambda)$ yields exact draws from $p(\lambda \mid d) \propto \mathrm{IG}(\lambda; c, C)\,\mathrm{N}(r; 0, S(\lambda))$.

Proof. First, $|S(\lambda)| = |S_0|\,|I + \lambda U'S_0^{-1}U| \ge |S_0|$ because $U'S_0^{-1}U \succeq 0$. Second, for $\lambda > 0$ the function $f(u) = (r - Uu)'S_0^{-1}(r - Uu) + u'u/\lambda$ is minimised at $u^* = (U'S_0^{-1}U + I/\lambda)^{-1}U'S_0^{-1}r$, with minimum $r'S_0^{-1}r - r'S_0^{-1}U(U'S_0^{-1}U + I/\lambda)^{-1}U'S_0^{-1}r = r'(S_0 + \lambda UU')^{-1}r = q(\lambda)$ by the Woodbury identity. Dropping the non-negative term $u'u/\lambda$ gives $q(\lambda) \ge q_\infty$. Hence $\mathrm{N}(r; 0, S(\lambda)) \le (2\pi)^{-m/2}|S_0|^{-1/2}e^{-q_\infty/2}$, and $g(\lambda)$ is the ratio of the two sides. The usual rejection argument completes the proof. ∎

The same argument shows that $q(\lambda)$ is non-increasing and tends to q_∞ as $\lambda \to \infty$, which explains the name. Proposals are drawn in batches; the first accepted proposal of a batch is the sequential rejection draw, so the batched sampler is exact. The repository notes record an acceptance rate of 0.6–1.0 on the test rows. The log prior with the rows is exact up to the intractable evidence, which is enough for every density-ratio identity and every Geweke and SBC statistic. The reference is checked against the closed forms to 10^{-10} and by Kolmogorov–Smirnov tests of the λ draws against a quadrature CDF; Geweke row 205 gives a G-small maximum $|z|$ of 2.24, and the repository notes record G-full 2.76 and an SBC minimum p of 0.018 for the same row. In the sampler, Mode II rows are weighted $1/\lambda_i$ like data rows, so the conditional of λ_i gains $m/2$ in its shape and the rows' λ-free squared residuals in its scale.

B.6 The successive-conditional chain of a scalar conjugate model

Take $\theta \sim \mathrm{N}(0, \tau^2)$ and $y \mid \theta \sim \mathrm{N}(\theta, s^2)$, where s^2 stands for the data variance ($s^2 = s_0^2/T$ for T observations), and let $\zeta = \tau^2/(\tau^2 + s^2)$. One successive-conditional sweep of a Geweke test draws $y_{m+1} \sim \mathrm{N}(\theta_m, s^2)$ and then $\theta_{m+1} \mid y_{m+1} \sim \mathrm{N}(\zeta y_{m+1}, \zeta s^2)$, so

$$\theta_{m+1} = \zeta(\theta_m + e_m) + \sqrt{\zeta s^2}\, z_m, \qquad e_m \sim \mathrm{N}(0, s^2),\ z_m \sim \mathrm{N}(0,1). \tag{B.11}$$

This is an AR(1) with coefficient ζ and stationary variance $\zeta^2(\tau^2 + s^2) + \zeta s^2 = \tau^2$, as it must be. With $r = \mathrm{Var}_{\mathrm{prior}} \,/\, \mathrm{Var}_{\mathrm{post}} = \tau^2/(\zeta s^2) = 1/(1-\zeta)$,

$$\zeta = 1 - 1/r, \qquad \mathrm{IAT}(\theta) = \frac{1+\zeta}{1-\zeta} = 2r - 1, \qquad \mathrm{RNE} \approx \frac{1}{2r-1}. \tag{B.12}$$

If the posterior sampler is itself autocorrelated with coefficient ρ, the chain is AR(1) with coefficient $1 - (1-\zeta)(1-\rho)$. The design rule $r \le 5$ (IAT ≤ 9) for every scalar parameter follows, and it is targeted with small T (20 for G-small, 50 for G-full) and proper priors that are tight relative to T observations. This is a derivation for the scalar conjugate model; that the rule holds for ϕ, σ, p and ν in the registered compositions is not verified. The minimum relative numerical efficiencies measured at G-small range from 0.008 to 0.129 across compositions; in the four compositions for which the lowest-efficiency functions were extracted (`structural_constant`, `reduced_constant`, `structural_sv` and the univariate `zero_sv`) they occur on intercepts, on the data function $\mathrm{mean}_t \tanh y_t$ and, in `zero_sv`, on h levels, not on ϕ or σ, so they are not direct evidence either way; under the parallel-chain design low efficiency affects power, not size.

B.7 Why the CCM-2019 triangular step is exact under a flat prior

Write the reduced form in triangular form, $\tilde{y}_t = A y_t$, and take two equations. Equation 2 reads $\tilde{y}_{2,t} = a_{21} x_t' \pi^{(1)} + x_t' \pi^{(2)} + \lambda_{2,t}^{1/2} \varepsilon_{2,t}$. The map $(\pi^{(1)}, \pi^{(2)}) \mapsto (\pi^{(1)}, \gamma_2 = a_{21}\pi^{(1)} + \pi^{(2)})$ has unit Jacobian, and because all equations share the regressors x_t and A does not vary over time, equation 2 depends on $(\pi^{(1)}, \pi^{(2)})$ only through γ_2, whatever the volatility weights. Under a flat prior on $(\pi^{(1)}, \pi^{(2)})$ the posterior of $\pi^{(1)}$ therefore depends on equation 1 alone, and drawing $\pi^{(1)}$ from equation 1, as the triangular algorithm of Carriero et al. (2019) does, is exact. Under a proper prior $\pi^{(2)} \sim \mathrm{N}(\mu_2, V_2)$, independent of $\pi^{(1)}$, the term the algorithm omits is

$$p(\tilde{y}_2 \mid y_1, \pi^{(1)}) = \mathrm{N}\big(\tilde{y}_2;\ X(a_{21}\pi^{(1)} + \mu_2),\ XV_2X' + \Lambda_2\big), \tag{B.13}$$

which depends on $\pi^{(1)}$ whenever $a_{21} \neq 0$. The error is thus a prior effect: it is large with tight Minnesota priors and short samples and fades as the data dominate. Averaged over simulated datasets, per-dataset shifts of the posterior mean in both directions can cancel in the ranks of the parameters themselves, while a data-dependent quantity such as the log-likelihood keeps its power, as Modrák et al. (2025) discuss for SBC. [**TODO:** TODO 15: check the Modrák et al. theorem numbering against the journal version if a theorem is cited here.] This is the mechanism behind lesson 3 of the mutation suite: the block SBC of the π_0 draw catches M1 through the log-likelihood rank ($p = 2.8 \times 10^{-11}$, control minimum $p = 0.10$), whereas one-block invariance of the whole scan did not in a recorded run that has not been repeated ($q = 0.29$). The derivation is recorded in the design notes, not in a committed test. Bognanni (2022) showed that the triangular algorithm does not, in general, sample from the intended posterior, and the corrected algorithm is that of Carriero et al. (2022). With a time-varying A_t the shift $a_{21,t} x_t' \pi^{(1)}$ cannot be absorbed into one γ_2, and the flat-prior exactness fails.

B.8 The excess-pivot recursion and the QR whitened draw

The recursion. Let $Q = \mathrm{Lap}_w + \mathrm{diag}(\delta)$ be the scaled precision of the random-walk log-volatility path, with $\delta_t \geq 0$, edge weights $w_t \geq 0$ ($w_{-1} = w_T = 0$), $Q_{tt} = \delta_t + w_{t-1} + w_t$ and $Q_{t,t-1} = -w_{t-1}$. In the SV application $\delta_0 = \sigma^2/V_0$, $\delta_t = \sigma^2/v_{s_t}$ at observed periods (0 at masked ones; notation of Section 5.5) and the live edges have unit weight.

Proposition B.8. *The Cholesky pivots p_t (squared diagonal entries of the lower factor) of Q are $p_t = q_t + w_t$, where*

$$q_0 = \delta_0, \qquad q_t = \delta_t + \frac{w_{t-1}\, q_{t-1}}{w_{t-1} + q_{t-1}} \quad (q_t = \delta_t \text{ if } w_{t-1} = 0), \tag{B.14}$$

the subdiagonal of the factor is $-w_{t-1}/\sqrt{p_{t-1}}$, and $\log|Q| = \sum_t \log(q_t + w_t) = \sum_t[\log w_t + \mathrm{log1p}(q_t/w_t)]$, with $\log q_t$ for the terms with $w_t = 0$.

Proof. For a tridiagonal matrix, $p_0 = Q_{00}$ and $p_t = Q_{tt} - Q_{t,t-1}^2/p_{t-1}$. Substituting $p_t = q_t + w_t$ gives $q_t = \delta_t + w_{t-1} - w_{t-1}^2/(q_{t-1} + w_{t-1}) = \delta_t + w_{t-1}q_{t-1}/(w_{t-1} + q_{t-1})$. ∎

Every operation in (B.14) combines non-negative numbers by addition, multiplication and division, so nothing cancels. The assembled matrix, by contrast, stores $Q_{tt} = \sigma^2/v_{s_t} + 2$: when σ^2/v_{s_t} falls below half an ulp of 2, rounding returns the singular path Laplacian, and the level pivot, of order $\sigma^2(1/V_0 + \sum_t 1/v_{s_t})$, is lost. The recursion appears to be an instance of subtraction-free Gaussian elimination for diagonally dominant M-matrices parametrised by off-diagonals and diagonal excesses (Grassmann et al., 1985; Alfa et al., 2002; Ye, 2008), applied within the precision-sampler setting of Rue (2001); we claim the application, not the algorithm, and we do not prove an error bound here. [**TODO:** TODO 11: confirm the GTH / Alfa–Xue–Ye / Ye (2008) positioning (and GMRF samplers) by a prior-art search, then drop "appears to be".] In exact arithmetic the pivots are those of the assembled factorisation, so the sampler is the same function of the injected noise.

The QR whitened draw. For the contemporaneous row a_i with prior precision diag(prec), prior mean $\bar{a}$, regressors Z and weights w_t, let $\mathcal{A} = [\mathrm{diag}(\sqrt{\mathrm{prec}});\ \sqrt{w} \odot Z]$ and $\rho = [\sqrt{\mathrm{prec}} \odot \bar{a};\ \sqrt{w} \odot (\tilde{y}_i - g_i)]$, so that $\mathcal{A}'\mathcal{A} = K$ and $\mathcal{A}'\rho = b$ are the conditional precision and the canonical mean. With the Householder decomposition $\mathcal{A} = \mathcal{Q}R$, $\mathrm{diag}(R) > 0$, R is the upper Cholesky factor of K, and $\mathcal{Q}'\rho = (R')^{-1}b$, so

$$a = R^{-1}(\mathcal{Q}'\rho + z) = K^{-1}b + R^{-1}z. \tag{B.15}$$

The Jacobi-scaled precision draw computes $\theta = s \odot (L_s')^{-1}(L_s^{-1}(s \odot b) + z)$ with $s = \mathrm{diag}(K)^{-1/2}$ and $L_s L_s' = sKs$; since $s^{-1}L_s$ is the lower Cholesky factor of K, this is also $K^{-1}b + R^{-1}z$. The two draws are therefore the same function of z in exact arithmetic, but the QR route works with $\mathrm{cond}(\mathcal{A}) = \mathrm{cond}(K)^{1/2}$ and never squares it (Higham, 2002; Björck, 1996).

C Algorithms

The pseudocode below follows the committed code of the MCMC engine (file and symbol named in each caption) and is written for one chain and, where stated, one equation; equations and chains are batched by `vmap`. All arithmetic is float64. Every kernel is written as `core(noise, ...)` applied to `draw_noise(key, ...)`, so the noise z or u in each algorithm is drawn from the key named in the text and could be injected by a test. A step reports success through a flag `ok` and the smallest pivot it met; a failure is recorded as a status code and never raises inside compiled code (Algorithm C.11). Mask $m_t \in \{0, 1\}$ marks observed rows of the padded sample. Figure C.1 shows the plan as the package prints it, and Table C.2 the predictive contract that the forecast core (Algorithm C.12) fills.

Algorithm C.1 One Gibbs sweep of one chain (`engines/mcmc/sweep.py`, `_sweep_fn`). The order of the steps is the `Plan` built by `build_plan`.

Input: chain key k_c, absolute sweep index it (uint32), state, data, prior, cache

for each global pre-step g (reduced form: the CCCM step, Algorithm C.5) **do**
    update g's block with key `step_key`$(k_c, it, g.\text{block})$; record its status
end for
$\tilde{Y} \leftarrow$ pre-residual target (Y, or $Y - X\Pi$ in reduced form), masked
for all equations i in parallel **do**
    for each step of `plan.equation`, in order **do**
        rebuild the equation context from the *current* state; weights $m_t \exp(-\log \lambda_{i,t}^{\text{eff}})$
        key $\leftarrow$ `eq_key`(`step_key`(k_c, it, block), i)
        `equation`: draw $(\delta_i, a_i) \mid \lambda^{\text{eff}}$ (Algorithms C.2 and C.3); or `contemp`: draw $a_i \mid g_i$ (Algorithm C.4)
        `residual`: $e_i \leftarrow \tilde{y}_i - (\text{cols} \odot \text{cmask})\, a_i - g_i$ (zero on padded rows)
        `vol`: update the volatility block atomically (Algorithm C.6)
    end for
end for
keep the first non-zero code $1 + 1000 \cdot \text{block} + \text{eq}$ per chain and the running minimum pivot

Algorithm C.2 Structural (δ_i, a_i) draw with D3 centring, weighted path (`components/mean/linear.py`; `linalg.weighted_gram`, `precision_draw`). $F = [X \mid XM_c - Y_{:,1:n-1}]$ is shared by all equations; c_θ marks the active slots of equation i.

per fit: packed products $FF_{t,c} = F_{t,r_c} F_{t,s_c}$, $r_c \le s_c$, column-major ($T_{\text{pad}} \times q(q+1)/2$)
$\text{prec}_\theta \leftarrow [\text{pm.prec} \cdot e^{-\log \lambda_i \, 1\{\text{Mode II}\}} \mid \text{cm.prec}]$; $\text{mean}_\theta \leftarrow [\text{pm.mean} \mid \text{cm.mean}]$
$w_t \leftarrow m_t \exp(-\log \lambda_{i,t}^{\text{eff}})$ ▷ masked before the exponential
$G \leftarrow (w'FF)$ unpacked to $q \times q$; $f \leftarrow F'(w \odot y_i)$ ▷ one GEMM under `vmap`
$K \leftarrow (c_\theta c_\theta') \odot G + \text{diag}(\text{prec}_\theta)$; $b \leftarrow \text{prec}_\theta \odot \text{mean}_\theta + c_\theta \odot f$
$z \sim \text{N}(0, I_q)$ from `sub_key`(key, `Sub.Z`)
$s \leftarrow \text{diag}(K)^{-1/2}$; $L \leftarrow \text{chol}(sKs)$ ▷ Jacobi scaling, an exact change of variables
$\theta \leftarrow s \odot (L')^{-1}(L^{-1}(s \odot b) + z)$ ▷ $\sim \text{N}(K^{-1}b, K^{-1})$
ok $\leftarrow$ diag L finite and > 0, θ finite; min_pivot $\leftarrow \min \text{diag}(L)^2$
$\theta \leftarrow \theta \odot c_\theta$; $\delta_i \leftarrow \theta_{1:k}$; $a_i \leftarrow \theta_{k+1:q}$; $\gamma_i = \delta_i + M_c a_i$
coupling $\leftarrow 0$ (Mode I), or $(n_{\text{active}}, \frac{1}{2}\sum \tilde{P}_\theta(\theta - \text{mean}_\theta)^2)$ over the λ-scaled slots (Mode II, iw_equivalent)

Algorithm C.3 Graded eigen fast path for Mode I × Constant (`linalg.eigen_prepare`, `eigen_draw`). G_{const} is the time-constant Gram and $\text{FtY}_i = F'y_i$.

per fit, per equation:
$d \leftarrow \text{prec}_\theta$; $\tilde{G} \leftarrow d^{-1/2}(c_\theta c_\theta' \odot G_{\text{const}})\, d^{-1/2}$
graded decomposition $\tilde{G} = Q\,\text{diag}(g)Q'$: unit-diagonal rescaling, Cholesky (eigendecomposition fallback), Householder QR of the rescaled factor sorted by decreasing scale, QR-iteration SVD of R'
$u_1 \leftarrow Q'(d^{1/2} \odot \text{mean}_\theta)$; $u_2 \leftarrow Q'(d^{-1/2} \odot c_\theta \odot \text{FtY}_i)$
per sweep, $\lambda = \lambda_i$:
$v \leftarrow \lambda/(\lambda + g)$ ▷ g clipped at 0
$\theta \leftarrow d^{-1/2} \odot Q\,(v \odot u_1 + u_2/(\lambda + g) + \sqrt{v} \odot z)$ ▷ exact: $K = d^{1/2}(I + \tilde{G}/\lambda)d^{1/2}$
ok $\leftarrow$ g finite, $g \ge -10^{-10} \max|g|$, θ finite

The graded decomposition is needed because $\kappa_d = 10^7$ makes one diagonal entry of $\tilde{G}$ about 10^9 times the others; its accuracy is observed against 40–45-digit solves, not proven. With dummy rows the same per-fit decomposition is applied after factoring the dense prior precision, $K = L_P(I + L_P^{-1}G(L_P')^{-1}/\lambda)L_P'$.

Algorithm C.4 Contemporaneous row a_i by QR, for non-fusing means and the reduced form (`components/structure/recursive.py`, `contemp_core`; `linalg.lstsq_draw`). The target is $\tilde{y}_i - g_i = -\tilde{y}_{<i}\, a_i + e_i$.

1: $Z \leftarrow \text{cols} \odot \text{cmask}$ with cols $= -\tilde{Y}_{:,1:n-1}$; $\sqrt{w_t} \leftarrow \sqrt{m_t \exp(-\log \lambda_{i,t}^{\text{eff}})}$
2: $\mathcal{A} \leftarrow [\text{diag}(\sqrt{\text{prec}});\ \sqrt{w} \odot Z]$; $\rho \leftarrow [\sqrt{\text{prec}} \odot \text{mean};\ \sqrt{w} \odot (\tilde{y}_i - g_i)]$ ▷ $\mathcal{A}'\mathcal{A} = K_a$, $\mathcal{A}'\rho = b_a$
3: Householder QR $\mathcal{A} = \mathcal{Q}R$, signs fixed so that $\text{diag}(R) > 0$ ▷ R is the upper Cholesky factor of K_a
4: $a \leftarrow R^{-1}(\mathcal{Q}'\rho + z)$ ▷ $= K_a^{-1}b_a + R^{-1}z$, Appendix B.8
5: min_pivot $\leftarrow \min_j (R_{jj}/\|\mathcal{A}_{:,j}\|)^2$; ok $\leftarrow \text{diag}(R) > 0$ and a finite
6: $a \leftarrow a \odot$ cmask; coupling as in Algorithm C.2 if the row is λ-scaled

Algorithm C.5 One corrected triangular (CCCM) scan for the reduced-form coefficients (`components/mean/linear_reduced.py`, `cccm_core`), following Carriero et al. (2022).

1: $A \leftarrow I + \text{strict-lower}(a)$; $\omega_{i,t} \leftarrow m_t \exp(-\log \lambda_{i,t}^{\text{eff}})$
2: $W \leftarrow \omega'(A \circ A)$ ▷ $W_{tj} = \sum_i a_{ij}^2/\lambda_{it} = (\Sigma_t^{-1})_{jj}$
3: **for all** j in parallel **do** ▷ K_j does not depend on Π
4: $\quad K_j \leftarrow X' \,\text{diag}(W_{:,j}) X + \text{diag}(\text{prec}_{0,j})$; $(L_j, s_j) \leftarrow$ Jacobi-scaled Cholesky of K_j
5: **end for**
6: $E \leftarrow (Y - X\Pi)A'$ ▷ recomputed from scratch every sweep
7: **for** $j = 1, \ldots, n$ (sequential) **do**
8: $\quad f \leftarrow X\pi_j$; $G \leftarrow (E \odot \omega')A_{:,j}$ ▷ $(\Sigma_t^{-1}u_t)_j$
9: $\quad b \leftarrow \text{prec}_{0,j} \odot \text{mean}_{0,j} + X'(G + W_{:,j} \odot f)$
10: $\quad \pi_j^{\text{new}} \leftarrow s_j \odot (L_j')^{-1}(L_j^{-1}(s_j \odot b) + z_j)$
11: $\quad E \leftarrow E - X(\pi_j^{\text{new}} - \pi_j)A_{:,j}'$; $\Pi_{:,j} \leftarrow \pi_j^{\text{new}}$
12: **end for**
13: ok $\leftarrow$ all factorisations ok and Π finite

The scan is the textbook SUR Gibbs conditional with weight $A'\Lambda_t^{-1}A$, so it is a block Gauss–Seidel sweep on the joint precision of $\text{vec}\,\Pi$ (Fox and Parker, 2017); A is never inverted. The noise row z_j comes from `sub_key(eq_key(step_key, j), Sub.Z)`. Mutant M1 replaces the scan by the triangular step of Carriero et al. (2019).

Algorithm C.6 The volatility block of one equation, transitory then persistent, atomic (`components/vol/robust.py`, `VolBlockImpl.sample_eq`).

1: $\log e_t^2 \leftarrow \log e_t^2$ on observed rows, 0 on padded rows ▷ mask before the log (erratum E23)
2: transitory update $o \to p \to \nu \to \tau$ given the current persistent variance (Table C.1)
3: $\log \tilde{e}_t^2 \leftarrow \log e_t^2 - (2\log o_t + \log \tau_t)$ on observed rows
4: persistent core on $\log \tilde{e}^2$: Constant (inverse-gamma draw with the coupling channel) or SV (Algorithm C.7)
5: ok $\leftarrow$ core ok and every transitory leaf finite

Nothing runs between the indicator draw inside the SV core and the h draw, so the step-order rule of Del Negro and Primiceri (2015) holds by construction; the indicators are local to the core and never part of the state. The Constant core draws $\lambda_i \sim \text{IG}(c_i + T_{\text{eff}}/2 + n_{\text{active}}/2, C_i + \frac{1}{2}\sum_t m_t \tilde{e}_t^2 + \text{quad})$, where $(n_{\text{active}}, \text{quad})$ is the coupling returned by the coefficient step (zero in Mode I).

The collapsed steps are partially collapsed Gibbs blocks (van Dyk and Park, 2008) whose trimmed draws are never conditioned on before they are redrawn. The τ draws use a vectorised

Table C.1: Transitory steps (`components/vol/transitory.py`); all conditionals are exact. $\log z_t^2 = \log e_t^2 - \log \lambda_t^{\text{pers}}$, r_t^2 the corresponding squared standardised residual, $c(\nu) = \nu - 2$ (standardised t) or ν (unit scale).

Kind	Steps, in order	Conditionals
outlier	$o \mid e, h, p$; $p \mid o$	logits $\log w_k - \log k - z_t^2/(2k^2)$, $w_1 = 1-p$, $w_k = p/(K-1)$, $K = 20$; $p \sim \text{Beta}(a_p + N_1, b_p + T_{\text{eff}} - N_1)$
t	$\nu \mid e, h$ (τ integrated out); $\tau \mid e, h, \nu$	grid logits $\log \pi_g + T_{\text{eff}} A_g - \frac{\nu_g+1}{2} \sum_t m_t \operatorname{log1p}(r_t^2/c_g)$ on $\nu \in \{3, \dots, 40\}$; $\tau_t \sim \text{IG}\big(\frac{\nu+1}{2}, \frac{c(\nu)+r_t^2}{2}\big)$
outlier $+ t$	$o \mid e, h, \nu, p$ (τ int.); $p \mid o$; $\nu \mid e, h, o$ (τ int.); $\tau \mid e, h, o, \nu$	collapsed pmf $\log w_k - \log k - \frac{\nu+1}{2} \operatorname{log1p}\big(z^2/(c\,k^2)\big)$

Marsaglia–Tsang gamma sampler (Marsaglia and Tsang, 2000) with two pre-drawn candidates per element and a loop only for elements that rejected both; the first accepted candidate of an iid sequence is the rejection sampler, so the draw is exact.

Algorithm C.7 SV update of one equation, AR(1) log-volatility, with the exact correction and ASIS (`components/vol/sv.py`, `SVCore`). Steps (c), (a), (b) and (b**) follow the design document; $w(\cdot)$ is (B.5).

1: $y_t^* \leftarrow \text{logaddexp}(\log \tilde{e}_t^2, \log c_i)$ on observed rows ▷ $c_i = 10^{-12}\psi_i$
2: (c) $s_t \leftarrow$ inverse-CDF categorical draw from $\ell_{tj} = \log p_j - \frac{1}{2}\log v_j - (y_t^* - h_t - m_j)^2/(2v_j)$; keep $\log g(y_t^* - h_t)$
3: (a) assemble the non-centred AWOL tridiagonal system $\tilde{Q} = \sigma^2 Q$, $\tilde{b}$ given s; $\tilde{x}' \leftarrow$ tridiagonal precision draw (random walk: Algorithm C.8); $h' \leftarrow \mu + \sigma\tilde{x}'$
4: **if** exact **then** accept h' iff $\log u_h < w(h') - w(h)$ ▷ Del Negro and Primiceri (2015), Alg. 3
5: **end if**
6: (b) centred (γ, ϕ) by independence MH, rejecting $|\phi'| \geq 1$; σ^2 by MH under $B_\sigma \chi_1^2$ or an exact inverse-gamma draw
7: (b**) $\tilde{h} \leftarrow (h - \mu)/\sigma$; ϕ by MH; (μ', σ') from the exact 2×2 weighted regression of $y^* - m_s$ on $(1, \tilde{h})$
8: **if** exact **then** one accept/reject with $\log \alpha = \log \alpha_{\text{aux}} + w(\mu' + \sigma'\tilde{h}) - w(h)$, $w(h)$ from after step (a)
9: **end if**
10: **if** accepted and $\sigma' < 0$ **then** $(\sigma', \tilde{h}) \leftarrow (-\sigma', -\tilde{h})$ ▷ fold; h unchanged
11: **end if**
12: refresh padded states; ok $\leftarrow$ pivot > 0, $C_T > 0$, $\sigma > 0$, $|\phi| < 1$, new state finite

The cold start draws h once from the AWOL system after the initial values, because with $h \equiv \mu$ and a rejected first exact step the σ^2 proposal is the point 0, which is then absorbing; this was found on the pilot prototype. The AWOL precision draw follows Chan and Jeliazkov (2009) and McCausland et al. (2011).

Algorithm C.8 Random-walk SV path as a path Laplacian (`linalg.laplacian_precision_sample`): $Q = \text{Lap}_w + \text{diag}(\delta)$, $Q_{tt} = \delta_t + w_{t-1} + w_t$, $Q_{t,t-1} = -w_{t-1}$ (Appendix B.8).

1: $q_0 \leftarrow \delta_0$; $q_t \leftarrow \delta_t + w_{t-1}q_{t-1}/(w_{t-1} + q_{t-1})$ ($q_t = \delta_t$ after a zero edge) ▷ excess over the next edge
2: $p_t \leftarrow q_t + w_t$; $\ell_t \leftarrow \sqrt{p_t}$; $c_t \leftarrow -w_{t-1}/\ell_{t-1}$ ▷ all terms ≥ 0
3: forward: $v_t \leftarrow (b_t - c_t v_{t-1})/\ell_t$; backward: $x_t \leftarrow (v_t + z_t - c_{t+1}x_{t+1})/\ell_t$
4: $\log|Q| \leftarrow \sum_t [\log w_t + \operatorname{log1p}(q_t/w_t)]$ ($\log q_t$ where no next edge); min_pivot $\leftarrow \min_t p_t$

Replay costs one host copy of the start state per executable call and can be disabled (`Runtime(replay=False)`); no chain is ever dropped. The replay of an equation failure after the production CCCM step originally raised a `TracerArrayConversionError` on host arrays; it now converts the state to device arrays first.

Given the draw, the path up to $T + h - 1$ and the drawn λ_{T+h}, $y_{T+h} \sim \text{N}(m, LL')$ exactly.

Algorithm C.9 Burn-in and sampling bodies of one chain (`engines/mcmc/runner.py`, `_bodies`). Counters are uint32 throughout (erratum E7).

BURN$(k_c, \text{state}, it_0, n)$:
return `fori_loop` over $r = 0, \ldots, n-1$ of SWEEP$(k_c, it_0 + r, \cdot)$ ▷ traced trip count: no recompile across burn sizes
SAMPLE$(k_c, \text{state}, it_0, \text{thin}, \text{stop})$:
`scan` over $j = 0, \ldots, \text{chunk} - 1$ with carry (state, snap):
  base $\leftarrow it_0 + j \cdot \text{thin}$; run thin sweeps SWEEP$(k_c, \text{base} + r, \cdot)$
  snap $\leftarrow$ state if $j = \text{stop}$ ▷ exact state at the last kept draw
  record monitors and the parameter draw; optionally run the forecast core on selected draws (Algorithm C.12)
return (state, snap, records)

Algorithm C.10 Host driver (`runner.run_chains`).

with float64 and one OpenBLAS thread: check the static configuration and burn $+ \lceil \text{draws/chunk} \rceil \cdot \text{chunk} \cdot \text{thin} < 2^{32}$
chain keys $\leftarrow$ `fold_in` chain over (seed, `Stream.MCMC`, data salt, chain); init keys likewise from `Stream.INIT`
cache $\leftarrow$ PREPARE(data, prior); state $\leftarrow$ INIT, or a remapped warm state
plan storage from the abstract record shapes; raise `StorageBudgetError` before sampling if over budget
while $it <$ burn **do**
  $n \leftarrow \min(\text{burn} - it, \text{chunk} \cdot \text{thin})$; copy the start state to the host if replay is on
  state $\leftarrow$ BURN$(\cdot, it, n)$; CHECK (Algorithm C.11); $it \leftarrow it + n$
end while
zero the accumulators
for each sampling chunk c **do**
  $it_0 \leftarrow \text{burn} + c \cdot \text{chunk} \cdot \text{thin}$; host copy if replay is on; SAMPLE; CHECK; append the kept records
end for
final state $\leftarrow$ snapshot after exactly burn $+$ draws $\cdot$ thin sweeps; acceptance rates and latent moments from the accumulators

Algorithm C.11 Failures as values, and deterministic replay (`runner._Run`, `failure_detail`; `state.encode_error`).

inside compiled code, every block:
ok $\Leftrightarrow$ pivots finite and > 0, draws finite (and block-specific checks); a NaN pivot counts as 0
the sweep keeps the first (code $= 1 + 1000 \cdot \text{block} + \text{eq}$, it) per chain; nothing raises, no jitter is added
on the host, after every executable call (CHECK):
if some chain has a non-zero code **then**
  pick the chain with the smallest failing it; (block, eq) $\leftarrow$ decode(code)
  replay from the host copy of the start state to sweep it ▷ deterministic: keys depend on it only
  re-run the steps of sweep it before the failing one, then evaluate the failing block's exposed conditional: dimension, non-finite entries, non-positive diagonal, minimum eigenvalue, weight and prior-precision ranges; for a global step, locate the failing column
  raise `MCMCNumericalError`(block, equation, iteration, detail, chain)
else if a state invariant is violated without a code **then**
  bisect the replay for the first offending sweep
end if

Algorithm C.12 Forecast core for one parameter draw (`engines/mcmc/forecast.py`, `forecast_core`).

Input: draw, last p observations, shocks $\varepsilon_{1:H}$ and volatility, mean and global keys

1: $(\log \lambda_i^{\text{pers}}, \log \lambda_i^{\text{eff}})_{T+1:T+H} \leftarrow$ `draw_ahead_eq`: SV $h_{T+j} = \mu + \phi(h_{T+j-1} - \mu) + \sigma\eta_j$; one uniform per period for o; $\log \tau = \log(c(\nu)/2) - \text{loggamma}(\nu/2)$
2: **for** $h = 1, \ldots, H$, carrying the lag window **do**
3: $x \leftarrow$ regressors from the window at $T + h$
4: $m \leftarrow A^{-1}\Gamma x$ (structural) or $\Pi' x$ (reduced) ▷ unit-triangular solve
5: $L \leftarrow A^{-1}\,\text{diag}(\exp(\frac{1}{2}\log \lambda^{\text{eff}}_{\cdot,T+h}))$ ▷ conditional Cholesky factor at no extra cost
6: $y \leftarrow m + L\varepsilon_h$; shift y into the window
7: **end for**
8: **return** paths, cond_mean $= m$, cond_chol $= L$, scales

Forecast keys derive from (forecast seed, chain, kept-draw index), so in-loop and post-hoc forecasts agree, to 10^{-12} in the tests.

Structural SV(outliers, t): BVAR(Minnesota(structural, eb), SV(outliers,t), p=2)

```
>>> print(fit.plan.describe())
phase     step        block            writes
equation  equation    EQ_MEAN(0)       eq.mean, eq.struct.a
                                       (δ_i, a_i) | λ_eff,i joint Gaussian (fused contemporaneous draw)
equation  residual    EQ_MEAN(0)       –
                                       e_i = ỹ_i + Σ_{j<i} a_ij ỹ_j − g_i from the new draws
equation  vol         EQ_VOL(2)        eq.vol
                                       VolBlock.sample_eq: o → p → ν → τ, then s → h → θ_SV (atomic)
post      (none)


read sets (Step.reads of the same plan):
  equation    reads eq.coef, eq.mean, eq.struct, eq.vol, glob
  residual    reads eq.mean, eq.struct, glob.mean
  vol         reads eq.vol, glob.vol
```

Reduced-form SV: BVAR(Minnesota(reduced, eb), SV, p=2)

```
>>> print(fit.plan.describe())
phase     step        block            writes
pre       cccm        G_CCCM(16)       glob.mean.Pi
equation  contemp     EQ_CONTEMP(1)    eq.struct.a
                                       a_i | g_i, λ_eff,i: regress ỹ_i − g_i on −ỹ_{<i}
equation  residual    EQ_MEAN(0)       –
                                       e_i = ỹ_i + Σ_{j<i} a_ij ỹ_j − g_i from the new draws
equation  vol         EQ_VOL(2)        eq.vol
                                       VolBlock.sample_eq: o → p → ν → τ, then s → h → θ_SV (atomic)
post      (none)


read sets (Step.reads of the same plan):
  cccm        reads eq.struct.a, eq.vol, glob.mean.Pi
  contemp     reads eq.coef, eq.mean, eq.struct, eq.vol, glob
  residual    reads eq.mean, eq.struct, glob.mean
  vol         reads eq.vol, glob.vol
```

Figure C.1: The Gibbs plan as data: the output of `fit.plan.describe()` (each step's block and write set) for a structural SV(outliers, t) model and a reduced-form SV model, fitted to the simulated data of Listing 2, followed by each step's declared read set (`Step.reads`). The mixture indicators are local to the `vol` step, so no step runs between the indicators and h.

The contract tests run for every registered analytic model and for 12 MCMC compositions: structural × {SV, SV(outliers), SV(t), SV(outliers, t), SV(rw), Constant(ig), Constant(iw), Constant(iw_equivalent)} and reduced × {SV, SV(outliers), Constant(ig), Constant(iw_equivalent)}. Reduced × SV(t), reduced × SV(rw) (the primary arm of the replication) and reduced × SV(outliers, t) are not in this list. [**TODO:** TODO 3: row 18, `reduced_sv_rw` (reduced form × SV(rw)), and a `reduced-SV(rw)` contract fit were committed in 7f410dc, after the acceptance commit c823d20; the SV(rw,

Table C.2: The predictive contract: fields of `ForecastResult`, which wraps an `xarray.Dataset`. The accelerator variables are carried by MCMC forecasts only, are ignored by `check_structure` and by every consumer, and are never contract fields.

Field	Dimensions	Meaning
`paths`	(chain, draw, horizon, variable)	joint simulated paths on the transformed scale; each slice is one path from one parameter draw
`paths_level`	same	pathwise inverse of the tcodes to raw levels (Table A.2)
`cond_mean`	same	$m_{s,h}$: mean of y_{T+h} given the draw, the simulated path to $T+h-1$ and λ_{T+h}
`cond_chol`	(…, variable, variable_)	$L_{s,h}$: lower-triangular factor of the conditional covariance
attrs (all engines)	—	`origin`, `freq`, `model`, `engine`, `spec`, `spec_hash`, `tcodes`, `seed`, `converged`, `warnings`, `rai_bvar_version`; analytic adds `mh_seed`
attrs (MCMC)	—	`ordering`, `n_chains`, `mixture`, `exact`, `transitory`, `outlier_K`, `t_scale`, `fseed` and the fit's reproducibility metadata (Appendix G)
Accelerator variables (MCMC only; not contract fields)		
`cond_A`, `cond_scale`	(chain, draw, variable, variable_), (chain, draw, horizon, variable)	A and the effective scales, so that cond_chol $= A^{-1}\,\mathrm{diag}$(cond_scale); $\mathrm{O}(n^2)$ factored densities
`cond_scale_persistent`; `log_p`, `log1m_p`, `nu`	(chain, draw, horizon, variable); (chain, draw, variable)	inputs of the opt-in transitory Rao–Blackwellisation (`rb_transitory=True`)
`lag_coefs`	(chain, draw, lag, variable, variable_)	A_l of each draw, used by the path-integrated estimator at $h > 1$ (D-34)

Invariants. L^{-1}(paths$-$cond_mean) is iid $\mathrm{N}(0, I)$; draws are joint across horizons and variables; stacking (chain, draw) gives the shape (draws, H, n) in chain-major order. Analytic forecasts carry no `ordering` attribute; their ordering is the `variable` coordinate.

t) variant is still not covered. If the release commit includes it, update this list, Appendix D.3 and the main-text caveats.]

D Validation detail

This appendix gives the configurations, statistics and run records behind Section 8. The tables here are to be generated on the release commit, from the JSON records that the Geweke, SBC and mutation tests write when `RAI_BVAR_VALIDATION_RECORD` points to a directory and, for the tiers without such a hook, from the test run's output (Appendix G). Where a number below was reproduced on an earlier commit, the commit and the machine are stated. [**TODO:** TODO 7: record G-small, G-full, SBC, composition-level B, the `exact=False` rows, `parity_long`, the analytic twins and the mutation suite on the release commit (Apple M5 Max and x86-64 Linux) and commit the JSONL files.]

D.1 Geweke tests at the per-PR size

The G-small configuration runs $C = 1000$ independent successive-conditional chains of $M_c = 100$ sweeps, each started at a prior draw with no burn-in, against $M_1 = 2 \times 10^5$ iid marginal-conditional draws, with family level $\alpha = 10^{-3}$, t_{C-1} critical values and the sequential wrapper of Gandy and Scott (2021) with $k = 3$ stages and multiplier $\Delta = 4$. The test data have $T_{\text{eff}} = 20$, never a multiple of the 32-row bucket, $n = 2$ in structural and $n = 3$ in reduced form (with the design table's overrides: $n = 3$ for row 205 and $n = 1$ for the univariate row 17), $p = 1$ and an intercept. Table D.1 gives the result for each of the 15 per-PR compositions. On commit 62d15f6 (Apple M5 Max) every composition passed: the largest final-stage $|z|$ was 3.03 against critical values of 4.10–4.50; row 201 went to a second, fresh and larger stage after a first-stage Bonferroni q of 0.0012 ($|z| = 4.21$ on a log-scale data autocorrelation) and passed there with $q = 1.0$; and the minimum relative numerical efficiency per composition was 0.008–0.129. The sequential design bounds the probability of a false rejection by $\sum_i \gamma^{i-1}\beta_i = \alpha$, with $\beta_1 = \alpha/k$, $\gamma = \beta_1^{1/k}$ and $\beta_{i+1} = \beta_i/\gamma$; a stage fails if $q_i \le \beta_i$, passes if $q_i > \gamma + \beta_i$, and otherwise hands the decision to a new, independent experiment, enlarged once by Δ. At the per-PR settings $\gamma = 0.0693$ and $\beta = (3.33 \times 10^{-4}, 4.81 \times 10^{-3}, 0.0693)$, so a correct sampler is rerun at four times the size with probability at most about 7%.

Table D.1: G-small Geweke results for the 15 per-PR compositions: number of test functions G, t_{C-1} critical value, maximum $|z|$ and its argument, Gandy–Scott stage and Bonferroni q, minimum relative numerical efficiency, and time. Generated from `geweke.jsonl` (`pytest tests/geweke -m "geweke and not slow"`) on the release commit.

[**TODO:** Generate with `tab_geweke.py` from the release `geweke.jsonl`.]

D.2 Geweke test functions and the finite-variance rule

The test functions of every composition are drawn from fixed families: every scalar parameter; latent states at fixed dates ($h_{i,0}$, $h_{i,1}$, $h_{i,\lceil T/2 \rceil}$, $h_{i,T}$) and the path means $\text{mean}_t\, h_{i,t}$ and $\text{mean}_t\, |h_{i,t}|$; squares where their variance is finite; cross-products within blocks ($a_{ij}\gamma_{i,(\text{lag}\,1,j)}$, $\mu\phi$, $\phi\sigma$, $p\,N_1$, $\nu \cdot \text{mean}\log\tau$); cross-equation products of reduced-form coefficients; the exact log-likelihood given the latent states and the log prior density $\log p(\theta)$; the standardised residual sums $\sum_t e_{it}^2/\lambda_{it}$; and data functions. The number of functions per per-PR composition ranges from 22 (univariate SV) to 134 (reduced form with SV and outliers). Geweke's statistic is asymptotically normal only if $\text{var}\, g(\theta, y) < \infty$ (Geweke, 2004). Under the SV test prior $\text{E}[y^2] = \text{E}[e^h]$ is infinite, because σ^2 has a χ^2 tail and ϕ has prior mass near 1; under IG(3, 2) the variance of λ^2 is infinite; and with Student-t errors $\text{E}[y^{2k}]$ is finite only for $\nu > 2k$. The rules adopted are therefore: under SV only log-scale or bounded data functions ($\text{mean}_t \log y_t^2$, sample correlations, first autocorrelations, $\text{mean}_t \tanh y_t$); $\log\lambda$ in place of λ^2; and a Constant test prior IG(5, 4). Each reference function carries a finite-variance flag.

The test priors are proper and tight, $M \neq 0$ so that D3 centring is exercised, $a_{ij} \sim \mathrm{N}(0, 0.7^2)$, the outlier probability is Beta(2, 18) so that outliers occur at $T = 20$, $\nu \in \{5, \ldots, 12\}$, and the in-window recomputation of hyperparameters is off, so that no hyperparameter depends on y. The size of the test needs only that the sweep leaves the posterior invariant: a chain started at a prior draw is stationary from its first sweep under the null, and its mean is an unbiased estimate of $\mathrm{E}_p[g]$; ergodicity affects only power. At $G = 100$ and $\alpha = 10^{-3}$ the normal critical value would give an actual size of 1.64×10^{-3} at $C = 200$ and 1.11×10^{-3} at $C = 1000$, which is why the t_{C-1} quantile is used.

D.3 G-full, the approximate rows, Tier B and SBC

G-full uses $C = 4000$, $M_c = 500$, $M_1 = 10^6$ and $\alpha = 10^{-2}$ at $n = 3$, $p = 2$ and $T_{\mathrm{eff}} = 50$ (rows 14–16: $n = 2$; row 17: $n = 1$); the eleven `exact=False` rows (the auxiliary-mixture sampler) are recorded but do not gate, because that sampler targets the mixture approximation rather than the exact posterior; only a non-finite test function, which is a bug whatever the approximation, fails them. Composition-level Tier B consists of block-conditional SBC ($N = 200$, $L = 99$) of every exact-draw block, with ranks of the block's coordinates and of the exact log-likelihood, and one-block invariance ($N = 1000$ joint draws, $L = 20$ applications of the live blocks, two-sample Kolmogorov–Smirnov tests) for Metropolis–Hastings, ASIS and Gibbs-scan blocks; its 43 tests run weekly, while each kernel suite runs its own Tier B per PR. SBC uses $N = 1000$ replications, $L = 99$ kept draws, burn-in 300, a pilot of 1000 sweeps, $t_{\max} = 200$ and $\alpha = 10^{-2}$ for the 22 compositions, inside the same sequential wrapper. [**TODO:** TODO 3: row 18, `reduced_sv_rw` (reduced form × SV(rw)), and a `reduced-SV(rw)` contract fit were committed in 7f410dc, after the acceptance commit c823d20; the SV(rw, t) variant is still not covered; NOTES item 6 records that it passes the ESS rule only with a per-row $t_{\max}$ of 400. If the release commit includes it, update the composition count and the cap here.] Each replication draws $(\tilde{\theta}, \tilde{y})$ from the joint model, applies only the data filter $\mathrm{finite}(\tilde{y}) \wedge \max|\tilde{y}| < 10^3$, runs a pilot chain from an independent prior draw, sets its thinning to $t_i = \min(\lceil 2\,\mathrm{IAT}_i \rceil, t_{\max})$ with the Geyer initial-monotone IAT (Geyer, 1992) of the quantities and of their tail indicators, and then keeps L draws from a fresh chain; ranks are uniform conditionally on $\tilde{y}$, so data-dependent thinning preserves uniformity. Uniformity is tested with the γ statistic of Säilynoja et al. (2022), whose null distribution is computed exactly by a forward recursion that accumulates the mass leaving the interior at each step, so that p-values in the far tail keep their relative accuracy. The quantities include the parameters, h at fixed dates and its path means, the total and per-equation log-likelihood, $\log p(\theta)$ and the predictive unknowns h_{T+1} and y_{T+1} through the production forecast core; data-dependent quantities are needed because a "posterior" equal to the prior passes SBC on every data-free quantity (Modrák et al., 2025).

The ESS erratum. The design asked for a per-replication thinned ESS of at least $0.8L$ in 99% of replications. With $L = 99$ kept draws that cannot be certified for a single replication: even with iid draws the Geyer ESS of the kept draws reaches $0.8L$ only in about three replications out of four. The per-replication ESS is therefore predicted from the pilot IAT with an AR(1) form, $\rho_i = (\mathrm{IAT}_i - 1)/(\mathrm{IAT}_i + 1)$ and $\mathrm{ESS}_i = L(1 - \rho_i^{t_i})/(1 + \rho_i^{t_i})$, which is at least $0.96L$ whenever t_i is not capped, and a second gate requires the replication-pooled ESS of the kept draws, computed from autocovariances of ECDF indicators pooled over replications (a level is used only when Kish's effective number of replications is large enough), to be at least $0.8L$ for every quantity. [**TODO:** TODO 25: commit a small test for the 76% and $0.836L$ figures, or keep them out.] Table D.2 collects the G-full, `exact=False`, composition-level B and SBC records. No committed record of these runs existed at 62d15f6; the only committed numbers are those that the repository notes record for row 205 (G-full 2.76, SBC minimum p 0.018).

Table D.2: G-full, the recorded `exact=False` rows, composition-level Tier B and full SBC per composition: maximum $|z|$ or minimum p, stages, ESS gates and the number of replications at $t_{\max}$. Generated on the release commit.

[**TODO:** Generate with `tab_geweke_full_sbc.py` from the release records; until then no G-full or SBC result other than row 205 is stated.]

D.4 Analytic twins and exact oracles

Table D.3 lists the twins A1–A7 with their oracle, statistic, tolerance, negative control and cadence. Two methodological details matter for reading it. First, the twins compare quantiles in CDF form: each exact quantile x_q is tested by $z = (\hat{F}_{\text{MCMC}}(x_q) - q)/\sqrt{\text{MCSE}^2(\hat{F}) + q(1-q)/N_{\text{ref}}}$, with the indicator's own ESS. To first order this equals the quantile-MCSE test, but it avoids the density estimate of the order-statistic MCSE, whose noise made the usual statistic heavy-tailed at $n = 20$; the decision rule is $|z| \leq \max(4, z_{\text{Bonf}})$ at family level 10^{-2}. Second, the twins carry negative controls whose rejection is part of the test: for example, A2 rejects an NIW posterior with one extra degree of freedom (a change of about 1.5% in Σ), and A5 rejects an inverse-gamma shape off by $\frac{1}{2}$ (about 1.6% in $\text{E}[\lambda_i]$). The quadrature oracle of A5 checks itself on every call (doubling the rule and widening the interval changes nothing above 10^{-10}), and with a λ-scaled prior on all coordinates it must reproduce the closed form of Chan (2022). The twins are also the tier that caught a missing term the dense oracles of the coefficient step did not: in Mode II with dummy rows, the λ step must count the rows (Appendix B.5).

Table D.3: Analytic twins and exact oracles A1–A7: oracle, statistic and tolerance, negative control and its measured power, cadence, and result on the release commit.

[**TODO:** Generate with `tab_twins.py` from the release junit XML.]

D.5 Parity with independent implementations

Parity runs both implementations on the same committed datasets with one parameter set from which both sides' settings are derived, and compares the 5, 25, 50, 75 and 95% quantiles with $z = (\hat{q}_{\text{ours}} - \hat{q}_{\text{ref}})/\sqrt{\text{mcse}^2_{\text{ours}} + \text{mcse}^2_{\text{ref}}}$, passing if $|z| \leq \max(4, z_{1-0.01/(2m)})$ over the m comparisons; the MCSE is the larger of the quantile MCSE of Vehtari et al. (2021) and a batch-quantile MCSE. Table D.4 summarises the rows used as evidence in this paper; the parametrisation notes in `tests/parity/` record every convention that had to be matched.

- *Analytic engine.* R BVAR 1.0.5 (Kuschnig and Vashold, 2021) and the Bank of England bvar 0.3.5 (Labonne et al., 2026). The tests assert log marginal likelihoods at fixed hyperparameters to a relative tolerance of 10^{-8}; the recorded agreement is below 10^{-11} for BVAR in the exact cases and at most 1.5×10^{-11} for the Bank of England package. Posterior moments are tested to a relative tolerance of 10^{-8} (the largest recorded deviation is 6×10^{-9}), empirical-Bayes modes to 10^{-3}, and Metropolis–Hastings and forecast quantiles within Monte Carlo error.
- *Univariate SV.* stochvol 3.2.1 (Kastner, 2016; Hosszejni and Kastner, 2021) against `ZeroMean` $\times$ SV with $n = 1$, on the main and default-prior rows of four datasets: maximum $|z|$ 1.63–2.59 at the per-PR size and 1.64–3.57 at the size of the weekly tier, against a bound of 4.00.
- *Standardised Student-t SV.* The exit criterion for this sampler is an exact posterior obtained by self-normalised importance sampling on a four-observation problem, against which the

maximum $|z|$ is 1.83. A small problem with an exact posterior is an oracle that does not depend on any other implementation. [**TODO:** TODO 13: re-test the third-party Student-t and sign-fold observations on the current releases and contact the maintainers before any statement about another package's sampler; until then this appendix states none.]

- *Reduced form.* bayesianVARs (Gruber et al., 2026) could not be built in the test environment (no Fortran compiler), so no finding about it is stated. The exit criterion was met against a substitute: an exact system-wide Gibbs sampler written in R in rai-bvar's own conventions ($\mathrm{vec}\,\Pi \mid A, \Lambda$ jointly, A rows by regression, SV by one sweep of stochvol's sampler at offset 0, or $\lambda_i \mid e$ inverse gamma), which gives maximum $|z|$ 2.76, 3.45 and 3.56 against bounds of 4.05–4.51.

Parity does not establish everything: there is no external VAR-level oracle for structural Mode I × SV, and exact mode has no external oracle beyond the reweighted stochvol rows.

Table D.4: Parity summary: oracle and version, rows, statistic, bound, and maximum $|z|$ or relative tolerance.

[**TODO:** Generate with `tab_parity.py`; neutral wording only (TODO 13).]

D.6 The mutant catalogue

A mutant is injected only through a production seam: a named kernel internal overridden through `compose(static, overrides=...)`, the volatility slot, the sampler's prior arrays, the observables simulator, or the configuration. A tier catches a mutant when its decision rule fails on the mutant and passes on the unmutated control with common seeds; for a U check, the mutant's error must exceed the tolerance and the control's must not, at the same joint draw. `test_registry` requires every mutant to have at least one confirmed catching tier, and a mutant caught by none blocks the release. The U checks use a tolerance of 10^{-8} (M8 and M11: 10^{-12}; M12: 10^{-4}; M15: 10^{-10}); the B checks are the block SBC of π_0 or one-block invariance of the volatility block; G is `geweke_sequential` at G-small; S is a short SBC ($N = 200$, four times that after stage 1, $L = 99$, burn-in 100, pilot 300, $t_{\max} = 30$). The mutant and control statistics are in Table 4; below, each entry gives the seam, the error as a mathematical statement, its source where it has one, and the catching tiers.

M1 (`cccm_core`; reduced Constant.) π_j is drawn from equation j alone, the triangular step of Carriero et al. (2019), instead of the corrected full conditional (Carriero et al., 2022). Caught by U, B (through the log-likelihood rank) and G; one-block invariance of the whole scan does not catch it (recorded; Appendix B.7).

M2 (volatility slot, `exact=False`; structural SV.) Stale indicators: s is carried across sweeps and drawn after h, the order of Algorithm 1 of Primiceri (2005) corrected by Del Negro and Primiceri (2015). Caught by G.

M3 (`JointSpec(fresh_latents=True)`; structural SV.) The observables simulator draws fresh latent states instead of using the current ones, the simulator error of Geweke (2004). Caught by G.

M4 (`tau_conditional`; structural SV(t).) $\tau_t \mid \cdot$ has shape $\nu/2$ instead of $(\nu + 1)/2$. Caught by U and G.

M5 (sampler prior arrays; structural SV.) B_σ is doubled in the sampler only, so the sampler's prior differs from the model's. Caught by G.

M6 (`nc_fold`; univariate SV.) The non-centred step takes $|\sigma|$ without flipping $\tilde{h}$, which is not measure-preserving (Appendix B.3). Caught by U and G.

M7 (`nc_accept`, with the one-component table COARSE1 in mutant and control; univariate SV.) The non-centred move is accepted without the w-correction in exact mode. With OCSN-10 the missing correction is too small to see at test sizes. Caught by U and G.

M8 (`structural_gamma`; structural Constant.) D3 centring is dropped, $\gamma_i = \delta_i$. Caught by U, G and S.

M9 (`outlier_logits`; structural SV(outliers).) The outlier pmf omits the factor $1/k$. Caught by U and G.

M10 (`p_conditional`; structural SV(outliers).) $p \mid o \sim \mathrm{Beta}(a_p + N_1, b_p + T)$ instead of $b_p + T - N_1$. Caught by U and G.

M11 (`indicator_logits`; auxiliary sampler.) The indicator pmf omits $v_j^{-1/2}$. Caught by U, B and G under `exact=False`; in exact mode it is not a bug, and the G check passes there (recorded).

M12 (`mixture_table`; auxiliary sampler.) The effective component means are shifted by -1.2704 twice. Caught by U, B and G under `exact=False`; not a bug in exact mode (recorded).

M13 (`constant_conditional`; structural Constant.) The inverse-gamma shape uses T_{pad} instead of T_{eff}. Caught by U and G.

M14 (`contemp_conditional`; reduced Constant.) The A-row regression uses $+u_{<j}$ instead of $-u_{<j}$. Caught by U and G.

M15 (`nc_step`; univariate SV.) The ASIS map omits the rescaling of $\tilde{h}$. Caught by U, B and G.

M16 (configuration, `exact=False`; univariate SV.) Offset $c = 0.1$ at $\lambda \approx 0.01$ (prior level centred at $\log 0.01$) against a control offset of 10^{-12}. In exact mode a large offset is not a bug, so the mutant is defined for the auxiliary sampler only. Caught by G.

M17 (`linear_conditional`; structural Constant with dummies.) Mode I SOC/SUR rows are weighted by the current $1/\lambda_i$ instead of the fixed $1/\bar{\sigma}_i^2$. Caught by U and G; its G control runs on the dense eigen path (control statistic 1.94; 2.44 on the `cholesky_const` path).

M18 (`cccm_core`; reduced Constant with dummies.) The SOC/SUR rows are dropped from the CCCM step. Caught by U and G.

Coverage is U 14 of 18, G 18 of 18, B 4 (every mutant with a registered B check) and S 1, which is 37 mutant $\times$ tier checks. The four mutants without a U check, M2, M3, M5 and M16, carry `u_check=None` in the registry: they are errors of step order, of the simulator, of a sampler–model prior mismatch and of a model approximation, and no deterministic identity is registered for them. Whether an identity could detect them is not tested; for M5, a density-ratio identity of the σ^2 step against the model prior is a plausible detector. The design expected U catches for nine mutants; the suite confirmed fourteen, and G for all eighteen. The independent x86-64 review also caught four further hand-injected dummy-row errors (Mode II rows weighted $1/\bar{\sigma}^2$, rows counted twice in the λ step, CCCM rows weighted by $(\Sigma^{-1})_{jj}$, a fixed-factor weight $2w$); these are not registered mutants.

Figure D.1 shows, for each mutant and tier, the mutant's statistic relative to the tier's decision threshold, and Figure D.2 places the G statistics of the mutants against the per-composition controls (G-full rows of the validation record and the mutation suite's G-small controls). Statistics agree across machines to two or three significant figures (for M2, 702 and 705 for the mutant, 2.47 and 2.48 for the control); they are not expected to agree bitwise across platforms.

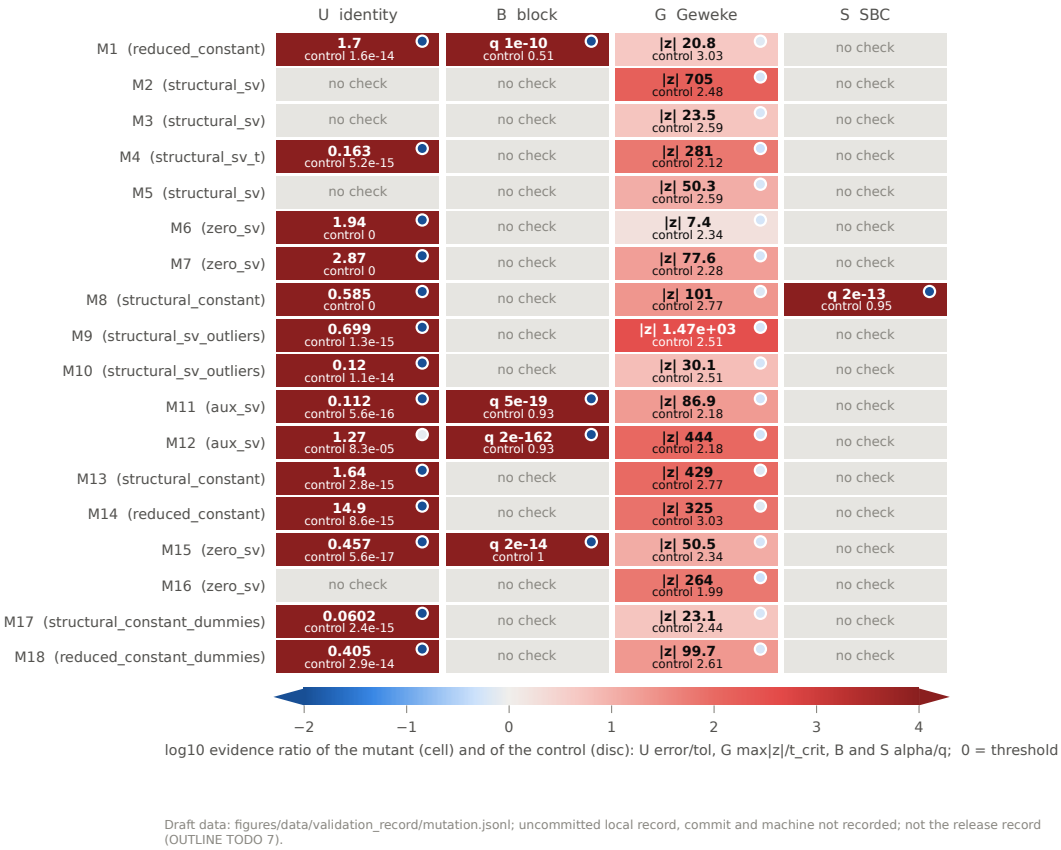


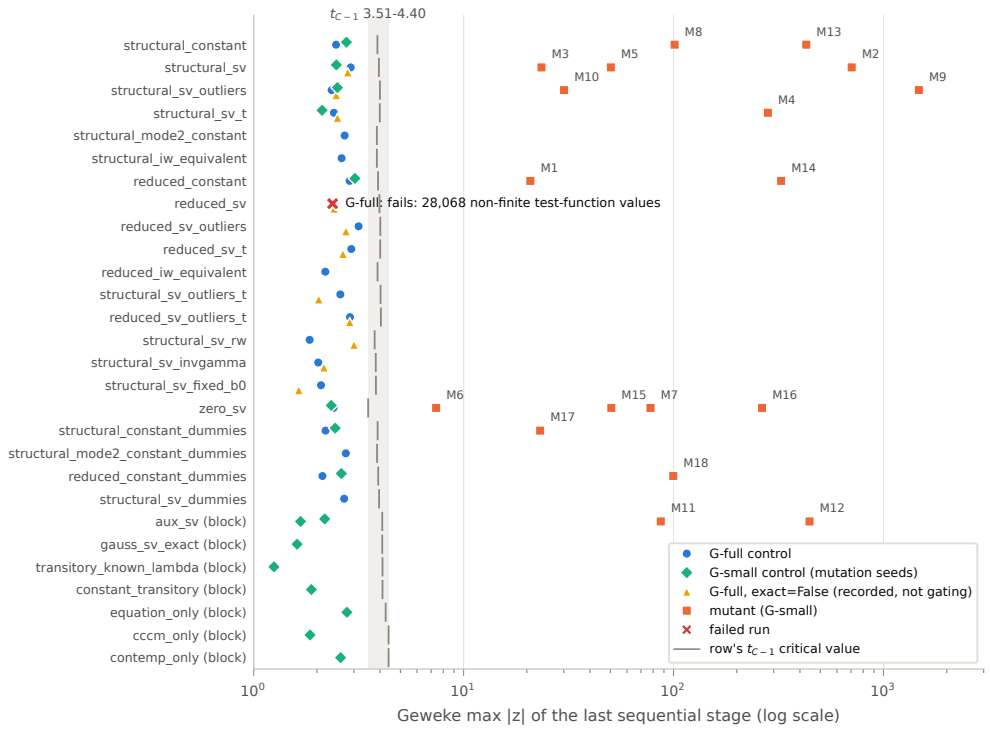
Figure D.1: Mutation power: mutants M1–M18 × tiers U, B, G, S. Cell colour is $\log_{10}$ of the mutant's statistic divided by the tier's decision threshold; a marker shows the control; grey means that no check is registered.

D.7 Runtime convergence diagnostics

The convergence flag is computed by in-repository NumPy code: rank normalisation with normal scores $(r - 0.375)/(S + 0.25)$, split chains, R-hat as the maximum of the bulk and folded versions (Vehtari et al., 2021), ESS with Geyer's initial positive and monotone sequence truncation (Geyer, 1992), and tail ESS, mean ESS and the MCSE of means and quantiles. The diagnostics are tested against reference values from ArviZ 0.23.4 and arviz-stats 1.3.3 (Kumar et al., 2019), which agree with each other to the last digit on every case and are hard-coded in the tests (ArviZ is not a dependency), to a relative tolerance of 10^{-10}; against stationary Gaussian AR(1) chains with closed-form IATs, including the indicator IAT; and by statistical bounds calibrated over 30 seed sets. The gate is decided by the primary monitor set only: at $n = 20$ it has 41 quantities for Constant, 101 for SV, 61 for random-walk SV and 101 for SV with outliers and t, against secondary sets of 211, 251, 251 and 291, which are reported but never gate. A single chain is never converged, because split R-hat cannot see disagreement between chains.

D.8 Suite size and runtimes

At commit 62d15f6 the suite collected 4,964 parametrised test IDs: 4,520 in the per-PR fast suite (`not slow and not network and not bench`), 441 in the CI job named "nightly", which runs weekly, and 3 benchmarks. The harness code in `src/rai_bvar/validation/` has 4,230 lines. On



Draft data: figures/data/validation_record/geweke.jsonl, figures/data/validation_record/mutation.jsonl; uncommitted local record, commit and machine not recorded; not the release record (OUTLINE TODO 7).

Figure D.2: Geweke maximum $|z|$ per composition for the unmutated samplers (G-full rows and the recorded `exact=False` rows of the validation record, and the mutation suite's G-small controls at the mutation seeds) against the t_{C-1} critical band, with the mutants' G statistics overlaid on a logarithmic scale; failed runs are drawn as crosses.

[**TODO:** TODO 7: this draft figure is drawn from local, uncommitted records (`docs/paper/figures/data/validation_record/`) that carry no commit or machine field and hold no registered G-small rows; in them the G-full row of `reduced_sv` failed with non-finite test functions (seed 20261023), which is either the float64 failure that 3d3f51b fixed (a stale record) or a regression; the record cannot say which. Establish their provenance or redraw from the release records.]

an Apple M5 Max with a cold compilation cache and other jobs running (load average 3.5–18), the fast suite gave 4,507 passed, 12 skipped and 1 expected failure, with no failures, in 1,003 s, and 1,006.9 s on a re-run; the skips are six live ArviZ comparisons (ArviZ not installed), three OpenBLAS thread-API tests (not available on macOS Accelerate) and three phase-gating checks. G-small for all per-PR compositions took 94.6 s and 99.7 s in two runs, and the mutation suite 181.9 s and 177.4 s, with identical statistics. The test modules record per-composition times for G-full (34 s to 9 min at the first stage, about 1 h for 17 rows) and for SBC (25 s to 17 min, about 2.5 h for 17 rows run sequentially) on a machine they label M1 Pro, two SBC processes at a time. [**TODO:** TODO 8: fix the machine labels (M1 Pro vs M5 Max) in these docstrings and in the `mutants.py` EVIDENCE header before quoting them.] [**TODO:** TODO 7: replace this paragraph with the counts in `collect.txt` and the timings of the release run; b9c66c0 reports "Fast suite 4655 passed".]

E Numerical robustness and performance detail

E.1 The three float64 failures

Table E.1 summarises the three failures of Section 9. None was a user report: the first two occurred at states that are legitimate under the model and were found by validation or replication runs, and the third was a bug in the failure-reporting path that the second exposed. Status codes decode as 1003 = (`EQ_CONTEMP`, equation 2) and 2001 = (`EQ_VOL`, equation 0).

Table E.1: Float64 failures F1–F3: state, mechanism, how each was found, the fix and its evidence. The lower panel, generated by `tab_rw_pivots.py`, recomputes the minimum pivot, the log-determinant and h_0 of the random-walk AWOL step at $\sigma = 5 \times 10^{-9}$ for the assembled factorisation, the excess-pivot recursion and a 60-digit reference.

	State	Mechanism	Found by	Fix
F1	G-full `reduced_sv`, chain 3214 of 4000, sweep 194; equation 0 at $h \approx 31$–37, $\lvert y\rvert \approx 2 \times 10^8$	equation 2's a-row precision has exact eigenvalues 14.8 and 1.9×10^{17}; forming $Z'WZ$ loses the small one, pivot 0 (code 1003); the conditional itself is benign	Geweke tier (weekly), with a heavy-tailed test prior	Householder QR of the whitened rows (Algorithm C.4); same noise map
F2	first full CR attempt (8a622c5), SV(rw), reduced form, origin 1987Q2, ordering GDP, TBILL, INFL, UR, burn-in sweep 318, $\sigma = 4.98 \times 10^{-9}$	$\sigma^2/v_{s_t} \approx 10^{-17}$ is below the ulp of the Laplacian diagonal (2); the assembled matrix is the singular path Laplacian; level pivot 0 (code 2001)	the replication's 24-ordering stage	excess-pivot recursion (Algorithm C.8, Appendix B.8)
F3	the replay of F2	the replay re-ran the production CCCM step on host arrays: `TracerArrayConversionError`; the first crash's diagnostics were lost	the replay itself	convert the replayed state to device arrays (93f2bcd), with a test on the production program

[**TODO:** Generate with `tab_rw_pivots.py` (pairs with the old-path-fails test of TODO 23).]

The recorded diagnostics of F1 at the real state (the QR draw matches the exact conditional to 1.6×10^{-8} standard deviations) and of the old path of F2 at the test state (minimum pivot 2.22×10^{-16} against an exact 5.308×10^{-15}; log-determinant -36.04 against -32.8695; $h_0 = -73.90$ against -3.024) come from commit messages and scratch computations, not from committed tests. The random-walk `aux_log_likelihood` had the same flaw (17534.8 instead of -1650.8 at $\sigma = 5 \times 10^{-9}$); the fix is tested against the dense Gaussian likelihood under the closed-form random-walk covariance $V_0 + \sigma^2 \min(s, t)$. Each fix was checked for neutrality elsewhere, on one machine. After the F1 fix every structural program was bitwise unchanged (only non-fusing means use the step), and the reduced-form states agreed with the old draw to 3.7×10^{-12} (relative) after 3 Geweke sweeps and 4.1×10^{-10} after 60-sweep fits; after the F2 fix 639 of 656 arrays were bitwise identical (every non-random-walk composition and fit), and random-walk states agreed to 3.3×10^{-11} after 3 sweeps and 6.7×10^{-10} after 60-sweep fits. The rejected alternative for F1, narrowing the test prior's h tail, would have hidden a real numerical weakness and dropped high-persistence coverage. The other formed-Gram draws, the structural coefficient step and the CCCM Π step, still accept any positive pivot, without a relative floor; whether reachable states expose them is not known. [**TODO:** TODO 23: commit tests that assert the old-path failures at the F1 and F2 states, re-run the tiny-σ and replay tests on Linux, and decide on a relative pivot floor.] Figure E.1 shows the level pivot and the error in the level of h as σ decreases, for both factorisations.

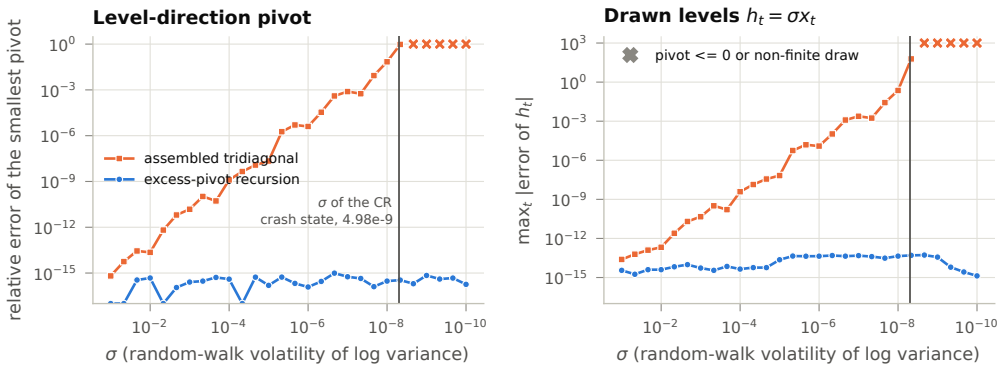


Figure E.1: The random-walk SV path system at small σ: level-direction pivot and error in the level of h against σ, for the assembled factorisation and the excess-pivot recursion, measured against a 60-digit reference, at the synthetic state of `test_rw_awol_at_a_tiny_sigma` (88 periods), not at the production fit of F2.

[**TODO:** TODO 23 (author decision): the figure also shows that at $\sigma \leq 10^{-8}$, where the assembled path does not fail, its pivot and the level of h are badly wrong (relative pivot error up to about 0.95, h off by up to about 61), which Section 9 deliberately does not state. Keep the figure and state this as a recomputation at a synthetic state, or crop it to the failing range, and commit the old-path test of TODO 23 either way.]

E.2 Performance

Three sources of performance numbers exist in the repository, and only the third measures the production engine: the stand-in scratch scripts of the design stage (a simplified SV block, data closed over as constants), the prototypes of the pilot (the real SV block, on a loaded machine), and the BENCH-P2 baseline of the engine itself through `BVAR.fit`. The paper quotes only the third and never blends them. BENCH-P2 fits structural Linear × SV with fixed hyperparameters to a simulated VAR(4) with SV ($n = 20$, $T = 300$; $k = 81$, $q = 100$, $T_{\text{pad}} = 320$), with 4 chains of 1000 burn-in and 10,000 kept draws on one CPU device, and times the whole fit, including compilation, storage and diagnostics, in fresh processes; wall and compilation times are medians over three cold or three warm runs, and the per-sweep time and peak memory are medians over all six. Table E.2 gives the result recorded at commit deb3929 on an Apple M5 Max that was not idle (one-minute load 2–6, desktop applications open): 52.3 s cold and 49.6 s warm, compilation 3.46 s cold and 1.00 s from the persistent cache, 3.90 ms per sweep, and a peak resident set of 3,043 MiB. `converged` was false in all six runs; according to the commit message, 4 of the 101 primary quantities missed R-hat 1.01, while every group's minimum bulk ESS exceeded 400. The slowest-mixing quantities were σ_i and ϕ_i (minimum ESS per second 10.2 and 11.2), against about 400 for the own first-lag coefficients. Table E.3 gives the kernel microbenchmarks at 80 equation–chain lanes, with XLA's flop and byte counts and the design budget lines: the packed Gram halves the flops of the batched oracle and is about twice as fast, and the packed Gram, the precision draw and the SV core together take 3.44 ms of the 3.87 ms sweep. These figures are from one non-idle machine at one commit and are not portable. [**TODO:** TODO 9: re-measure BENCH-P2 on an idle machine on the release commit; add an idle x86 entry and, optionally, the 4-device run.]

Table E.2: BENCH-P2 on the production engine: wall time cold and warm, compilation time, time per sweep, peak RSS, ESS and ESS per second by quantity group, the convergence flag, and the machine, load and commit.

[**TODO:** Generate with `tab_bench.py` from `tests/bench/baseline.json`.]

Table E.3: Kernel microbenchmarks at 80 lanes: packed and batched Gram, precision draw, SV indicators, AWOL draw, exact correction, SV core and sweep; XLA flops and bytes accessed; and the design budget lines.

[**TODO:** Generate with `tab_kernels.py`.]

Not measured on the production engine. The 4-device stretch target (only pilot projections exist), the reduced-form CCCM sampler, SV(t) and SV with outliers and t, Linear × Constant without dummy rows, and an idle x86 machine; the only x86 baseline in the history was measured under heavy load on four cores and is labelled provisional.

One OpenBLAS thread while chains run. On CPU, jaxlib lowers Cholesky factorisations and triangular solves to LAPACK and BLAS calls into SciPy's library, and XLA already runs those calls on its own threads; the engine therefore restricts OpenBLAS to one thread while chains run. The function is a no-op where no OpenBLAS with a thread API is loaded, as on macOS Accelerate, so the reference baseline did not exercise it. The claim that results do not depend on the thread count for factors smaller than 128×128 is recorded in the module docstring, not tested; the committed tests check only the set and restore semantics.

Dense eigen path with dummy rows. For Mode I × Constant with SOC/SUR rows at $n = 20$, $p = 4$, $T = 300$, the dense eigen path took 0.46 ms per sweep against 1.53 ms for the per-sweep Cholesky, on an Apple M5 Max under a concurrent load of 3–5; the two paths agree within the path-versus-path envelope of Section 8.

E.3 Platform caveats

On macOS with NumPy and SciPy on Accelerate, `slogdet` and `det` emitted runtime warnings on two well-conditioned symmetric positive definite tridiagonal matrices of a dense oracle, which the suite's warning filter turned into failures; the oracle now takes the log-determinant from the Cholesky factor, and the code under test is unchanged. A Phase 1 empirical-Bayes test intermittently crashes the interpreter on the same platform; the root cause is unknown and it is not tied to the MCMC code. Linux CI is the reference platform; neither observation is attributed to a particular library.

F Clark–Ravazzolo supplement

This appendix documents the replication of Section 11: the provenance of the acceptance run, its pre-registration, the criteria and deviations exactly as coded, the run history, the data and models, and further tables and figures. Every number of the acceptance run is a placeholder filled from `benchmarks/reports/clark_ravazzolo_acceptance.json`; numbers of the first full run (report commit 551f181, code 4bb978a, path-conditioned estimator, 4×5000 draws) are history only and are marked as such. Line numbers refer to `benchmarks/clark_ravazzolo.py` at commit c823d20.

F.1 Provenance of the acceptance run

The acceptance configuration and its pre-registration were committed together in c823d20, before the run was launched. According to the run record, the run was started afterwards from a detached worktree at that commit with a clean working tree, with the command `benchmarks/clark_ravazzolo.py benchmarks/reports/clark_ravazzolo_acceptance --config acceptance --n-jobs 10` and a fresh score cache named after the commit. [**TODO:** TODO 2: archive the run log with its

UTC start time as evidence that the launch followed c823d20, and commit `clark_ravazzolo_acceptance.{json,md}` from the acceptance worktree.] The script labels a report as the acceptance run only when `acceptance_gaps` (l. 1022) returns nothing: the configuration must be the registered one (estimator, MCMC budget, forecast draws, origin and ordering steps, noise replications, backtest seed 0), every stage must have run (the structural pair and all 24 orderings), and the commit must be known with no uncommitted changes to tracked files. The acceptance report records `config.name` = [ACC: `config.name`], `config.lpd_method` = [ACC: `config.lpd_method`], `config.draws` = [ACC: `config.draws`], `config.mcmc` = [ACC: `config.mcmc`], `config.seed` = [ACC: `config.seed`], `versions.git_commit` = [ACC: `versions.git_commit`] and `versions.git_dirty` = [ACC: `versions.git_dirty`], and whether the JSON has no `pending` key ([ACC: `absent(pending)`]). The values that `acceptance_gaps` requires are `"acceptance"`, `"integrated"`, 8000, 4 chains × 20,000 kept draws with burn-in 1000, seed 0, a known commit and `git_dirty` = `false`, with no `pending` key. The function does not compare the commit with c823d20; that the run is at c823d20, the commit that adds the pre-registration, is a requirement of the pre-registration itself and is checked from `versions.git_commit`. Two sentences of the repository's ledgers that still said the acceptance configuration "keeps 1000 forecast draws" were corrected after the launch, in d58c9ef, which changes documentation only; the script at c823d20 uses 8000 (l. 194), as pre-registered.

F.2 The pre-registration

The pre-registration is reproduced below as committed at c823d20, in the file `benchmarks/reports/clark_ravazzolo_preregistration.md`. Only the typography has changed (Markdown to LaTeX); the words, numbers and structure are those of the file.

> **Clark–Ravazzolo replication: pre-registration of the acceptance run**
>
> Written 2026-10-23, before any score of the acceptance run was computed. The acceptance run is `benchmarks/clark_ravazzolo.py benchmarks/reports/clark_ravazzolo_acceptance --config acceptance` with a fresh cache (`--cache ~/.cache/rai-bvar-cr-<short commit>`; the command is in the script's docstring), on a clean working tree at the commit that adds this file. The report header records that commit and whether tracked files differ from it; the report is labelled the acceptance run only when they do not, every stage ran, and the config, estimator and backtest seed are the ones below. The previous full run (`--config full`, report `benchmarks/reports/clark_ravazzolo.md` committed in `551f181`) is the reference this document departs from.
>
> **Why there is a second run**
>
> The full run met criteria 1, 2, 3 and 5 and missed 4 and 6 (N08 §C5, as coded in `benchmarks/clark_ravazzolo.py`). The two misses had diagnosable causes. Neither was a sign that SV(rw) loses to Constant.
>
> - **Criterion 6** failed only on `compare()`'s convergence gate. The SV(rw) verdict was "better" in 24/24 orderings with the gate off and 0/24 with it on, because SV(rw) converged at 57–82% of origins against the 95% gate.
> - **Criterion 4** failed on one cell: full-sample UR, ΔLPS(h=8) = +1.954 > ΔLPS(h=1) = +0.159. A handful of 2007–08 origins drive it. There the path-conditioned Rao–Blackwellised estimator is degenerate: one forecast draw carries all the weight, and the score is biased low by up to tens of nats, more for the thinner-tailed Constant arm.
>
> These changes are motivated by that failure analysis, so the run is not blind to the previous results. What is pre-registered is that the settings below were fixed before any score of the new run existed, and that the criteria are evaluated on them as coded, whatever the outcome.
>
> **What changes (and only this)**

1. **MCMC budget for every MCMC arm, in every stage:** 4 chains × 20,000 kept draws, burn-in 1000, thin 1 (previously 4 × 5000, burn 1000). With the default `store_max = 2500` the parameters of one in $\lceil 20{,}000/2500 \rceil = 8$ kept draws are stored (10,000 in all), and the forecast draws are taken from those; R-hat and ESS use every kept draw.
    - Evidence: fit-level convergence diagnostics only, with no forecasts or scores. The data are the same and the fit seeds are the CR main-stage ones.
    - SV(rw), identity ordering: 72/106 origins converge at 4 × 5000; 79/106 with burn 5000; 92/106 with 8 chains; **106/106 at 4 × 20,000** (max R-hat 1.0061, min bulk ESS 1763).
    - The two worst orderings of the previous run (INFL > UR > GDP > TBILL and INFL > UR > TBILL > GDP): 71/106 and 63/106 at 4 × 5000 with the main-stage fit seeds; **106/106 and 106/106 at 4 × 20,000** (max R-hat 1.0049 and 1.0046, min ESS 1689 and 1717). The previous run's ordering stage, which fits each ordering with its own backtest seed, had 63/106 and 60/106 there (and 69/106 for the identity ordering against 72/106 in the main stage), so this evidence is at the main-stage seeds, not at the ordering stage's.
    - All 520 + 424 diagnostic fits finished without error. Every base-budget failure is on a σ quantity and is marginal (R-hat < 1.03), consistent with R-hat sampling noise at bulk ESS ∼600, not with a stuck sampler.
2. **Primary log predictive density: the path-integrated Rao–Blackwellised estimator** (`lpd_method="integrated"`; DESIGN, the decision row added with it).
    - For h > 1 it conditions each draw on its parameters and its drawn latent scale path only, and integrates the intermediate y's out analytically.
    - At h = 1 it is identical to the previous estimator.
    - The previous (path-conditioned) estimator is reported as a secondary number in criterion 4's detail, and per cell in the JSON (`dlps_conditioned`, `dlpd_conditioned`); it never enters a rule.
3. **Forecast draws per origin: 8000** (previously 1000), in every stage.
    - Reason: the integrated LPD is exact given each draw but is still an average over draws, and at crisis origins the weights concentrate on a few of them. In the review of the estimator (2008Q2, UR h = 8, at the budget of item 1; precision only, no criterion evaluated), Constant's 1000-draw value sat on average about 0.3–0.6 nats below the 80,000-draw value, with an sd of 0.75–1.06 nats across thinning offsets. SV(rw)'s varied with sd ≈ 1.4 nats across forecast seeds, more than its MCSE, and its expected bias was as large as Constant's, so the error favours neither arm.
    - With 8000 draws the thinning error dropped to an sd of 0.09–0.19 nats with bias ≈ 0.
    - It does not remove the MCMC component: the 80,000-draw values still differ by about 1.3 nats across fit seeds at that origin.
    - The cost is small next to the fits.

What does not change

- **Data:** FRED-QD/FRED-MD vintage 2026-09 with the same SHA-256 checks, the four series and their transformations, and the estimation start 1965Q1.
- **Models:** Constant(iw_equivalent), SV(rw), SV, SV(rw,t10), AR-SV(0.9), and the structural pair. The mean prior is `Minnesota(form='reduced', hierarchy='fixed', lambda_=0.2, alpha=2, cross=1, mean=0)` with p = 4.
- **SV priors:** the library defaults. CR's own RW-SV σ^2 prior IG(1/2, 0.0175) is **not** adopted. It stays a documented deviation, and adopting it now would be a modelling change decided after seeing results.
- **Origins and horizons:** origins 1985Q1–2011Q2 (106) with the 1985Q1–2007Q4 pre-crisis subset (92); expanding window; horizons 1, 2, 4, 8.
- **Seeds:** backtest seed 0, per-origin fit seeds (DESIGN §11.6), cold starts; the ordering stage with all 24 orderings and the same noise replications.
- **Rules:** the criteria 1–6 rules and thresholds exactly as coded. That includes the 95% convergence gate of `compare()` (R-hat < 1.01 and bulk ESS > 400 on every primary quantity) and the RB-degenerate flag (joint `lpd_mcse` ≥ 0.9; descriptive only).

Stated in advance

- **Criterion 4 may still fail on UR.** The WP23 review's estimate with corrected tail origins was UR h = 8 ≈ +0.23 against +0.157 at h = 1, which breaks the coded rule ΔLPS(h=8) < ΔLPS(h=1). If it fails, it is reported as a difference from CR, attributed with evidence where possible (final vintage instead of real-time data, Chan's iw_equivalent baseline, library-default SV priors). It is not tuned.
- **Criterion 6** is expected to pass if the convergence gate is met. Its h = 1 scores are estimator-independent.
- **The integrated LPD still carries Monte Carlo error at tail origins** (item 3): about 1.3 nats per arm at a crisis origin from the MCMC itself. That amounts to a few hundredths of a nat in a 106-origin mean, but it is not zero. The RB-degenerate flag stays descriptive.
- **No further run with different settings** is made after this one's scores are seen, except to fix a bug. A bug fix is documented, and the affected stages are rerun in full with these settings and a fresh cache (the score-cache keys do not change with the code).

The fit-level diagnostic that the first item cites is not yet committed, so its numbers are those of the pre-registration (Table F.7). [**TODO:** TODO 4: commit the diagnostic script and its output (e.g. `benchmarks/cr_convergence_diagnostic.py` and `benchmarks/reports/cr_convergence_diagnostic.json`); an uncommitted copy of a script and of 520 + 424 fit records sits under `docs/paper/figures/data/convergence_diagnostic/` in the paper worktree.]

F.3 The criteria as coded

The six criteria of the replication plan are qualitative; the script turns them into the operational rules below, which are the project's own choices (for example, "about 1" became ≤ 1.02 and "systematically" became at least 3 of 4 joint horizons or 8 of 16 cells). The rules are quoted from the `rule` strings of `criteria` (l. 666), with typography changed only.

1. "ΔLPS(SV(rw) − Constant) > 0 at h = 1, 2 for all four variables in the full and pre-crisis samples, and the joint ΔLPD at h = 1 > 0 with one-sided DM/HLN p < 0.10 in both samples"
2. "CRPS ratio SV(rw)/Constant < 1 at h = 1, 2 for all four variables (full sample)"
3. "RMSE ratio SV(rw)/Constant ≤ 1.02 ('about 1') at h = 1, 2 for all four variables (full sample); CR's gains for GDP and the bill are 5–10%"
4. "full sample: ΔLPS(h=8) < ΔLPS(h=1) for UR and TBILL; pre-crisis: ΔLPS(h=8) > 0 for GDP and INFL"
5. "neither SV(rw,t10) nor AR-SV(0.9) is better than both SV(rw) and SV (one-sided DM/HLN p < 0.10, positive ΔLPD) at ≥ 3 of 4 joint horizons or ≥ 8 of 16 marginal cells (full sample)"
6. "in every one of the 24 orderings: compare() verdict SV(rw) vs Constant at h = 1 is 'better', and the one-sided DM/HLN p-value of the marginal log score is < 0.10 for GDP and TBILL at h = 1"

Two features of the coded rules are worth stating. Criterion 4's rule compares the $h = 8$ gain with the $h = 1$ gain only, whereas the qualitative wording is "shrinks or turns negative". Criterion 6 is met only if the verdict is "better" in every ordering both with the convergence gate on, as `compare()` returns it, and with the gate off, and if both p-values are below 0.10 in every ordering. The outcome of each criterion and the numbers behind it are [ACC: `criteria[*].met`] and [ACC: `criteria[*].detail`]; under the integrated estimator, criterion 4's detail also reports the path-conditioned values as a secondary number.

F.4 Deviations from the target

The report prints the following list, quoted from `DEVIATIONS` (l. 1380), followed, for the integrated estimator, by `LPD_INTEGRATED_DEVIATION` (l. 1372). References to sections are to the repository's replication plan (N08) and design documents.

1. "Not a deviation from CR, but from the library default: every own-lag prior mean is 0, as in CR (WP App. 6.1), including the UR and TBILL levels; the library warns that this contradicts their level transformation codes (it would centre them on 1)."

2. "Pseudo real time on the final FRED-QD 2026-09 vintage instead of CR's RTDSM real-time vintages and second-release actuals (D4; §C2). Score levels differ; rankings are expected to be robust (§C6)."

3. "Estimation starts in 1965Q1, not 1955Q1 (FRED-QD starts in 1959; §C2)."

4. "Inflation uses the chain-type GDPCTPI (FRED-QD has no GDPDEF; §C6)."

5. "Baseline: Constant('iw_equivalent') (Chan 2022) stands in for CR's normal-diffuse VAR (§C3); the SV arms use the library's default SV priors except the two CR variants (no prior was tuned). CR's RW-SV σ^2 prior IG(1/2, 0.0175) and its 36-observation training prior for h_0 are not used (the latter would violate R9, §C6)."

6. "AR-SV variant: CR's $a_1 \sim \mathrm{N}(0.9, 0.05^2)$ is mapped to $(\phi{+}1)/2 \sim \mathrm{Beta}(71.25, 3.75)$ (same mean and sd on ϕ); the SV level μ keeps the library's in-window data-centred prior instead of CR's $a_0 \sim \mathrm{N}(0, 0.4^2)$."

7. "LPD: the primary score is the Rao–Blackwellised predictive density; CR's Gaussian approximation is reported alongside (§C4, §C6)."

8. "Significance: one-sided DM/HLN p-values are descriptive under an expanding window; compare() therefore decides 'better' through the Model Confidence Set (SPEC §9.5)."

9. "compare() turns a verdict 'inconclusive' when fewer than 95% of origins converged (R-hat/ESS targets of DESIGN §8.2); verdicts are reported with that gate on (as returned) and off."

   `LPD_INTEGRATED_DEVIATION`: "LPD estimator: the primary score is the library's path-integrated Rao–Blackwellised estimator (`lpd_method='integrated'`, DESIGN §7.3, D-34), not the path-conditioned SPEC §7 default; at h = 1 the two are identical, so every h = 1 cell and criterion 6 (h = 1 only) are estimator-independent. The path-conditioned ΔLPS is reported for the criterion-4 cells only, as a secondary number (never in a rule)."

The Gaussian-approximation log score of item 7 (Adolfson et al., 2007) equals, up to an additive constant, $-\frac{1}{2}$ times the Dawid–Sebastiani score, which is proper but not strictly proper (Dawid and Sebastiani, 1999; Gneiting and Raftery, 2007); it is a parity metric only.

F.5 Run history and the first attempt's crash

Table F.1 gives each run with its commit, configuration, machine and load, wall time and outcome. In order: a quick run with the library's default MCMC settings (4×2000), on a shared four-core x86-64 virtual machine, main stage only, labelled as not the acceptance run; the first full attempt on 8a622c5 (4×5000), which aborted in the SV(rw) ordering stage on F2; the fixes 93f2bcd (failure replay) and f60d736 (excess-pivot recursion); the second full run on 4bb978a, which contains both fixes and the QR a-row draw 3d3f51b, report committed in 551f181; the follow-ups of an independent review of f60d736 in 4fc0162 (including the random-walk auxiliary likelihood), committed after the second run had started and therefore not part of it; the path-integrated estimator in b9c66c0; the

pre-registration and acceptance configuration in c823d20; and the acceptance run. The second full run took 3950 s (1.10 h) from a fresh cache with six worker processes, while weekly-tier jobs ran alongside. It differs from the aborted attempt in 41 files, not only by the fix: the dummy-row work, the QR a-row draw, the replay fix and the random-walk fix all entered between the two, so the second run is not described as differing "only by the fix". The acceptance run's wall time and stage times are [ACC: `seconds`]; its machine and versions are [ACC: `versions`].

The crash of the first attempt unfolded as follows. After the main stage and the Constant orderings had finished, an SV(rw) fit (reduced form, origin 1987Q2, ordering GDP, TBILL, INFL, UR) raised `MCMCNumericalError` in `EQ_VOL`, equation 0, chain 0, at burn-in sweep 318, with code 2001 and pivot 0. The deterministic replay that should have attached diagnostics itself raised (F3), so the state was reconstructed by a separate replay, which showed $\sigma = 4.98 \times 10^{-9}$ for GDP. The fix f60d736 replaced the assembled factorisation by the excess-pivot recursion, and the second full run, on 4bb978a, completed all 24 orderings without a crash. An independent review of f60d736 found no defect; its follow-ups (4fc0162) came after that run had started and left the 656-array invariance set bitwise unchanged, so the run's report stands. [**TODO:** TODO 29: decide whether to say that the committed quick report, which predates f60d736, may have been affected by the old random-walk path; cite quick-run numbers only as context.]

Table F.1: Run history of the replication: commit, configuration, machine and load, wall time and outcome of each run, from the quick run to the acceptance run.

[**TODO:** Generate with `tab_runs.py`.]

F.6 Data and models

Tables F.2 and F.3 give the data and the model arms as coded.

Table F.2: Data of the replication. Rates are non-annualised quarterly rates; the estimation sample starts in 1965Q1 (one raw quarter earlier is kept for the difference).

Variable	CR definition	FRED-QD series	Transformation in the script
GDP	real GDP growth, $100\,\Delta\log$	`GDPC1`	$100\log$ level, then tcode 2
INFL	inflation in the GDP price index, $100\,\Delta\log$	`GDPCTPI`	$100\log$ level, then tcode 2 (chain-type index; FRED-QD has no deflator series)
UR	unemployment rate, quarterly average of monthly values	`UNRATE`	level, tcode 1
TBILL	3-month Treasury bill rate, quarterly average	`TB3MS`	level, tcode 1

Sample. Vintage 2026-09 of FRED-QD and FRED-MD, SHA-256 checked (Appendix G); estimation from 1965Q1; expanding window; origins 1985Q1–2011Q2 (106) and the pre-crisis subset 1985Q1–2007Q4 (92); horizons 1, 2, 4, 8; $p = 4$.
Run-time checks. (i) FRED-QD `UNRATE` and `TB3MS` equal the within-quarter means of the FRED-MD monthly series of the same vintage (tolerance 0.01 over more than 200 quarters; first full run: maximum absolute difference 3.3×10^{-5} over 270 quarters; acceptance run: [ACC: `data_checks.aggregation`]). (ii) No look-ahead: at origins 1985Q1, 2007Q4 and 2011Q2 the resolved prior, hyperparameters and padded data are bitwise identical when the vintage is cut at the origin, checked for the primary arm only ([ACC: `data_checks.in_window`]). (iii) The checksums of the vintage files.

The FRED-QD series are those of McCracken and Ng (2021) and the monthly series those of

McCracken and Ng (2016); the data are downloaded on demand and not bundled. CR's estimation used real-time vintages (Croushore and Stark, 2001); the replication runs in pseudo real time on the final vintage, with the start date of Clark (2011). [**TODO:** TODO 21: confirm the official FRED-QD 2026-09 tcodes of these four series (the script loads raw levels and sets its own codes), and align `fred.py` to cite McCracken and Ng (2021).]

Table F.3: Model arms. Every arm uses the mean block `Linear(prior=Minnesota(form="reduced", hierarchy="fixed", lambda_=0.2, alpha=2.0, cross=1.0, mean=0.0))` with $p = 4$, sampled by the corrected triangular algorithm (Carriero et al., 2022), except the structural pair, which uses `form="structural"` (Mode I). SV priors are the library defaults unless stated.

Arm	Volatility specification	CR counterpart	Role
Constant	`Constant("iw_equivalent")`, the order-invariant prior of Chan (2022)	VAR with a normal-diffuse prior	baseline of every relative measure; negative control of the ordering stage
SV(rw)	`SV(rw=True)`: $h_0 \sim \mathrm{N}(\log(\psi_\iota/\iota), 4)$, $\sigma^2 \sim 0.1\chi^2_1$, in-window	VAR-SV (random walk)	primary arm of criteria 1–4 and 6
SV	`SV()`: $\mu \sim \mathrm{N}(\log(\psi_\iota/\iota), 10)$, $(\phi + 1)/2 \sim \mathrm{Beta}(20, 1.5)$, $\sigma^2 \sim \chi^2_1$	—	second reference of criterion 5
SV(rw,t10)	`SV(rw=True, tails="t", nu_prior=DiscreteUniform(10, 10), t_scale="unit")`	VAR-SVt with $d = 10$	criterion 5 variant; ν fixed, so its grid-bound warning fires by design
AR-SV(0.9)	`SV(phi_prior=Beta(71.25, 3.75))`	VAR-ARSV with $a_1 \sim \mathrm{N}(0.9, 0.05^2)$ (same mean and sd on ϕ)	criterion 5 variant
Constant / SV(rw) [structural]	`Constant("ig")` / `SV(rw=True)` in structural Mode I	—	robustness pair

CR's target is Clark and Ravazzolo (2015); the numbers compared are those of the working paper's Tables 1–3 (Clark and Ravazzolo, 2012), as hard-coded in the script (`CR_FULL`, l. 147; `CR_PRE_DLPS`, l. 167). [**TODO:** TODO 22: obtain the JAE tables, or keep saying that the working-paper numbers are used; resolve the 40-versus-56 count of re-read numbers in N08.] CR's VAR uses a Minnesota prior with $\theta = 0.2$, $1/l^2$ decay and a normal-diffuse prior on the coefficients (Kadiyala and Karlsson, 1997); their VAR-SV samples random-walk log-volatilities by the single-move step of Jacquier et al. (1994), with a σ^2 prior $\mathrm{IG}(1/2, 0.0175)$ that the replication does not adopt. The working paper does not state a variable ordering, which is why all 24 orderings are run.

F.7 Further results of the acceptance run

Table F.4 gives the pre-crisis comparison of SV(rw) and Constant at $h = 1$ and $h = 8$, Table F.5 every SV arm against Constant with the convergence shares, the verdicts with the gate on and off, the route by which each verdict was reached and the variant comparisons of criterion 5, and Table F.6 the Constant arm's levels against CR's benchmark VAR together with the compute record. The reading rules of Section 11 apply to every number here. Under the expanding window, any "better" verdict can only come through the model confidence set (Hansen et al., 2011). In the first full run the pre-crisis GDP $h = 8$ cell, which criterion 4 needs to be positive, was $+0.108$ under the path-conditioned estimator and -0.004 under the Gaussian approximation; its acceptance values are [ACC: `relative.pre_crisis['SV(rw)'].variables.GDP['8'].dlps`] and [ACC: `relative.pre_crisis['SV(rw)'].variables.GDP['8'].dlps_gauss`]. The fit-level diagnostic covered only SV(rw), and only three of the 24 orderings at the main-stage seeds, so the gate-on verdicts of SV, SV(rw,t10) and AR-SV(0.9) may stay inconclusive.

Table F.4: Pre-crisis sample (1985Q1–2007Q4), SV(rw) against Constant at $h = 1$ and $h = 8$: ΔLPS under the integrated (primary) and the path-conditioned estimator, the Gaussian-approximation ΔLPS and the number of RB-degenerate origins, with CR's working-paper values.

[**TODO:** Generate with `tab_cr_precrisis.py` from the acceptance JSON.]

Table F.5: All SV arms against Constant: joint ΔLPD by horizon, share of converged origins, the `compare()` verdict with the gate on and off and its route (DM or MCS), MCS membership; and the variant comparisons of criterion 5, full and pre-crisis samples.

[**TODO:** Generate with `tab_cr_arms.py` from the acceptance JSON.]

Table F.6: Constant-arm levels against CR's benchmark VAR at $h = 1$ (average log score and RMSE), and the compute record of the acceptance run: machine, number of worker processes, stage times and wall time. The two log-score rows use different estimators and data vintages, so this is a sanity check, not a parity test.

[**TODO:** Generate with `tab_cr_levels_compute.py`; stage times must be written to the JSON or the report.]

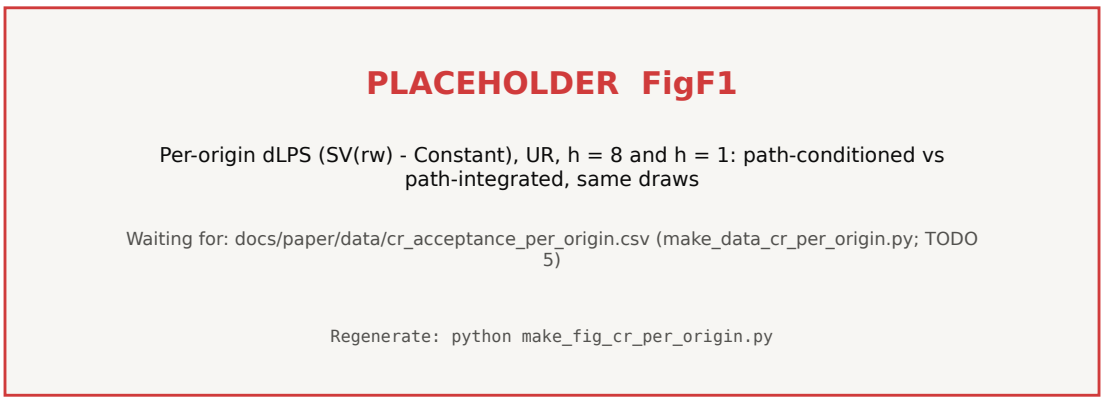


Figure F.1: Per-origin ΔLPS of SV(rw) against Constant for unemployment at $h = 8$ (and $h = 1$ for contrast), origins 1985Q1–2011Q2: path-conditioned against path-integrated, from the same forecast draws of the acceptance run.

[**TODO:** TODO 5: export the per-origin scores of the acceptance run to `cr_acceptance_per_origin.csv` in `docs/paper/data/`, or drop Figures F.1 and F.3.]

F.8 Convergence against the MCMC budget

Table F.7 gives the fit-level convergence diagnostic that set the acceptance budget: SV(rw) fits at the main-stage fit seeds, with no forecasts or scores, at 4×5000, with burn-in 5000, with 8 chains and at $4 \times 20{,}000$, for the identity ordering and the two worst orderings of the first run, together with the main-stage against ordering-stage seed comparison. Its numbers are those of the pre-registration until the diagnostic is committed. Figure F.2 compares the converged shares per ordering of the first full run and the acceptance run. The seed comparison is itself evidence of R-hat noise at the threshold: the same models in the same ordering converged at 72 (main-stage seeds) and 69 (ordering-stage seeds) of 106 origins, with joint $h = 1$ ΔLPD of $+0.809$ and $+0.838$.

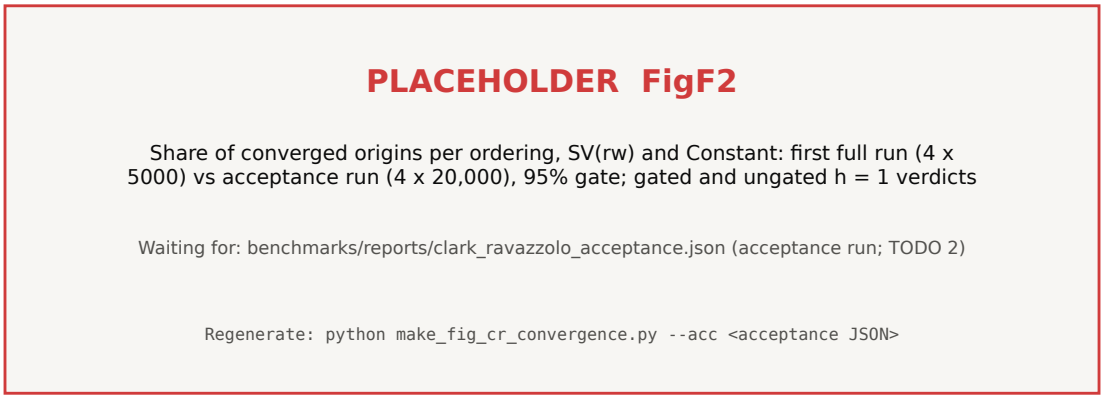


Figure F.2: Share of converged origins per ordering for SV(rw) and Constant: first full run (4×5000) against the acceptance run ($4 \times 20{,}000$), with the 95% gate; gated and ungated $h = 1$ verdicts are marked.

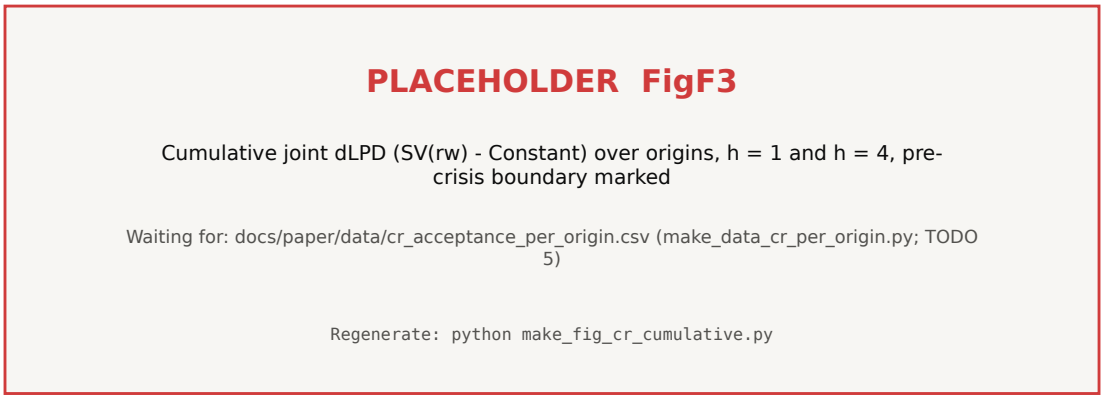


Figure F.3: Cumulative joint ΔLPD of SV(rw) against Constant over origins at $h = 1$ and $h = 4$, with the end of the pre-crisis sample marked.

Table F.7: Convergence against the MCMC budget for SV(rw), fit-level only: 4×5000, burn-in 5000, 8 chains and $4 \times 20{,}000$; identity ordering and the two worst orderings; converged origins, maximum R-hat and minimum bulk ESS; and the main-stage against ordering-stage seed comparison.

[**TODO:** Generate with `tab_cr_budget.py`; the numbers stay [PREREG] until TODO 4 is done.]

F.9 The first full run (history)

Table F.8 reproduces the first full run's tables for the record: the headline and pre-crisis comparisons, all arms, the variants, the structural pair, the RB-degenerate origins and the cells in which the RB and Gaussian ΔLPS differ in sign. They were produced by the path-conditioned estimator with 4×5000 draws and 1000 forecast draws, and they are history, not results; Section 11 summarises what they showed and why the acceptance run differs. Across all SV arms and both samples, the RB and Gaussian ΔLPS differed in sign in 28 marginal and 8 joint cells, all at $h \geq 2$. The order-invariant Constant arm was flagged "material" by the ordering diagnostic at two $h > 1$ cells (joint $h = 2$ and UR $h = 8$), with a noise floor resting on one replicate; this is reported as a limitation of the diagnostic, and the acceptance run's flag is [ACC: `orderings.results.Constant.material`]. Differences from CR's levels, such as inflation gains of 1.6–3.2 times CR's, are not attributed to any single cause; final-vintage chain-type prices and the iw_equivalent baseline are hypotheses only.

Table F.8: First full run, for the record: code 4bb978a, report 551f181, path-conditioned RB estimator, 4×5000 kept draws, 1000 forecast draws. Headline, pre-crisis, all arms, variants, structural pair, RB-degenerate origins and the RB/Gaussian sign-disagreement cells. These numbers are history, not results.

[**TODO:** Generate with `tab_first_run.py` from the first full run's JSON.]

G Reproducibility record

Release. Every table and figure of the paper is to be regenerated on one release commit of rai-bvar, whose hash, tag and archive are given here. [**TODO:** TODO 20: merge `phase-2-cr-acceptance` into `main`, fix the release commit hash, tag the release, mint a Zenodo DOI for the release and the replication outputs, add the repository URL, and re-run `ex_a.py` and `ex_b.py` on that commit.]

Validation records. With `RAI_BVAR_VALIDATION_RECORD` set to `docs/paper/data/release`, three test modules append one JSON line per run to that directory: `geweke.jsonl` from `tests/geweke`, for every Geweke tier (G-small, the block configurations, G-full and the recorded `exact=False` rows, distinguished by a `tier` field), `sbc.jsonl` from `tests/sbc` and `mutation.jsonl` from `tests/mutation`. The release record is planned to add the x86-64 mutation record (under a separate name), the composition-level Tier B, parity (`pytest -m parity_long`) and analytic-twin results, for which no record hook exists yet and pytest's JUnit XML is the fallback; `collect.txt` with `pytest --collect-only -q` and its per-marker variants; and `env.json` with the environment of Table G.2. [**TODO:** TODO 7: run and commit these records, adding record hooks (or JUnit capture) for composition-level Tier B, parity and the twins. Uncommitted Geweke (block, G-full, `exact=False`), SBC and mutation records of unknown commit and machine sit under `docs/paper/figures/data/validation_record/`; they are not release records.]

Data. The FRED vintages are downloaded on demand from the St. Louis Fed, verified against the SHA-256 checksums of the package's registry, and cached; they are not bundled.

Dataset, vintage	SHA-256 of the published file
FRED-MD 2026-09	`d6e239038afbdbf4667c2a63c13c942817bc196fedab4355cd7e0d353ecdd65d`
FRED-QD 2026-09	`a144b1547ddbad5c68fc934c90c0c0f9837a5139571ba8e64afcc2637aaa3feb`

Regeneration. The `Makefile` in `docs/paper` regenerates the figures from committed code and data (`make figures`) and builds the paper (`make`); each figure script names its data source in its header, and Table G.2 is to give the command for each float. The acceptance tables and figures need `benchmarks/reports/clark_ravazzolo_acceptance.json`, and Figures F.1 and F.3 the per-origin export of TODO 5. [**TODO:** TODO 20: the table generators named in the captions (`tab_*.py`) are not written yet; add them to the `Makefile`, then state the one command and its wall time on the release commit.]

What reproduces how. Table G.1 states each guarantee with its scope. Three levels must be kept apart. Bitwise equality holds for the listed variations within one platform and one version set, and part of it is established only with the runner's test doubles ("toys"), not with the production kernels. Equality to rounding holds where XLA vectorises the reductions of the production SV and CCCM kernels differently for different batch sizes: chains are then equal only until a last-bit difference flips an accept/reject decision, and in distribution afterwards. Nothing is guaranteed across platforms (OpenBLAS against Accelerate), JAX versions or PRNG implementations. In practice, on one machine and version set, re-runs reproduced the printed output of the analytic listing of Section 7 line for line, that of the MCMC listing except its timing lines, and the fixed-seed validation statistics to every printed digit; across machines the mutation statistics agreed to two or three significant figures, and MCMC results can be compared only within Monte Carlo error.

Every MCMC fit records the information needed to reproduce it: the jax and jaxlib versions, the PRNG implementation and whether threefry is partitionable, the platform, device count and chain method, the seed and the data salt, the resolved Gram mode, constant-volatility path and T_{pad}, the mixture, offset and exact flag, the ordering, the specification hash and the package version; the sweep range and the burn-in, draws and thinning are stored with the draws. Keys are counter-based (Salmon et al., 2011), derived by `fold_in` over seed, stream, data salt, chain, absolute sweep, block, equation and sub-draw, with uint32 counters and no key in any loop carry, so draws cannot

Table G.1: Reproducibility matrix: for a fixed specification, data, seed and device count, each variation, the guarantee, the test that establishes it, and its scope. "Toys" are the runner's test doubles in `tests/mcmc/runner/toys.py`, which implement the block protocols but are not the production kernels.

Variation	Guarantee	Scope	Tests
chunk size and burn-in piece size	bitwise	toys; production follows from the key layout [TC]	a
thinning with aligned sweeps	bitwise	toys	b
fused against unfused plan	bitwise	toys (3 configurations)	c
`vmap` over chains against a sequential loop	bitwise	toys	d
chain-count prefix, `chains="map"`	bitwise	toys and production `structural_sv`	e
chain-count prefix, `chains="vmap"`	toys: bitwise on macOS, to rounding on x86 Linux; production: rtol 10^{-9}	toys; production SV and CCCM kernels, short runs	f
`vmap` against `shard_map` over 4 devices	toys: rtol 10^{-12}; production: rtol 10^{-9}	toys; production `structural_sv`, short burn-in	g
leading fit axis against looped fits	rtol 10^{-10}	toys	h
in-loop against post-hoc forecasts	10^{-12}	production	i
memory-mapped against in-memory store	equal	toys	j
OpenBLAS thread count (factors $< 128 \times 128$)	bitwise	recorded (module docstring)	—
adding the dummy-row code, with no rows	bitwise (537 arrays)	recorded, one machine; structural test	k
platform, JAX version, PRNG implementation, partitionable threefry, spec fields	none	by design	—

Tests in `tests/mcmc/`: (a) `runner/test_chunking.py::test_chunk_and_burn_piece_size_invariance`; (b) `runner/test_chunking.py::test_thinning_with_aligned_sweeps`; (c) `runner/test_sweep.py::test_fused_equals_unfused_bitwise`; (d) `runner/test_sweep.py::test_vmap_over_chains_equals_sequential_loop`; (e) `runner/test_chunking.py::test_chain_prefix_invariance_with_mapped_chains`, `::test_mapped_chains_production_prefix`; (f) `runner/test_chunking.py::test_chain_prefix_invariance`, `::test_production_chain_prefix_with_vmapped_chains_to_rounding`, `::test_production_chain_prefix_with_vmapped_reduced_form_to_rounding`; (g) `runner/test_devices_bitwise.py::test_shard_map_matches_vmap_in_subprocess`; (h) `runner/test_runner.py::test_fit_axis_equals_looped_fits`; (i) `forecast/test_forecast_core.py::test_in_loop_forecast_equals_post_hoc_sv`, `::test_in_loop_forecast_equals_post_hoc_constant`, `::test_in_loop_forecast_equals_post_hoc_with_thinned_storage`; (j) `runner/test_storage.py::test_memmap_store_equals_memory_store`; (k) `blocks/test_dummies.py::test_dummy_path_is_closed_without_dummies`.

depend on chunking, burn-in pieces or thinning alignment; with partitionable threefry, the words drawn directly from a parent key equal the key data of its `fold_in` children, which JAX's key-reuse checker cannot see, and the engine avoids this by giving every consumer its own sub-key.

Table G.2: Environment and regeneration record: versions of Python, NumPy, SciPy, jax and jaxlib, scoringrules and the BLAS backend; the machines; the release commit; and the command that regenerates each table and figure.

[**TODO:** Generate with `tab_environment.py` on the release commit.]

H Licensing and source-reading practice

rai-bvar is released under the Apache License 2.0. The package's practice with respect to GPL-licensed reference software is stated in these words:

> No code was translated from GPL sources. Algorithms were implemented from the published papers. GPL packages were run as numerical oracles, and in a few documented cases (named in the repository notes) their source was read to pin down an oracle's behaviour, defaults or constants, never to write implementation code.

Table H.1 lists the repository notes in which GPL or unlicensed source was read, with the versions and the purpose each note records, and the notes in which none was read. The versions in the table are those of the source that was read; the parity fixtures were produced with stochvol 3.2.1 and R BVAR 1.0.5 (Appendix D.5).

Table H.1: Repository notes and the reading of third-party source they record.

Note	What was read, and for what
Notes in which source was read	
`docs/phase2/02_sv_sampler.md`	stochvol (GitHub sources and CRAN manuals), and the bayesianVARs 0.1.8 and bvarsv 1.1 sources, in the note itself; stochvol 3.2.9, bayesianVARs 0.1.8 and bvarsv 1.1 in its cross-check; read only for published defaults, constants and behaviour; no code transcribed
`docs/phase2/01_omori_mixture.md`	six GPL packages opened only to confirm the published mixture constants and observable behaviour (which table, which offset, whether a shift is applied)
`docs/phase2/03_sv_outliers_t.md`	greps of the stochvol C++ and R sources (the ν proposal, the update order, the t scaling of the simulator), and greps of the replication code of Carriero et al. (2024), which carries no licence (prior constants, outlier grid, mixture table, the ν log-likelihood constant)
`docs/phase2/07_jax_patterns.md`	in the cross-check, one stochvol header searched only for the 10 published mixture constants
`tests/parity/BVAR_PARAMETRIZATION.md`	R BVAR 1.0.5 sources and the JSS article, read to establish the behaviour of the oracle; "not a description of code to be ported"
Notes in which no GPL source was read	
`tests/parity/SV_PARAMETRIZATION.md` (packages only run; behaviour from manuals and experiments); `docs/phase2/04_cccm_corrected.md`; `docs/phase2/05_structural_equation_sampler.md`; `docs/phase2/06_validation_geweke_sbc.md`; `docs/phase2/08_common_sv_and_ordering.md`.	

The committed parity fixtures that the R oracles produced (in `tests/parity/fixtures/`) are outputs of GPL programs run as oracles; the project treats them as data, not code. [**TODO:** TODO 17: decide whether to cite Free Software Foundation (2026) for this position (as a web page, not legal advice).] The Bank of England bvar package (Labonne et al., 2026) is MIT-licensed. FRED-MD and FRED-QD (McCracken and Ng, 2016, 2021) are downloaded on demand and never bundled, because some series are third-party copyrighted; the named variable sets of the loader, and the committed fixtures, use only U.S. federal-agency series.

I Design-review errata relevant to numerics and reproducibility

The Phase 2 design was reviewed before implementation, and its errata table maps each of the 28 findings of the reviews (E1–E28) to its resolution. Four of those that concern numerics and

reproducibility are listed below; each was found in the proposals and resolved in the design before the code it concerned was written.

E7 The counter $it_0 + j \cdot \text{thin} + r$ promotes to int64 under 64-bit mode. All counters are uint32 with explicit casts, `step_key` raises on any other counter type, and the host checks that the sweep count stays below 2^{32} (Algorithm C.10).

E9 An earlier proposal validated the samplers with the blocks' own prior samplers and log priors. The design adopted an independent reference registry with separate owners, which became the two-implementation rule of Section 8.

E14 Two "runtime" settings, the Gram mode and padding to a maximum length, were not bitwise-neutral. The Gram mode is now resolved by a fixed shape rule and stored in the specification, there is no padding to a maximum, and `Runtime` holds only settings that have an invariance test, bitwise except for the device count, which is to rounding (Table G.1).

E23 $2 \log |e_t|$ is $-\infty$ on padded rows and produced NaN. The residual is masked before the logarithm (Algorithm C.6).

The review process is described in Appendix J; the generic lessons for other implementations are in Section 9.

J Development process and AI-assistance disclosure

[**TODO:** TODO 30: the author writes and approves this appendix, including the AI-assistance statement, worded to arXiv policy and to the target venue's; an AI system is not an author.] The following facts about the process are recorded in the repository and may be used. The Phase 2 design was produced from three competing proposals (correctness-first, performance-first and extensibility-first), ranked by two independent reviews ("judges"); the highest-ranked proposal was taken as the base and elements of the other two were grafted onto it, and every technical error either review listed was mapped to a resolution (errata E1–E28, Appendix I). The implementation was split into work packages with separate owners, under the two-implementation rule: the references of the validation harness were written by packages that wrote no kernel (one later reference, for the SOC/SUR dummy rows, was written by the kernels' owners from the model; Section 8.2), and a test enforces that no reference imports a kernel module. Independent review runs on x86-64 Linux re-executed parts of the validation suite, including the mutation suite, and caught further injected errors. [**TODO:** TODO 30: state how AI assistance was used (design, implementation, review, the dossiers and drafting of this paper), who verified what, and the authorship and acknowledgements.]

References

Oriol Abril-Pla, Virgile Andreani, Colin Carroll, Larry Dong, Christopher J. Fonnesbeck, Maxim Kochurov, Ravin Kumar, Junpeng Lao, Christian C. Luhmann, Osvaldo A. Martin, Michael Osthege, Ricardo Vieira, Thomas Wiecki, and Robert Zinkov. PyMC: A modern, and comprehensive probabilistic programming framework in Python. *PeerJ Computer Science*, 9:e1516, 2023. doi: 10.7717/peerj-cs.1516.

Malin Adolfson, Jesper Lindé, and Mattias Villani. Forecasting performance of an open economy DSGE model. *Econometric Reviews*, 26(2–4):289–328, 2007. doi: 10.1080/07474930701220543.

Attahiru Sule Alfa, Jungong Xue, and Qiang Ye. Accurate computation of the smallest eigenvalue of a diagonally dominant M-matrix. *Mathematics of Computation*, 71(237):217–236, 2002. doi: 10.1090/S0025-5718-01-01325-4.

Gianni Amisano and Raffaella Giacomini. Comparing density forecasts via weighted likelihood ratio tests. *Journal of Business & Economic Statistics*, 25(2):177–190, 2007. doi: 10.1198/073500106000000332.

Jonas E. Arias, Juan F. Rubio-Ramírez, and Minchul Shin. Macroeconomic forecasting and variable ordering in multivariate stochastic volatility models. *Journal of Econometrics*, 235(2):1054–1086, 2023. doi: 10.1016/j.jeconom.2022.04.013.

Marta Bańbura, Domenico Giannone, and Lucrezia Reichlin. Large Bayesian vector auto regressions. *Journal of Applied Econometrics*, 25(1):71–92, 2010. doi: 10.1002/jae.1137.

Dominik Bertsche and Robin Braun. Identification of structural vector autoregressions by stochastic volatility. *Journal of Business & Economic Statistics*, 40(1):328–341, 2022. doi: 10.1080/07350015.2020.1813588.

Åke Björck. *Numerical Methods for Least Squares Problems*. Society for Industrial and Applied Mathematics, 1996. doi: 10.1137/1.9781611971484.

Maximilian Boeck, Martin Feldkircher, and Florian Huber. BGVAR: Bayesian global vector autoregressions with shrinkage priors in R. *Journal of Statistical Software*, 104(9):1–28, 2022. doi: 10.18637/jss.v104.i09.

Mark Bognanni. Comment on "large Bayesian vector autoregressions with stochastic volatility and non-conjugate priors". *Journal of Econometrics*, 227(2):498–505, 2022. doi: 10.1016/j.jeconom.2021.10.008.

James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Yash Katariya, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. *JAX: Composable Transformations of Python+NumPy Programs*, 2018. URL `https://github.com/jax-ml/jax`. Python package version 0.12.0.1.

Paul-Christian Bürkner, Jonah Gabry, and Aki Vehtari. Approximate leave-future-out cross-validation for Bayesian time series models. *Journal of Statistical Computation and Simulation*, 90(14):2499–2523, 2020. doi: 10.1080/00949655.2020.1783262.

Andrea Carriero, Todd E. Clark, and Massimiliano Marcellino. Bayesian VARs: Specification choices and forecast accuracy. *Journal of Applied Econometrics*, 30(1):46–73, 2015. doi: 10.1002/jae.2315.

Andrea Carriero, Todd E. Clark, and Massimiliano Marcellino. Common drifting volatility in large Bayesian VARs. *Journal of Business & Economic Statistics*, 34(3):375–390, 2016. doi: 10.1080/07350015.2015.1040116.

Andrea Carriero, Todd E. Clark, and Massimiliano Marcellino. Large Bayesian vector autoregressions with stochastic volatility and non-conjugate priors. *Journal of Econometrics*, 212(1):137–154, 2019. doi: 10.1016/j.jeconom.2019.04.024.

Andrea Carriero, Joshua Chan, Todd E. Clark, and Massimiliano Marcellino. Corrigendum to "large Bayesian vector autoregressions with stochastic volatility and non-conjugate priors" [J. Econometrics 212 (1) (2019) 137–154]. *Journal of Econometrics*, 227(2):506–512, 2022. doi: 10.1016/j.jeconom.2021.11.010.

Andrea Carriero, Todd E. Clark, Massimiliano Marcellino, and Elmar Mertens. Addressing COVID-19 outliers in BVARs with stochastic volatility. *Review of Economics and Statistics*, 106(5): 1403–1417, 2024. doi: 10.1162/rest_a_01213.

Joshua C. C. Chan. Asymmetric conjugate priors for large Bayesian VARs. *Quantitative Economics*, 13(3):1145–1169, 2022. doi: 10.3982/QE1381.

Joshua C. C. Chan. BVARs and stochastic volatility. In Michael P. Clements and Ana Beatriz Galvão, editors, *Handbook of Research Methods and Applications in Macroeconomic Forecasting*, chapter 3, pages 43–67. Edward Elgar Publishing, 2024. doi: 10.4337/9781035310050.00007. Preprint: arXiv:2310.14438.

Joshua C. C. Chan. *bvar-toolkit: MATLAB Library for Large Bayesian VARs*, 2026a. URL `https://github.com/joshuaccchan/bvar-toolkit`.

Joshua C. C. Chan. *bvar-toolkit-python: Python Version of bvar-toolkit*, 2026b. URL `https://github.com/joshuaccchan/bvar-toolkit-python`.

Joshua C. C. Chan and Ivan Jeliazkov. Efficient simulation and integrated likelihood estimation in state space models. *International Journal of Mathematical Modelling and Numerical Optimisation*, 1(1/2):101–120, 2009. doi: 10.1504/IJMMNO.2009.030090.

Joshua C. C. Chan, Gary Koop, and Xuewen Yu. Large order-invariant Bayesian VARs with stochastic volatility. *Journal of Business & Economic Statistics*, 42(2):825–837, 2024. doi: 10.1080/07350015.2023.2252039.

Jean-Paul Chilès and Pierre Delfiner. *Geostatistics: Modeling Spatial Uncertainty*. Wiley Series in Probability and Statistics. John Wiley & Sons, 2nd edition, 2012. doi: 10.1002/9781118136188.

Todd E. Clark. Real-time density forecasts from Bayesian vector autoregressions with stochastic volatility. *Journal of Business & Economic Statistics*, 29(3):327–341, 2011. doi: 10.1198/jbes.2010.09248.

Todd E. Clark and Michael W. McCracken. Tests of equal forecast accuracy and encompassing for nested models. *Journal of Econometrics*, 105(1):85–110, 2001. doi: 10.1016/S0304-4076(01)00071-9.

Todd E. Clark and Francesco Ravazzolo. The macroeconomic forecasting performance of autoregressive models with alternative specifications of time-varying volatility. Working Paper 12-18, Federal Reserve Bank of Cleveland, 2012. Also circulated as Norges Bank Working Paper 9/2012.

Todd E. Clark and Francesco Ravazzolo. Macroeconomic forecasting performance under alternative specifications of time-varying volatility. *Journal of Applied Econometrics*, 30(4):551–575, 2015. doi: 10.1002/jae.2379.

Todd E. Clark, Florian Huber, Gary Koop, Massimiliano Marcellino, and Michael Pfarrhofer. Tail forecasting with multivariate Bayesian additive regression trees. *International Economic Review*, 64(3):979–1022, 2023. doi: 10.1111/iere.12619.

Timothy Cogley and Thomas J. Sargent. Drifts and volatilities: Monetary policies and outcomes in the post WWII US. *Review of Economic Dynamics*, 8(2):262–302, 2005. doi: 10.1016/j.red.2004.10.009.

Samantha R. Cook, Andrew Gelman, and Donald B. Rubin. Validation of software for Bayesian models using posterior quantiles. *Journal of Computational and Graphical Statistics*, 15(3):675–692, 2006. doi: 10.1198/106186006X136976.

Dean Croushore and Tom Stark. A real-time data set for macroeconomists. *Journal of Econometrics*, 105(1):111–130, 2001. doi: 10.1016/S0304-4076(01)00072-0.

A. Philip Dawid and Paola Sebastiani. Coherent dispersion criteria for optimal experimental design. *The Annals of Statistics*, 27(1):65–81, 1999. doi: 10.1214/aos/1018031101.

Marco Del Negro and Giorgio E. Primiceri. Time varying structural vector autoregressions and monetary policy: A corrigendum. *Review of Economic Studies*, 82(4):1342–1345, 2015. doi: 10.1093/restud/rdv024.

Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978. doi: 10.1109/C-M.1978.218136.

Francis X. Diebold and Roberto S. Mariano. Comparing predictive accuracy. *Journal of Business & Economic Statistics*, 13(3):253–263, 1995. doi: 10.1080/07350015.1995.10524599.

Alistair Dieppe, Romain Legrand, and Björn van Roye. The BEAR toolbox. Working Paper Series 1934, European Central Bank, 2016. URL `https://www.ecb.europa.eu/pub/pdf/scpwps/ecbwp1934.en.pdf`. Also available from SSRN, DOI 10.2139/ssrn.2811020.

Thomas Doan, Robert Litterman, and Christopher Sims. Forecasting and conditional projection using realistic prior distributions. *Econometric Reviews*, 3(1):1–100, 1984. doi: 10.1080/07474938408800053.

Saikat Dutta, Owolabi Legunsen, Zixin Huang, and Sasa Misailovic. Testing probabilistic programming systems. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '18)*, pages 574–586. ACM, 2018. doi: 10.1145/3236024.3236057.

James M. Flegal and Galin L. Jones. Batch means and spectral variance estimators in Markov chain Monte Carlo. *The Annals of Statistics*, 38(2):1034–1070, 2010. doi: 10.1214/09-AOS735.

Colin Fox and Albert Parker. Accelerated Gibbs sampling of normal distributions using matrix splittings and polynomials. *Bernoulli*, 23(4B):3711–3743, 2017. doi: 10.3150/16-BEJ863.

Free Software Foundation. Frequently asked questions about the GNU licenses. `https://www.gnu.org/licenses/gpl-faq.html`, 2026. Entries "Is there some way that I can GPL the output people get from use of my program?" and "In what cases is the output of a GPL program covered by the GPL too?"; accessed 2026-10-23.

Roy Frostig, Matthew James Johnson, and Chris Leary. Compiling machine learning programs via high-level tracing. In *SysML Conference 2018*, 2018. URL `https://mlsys.org/Conferences/doc/2018/146.pdf`.

Sylvia Frühwirth-Schnatter and Helga Wagner. Stochastic model specification search for Gaussian and partial non-Gaussian state space models. *Journal of Econometrics*, 154(1):85–100, 2010. doi: 10.1016/j.jeconom.2009.07.003.

Axel Gandy and James Scott. Unit testing for MCMC and other Monte Carlo methods, 2021. URL `https://arxiv.org/abs/2001.06465`. arXiv:2001.06465v2.

Andrew Gelman. Correction to Cook, Gelman, and Rubin (2006). *Journal of Computational and Graphical Statistics*, 26(4):940, 2017. doi: 10.1080/10618600.2017.1377082.

John Geweke. Getting it right: Joint distribution tests of posterior simulators. *Journal of the American Statistical Association*, 99(467):799–804, 2004. doi: 10.1198/016214504000001132.

John Geweke and Gianni Amisano. Comparing and evaluating Bayesian predictive distributions of asset returns. *International Journal of Forecasting*, 26(2):216–230, 2010. doi: 10.1016/j.ijforecast.2009.10.007.

Charles J. Geyer. Practical Markov chain Monte Carlo. *Statistical Science*, 7(4):473–483, 1992. doi: 10.1214/ss/1177011137.

Raffaella Giacomini and Halbert White. Tests of conditional predictive ability. *Econometrica*, 74 (6):1545–1578, 2006. doi: 10.1111/j.1468-0262.2006.00718.x.

Domenico Giannone, Michele Lenza, and Giorgio E. Primiceri. Prior selection for vector autoregressions. *Review of Economics and Statistics*, 97(2):436–451, 2015. doi: 10.1162/REST_a_00483.

Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007. doi: 10.1198/016214506000001437.

Tilmann Gneiting and Roopesh Ranjan. Comparing density forecasts using threshold- and quantile-weighted scoring rules. *Journal of Business & Economic Statistics*, 29(3):411–422, 2011. doi: 10.1198/jbes.2010.08110.

Winfried K. Grassmann, Michael I. Taksar, and Daniel P. Heyman. Regenerative analysis and steady state distributions for Markov chains. *Operations Research*, 33(5):1107–1116, 1985. doi: 10.1287/opre.33.5.1107.

Roger B. Grosse and David K. Duvenaud. Testing MCMC code, 2014. URL `https://arxiv.org/abs/1412.5218`. arXiv:1412.5218. Presented at the 2014 NIPS Workshop on Software Engineering for Machine Learning.

Luis Gruber and Gregor Kastner. Forecasting macroeconomic data with Bayesian VARs: Sparse or dense? It depends! *International Journal of Forecasting*, 41(4):1589–1619, 2025. doi: 10.1016/j.ijforecast.2025.02.001.

Luis Gruber, Stefan Haan, and Gregor Kastner. *bayesianVARs: MCMC Estimation of Bayesian Vectorautoregressions*, 2026. URL `https://CRAN.R-project.org/package=bayesianVARs`. R package version 0.1.8.

A. K. Gupta and D. K. Nagar. *Matrix Variate Distributions*, volume 104 of *Monographs and Surveys in Pure and Applied Mathematics*. Chapman & Hall/CRC, Boca Raton, 2000. ISBN 1-58488-046-5.

Peter R. Hansen, Asger Lunde, and James M. Nason. The model confidence set. *Econometrica*, 79 (2):453–497, 2011. doi: 10.3982/ECTA5771.

Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2.

David Harvey, Stephen Leybourne, and Paul Newbold. Testing the equality of prediction mean squared errors. *International Journal of Forecasting*, 13(2):281–291, 1997. doi: 10.1016/S0169-2070(96)00719-4.

Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, 2nd edition, 2002. doi: 10.1137/1.9780898718027.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979. URL `https://www.jstor.org/stable/4615733`.

Darjus Hosszejni and Gregor Kastner. Modeling univariate and multivariate stochastic volatility in R with stochvol and factorstochvol. *Journal of Statistical Software*, 100(12):1–34, 2021. doi: 10.18637/jss.v100.i12.

Stephan Hoyer and Joe Hamman. xarray: N-D labeled arrays and datasets in Python. *Journal of Open Research Software*, 5(1):10, 2017. doi: 10.5334/jors.148.

Florian Huber and Luca Rossini. Inference in Bayesian additive vector autoregressive tree models. *The Annals of Applied Statistics*, 16(1):104–123, 2022. doi: 10.1214/21-AOAS1488.

Eric Jacquier, Nicholas G. Polson, and Peter E. Rossi. Bayesian analysis of stochastic volatility models. *Journal of Business & Economic Statistics*, 12(4):371–389, 1994. doi: 10.1080/07350015.1994.10524553.

Eric Jacquier, Nicholas G. Polson, and Peter E. Rossi. Bayesian analysis of stochastic volatility models with fat-tails and correlated errors. *Journal of Econometrics*, 122(1):185–212, 2004. doi: 10.1016/j.jeconom.2003.09.001.

Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011. doi: 10.1109/TSE.2010.62.

Alexander Jordan, Fabian Krüger, and Sebastian Lerch. Evaluating probabilistic forecasts with scoringRules. *Journal of Statistical Software*, 90(12):1–37, 2019. doi: 10.18637/jss.v090.i12.

André G. Journel and Charles J. Huijbregts. *Mining Geostatistics*. Academic Press, London, 1978.

K. Rao Kadiyala and Sune Karlsson. Numerical methods for estimation and inference in Bayesian VAR-models. *Journal of Applied Econometrics*, 12(2):99–132, 1997. doi: 10.1002/(SICI)1099-1255(199703)12:2<99::AID-JAE429>3.0.CO;2-A.

Gregor Kastner. Dealing with stochastic volatility in time series using the R package stochvol. *Journal of Statistical Software*, 69(5):1–30, 2016. doi: 10.18637/jss.v069.i05.

Gregor Kastner. Sparse Bayesian time-varying covariance estimation in many dimensions. *Journal of Econometrics*, 210(1):98–115, 2019. doi: 10.1016/j.jeconom.2018.11.007.

Gregor Kastner. *factorstochvol: Bayesian Estimation of (Sparse) Latent Factor Stochastic Volatility Models*, 2026. URL `https://CRAN.R-project.org/package=factorstochvol`. R package version 1.1.2.

Gregor Kastner and Sylvia Frühwirth-Schnatter. Ancillarity-sufficiency interweaving strategy (ASIS) for boosting MCMC estimation of stochastic volatility models. *Computational Statistics & Data Analysis*, 76:408–423, 2014. doi: 10.1016/j.csda.2013.01.002.

Sangjoon Kim, Neil Shephard, and Siddhartha Chib. Stochastic volatility: Likelihood inference and comparison with ARCH models. *Review of Economic Studies*, 65(3):361–393, 1998. doi: 10.1111/1467-937X.00050.

Young Geun Kim and Changryong Baek. *bvhar: Bayesian Vector Heterogeneous Autoregressive Modeling*, 2026. URL `https://CRAN.R-project.org/package=bvhar`. R package version 2.4.1.

Leslie Kish. *Survey Sampling*. John Wiley & Sons, New York, 1965.

Fabian Krueger. *bvarsv: Bayesian Analysis of a Vector Autoregressive Model with Stochastic Volatility and Time-Varying Parameters*, 2015. URL `https://CRAN.R-project.org/package=bvarsv`. R package version 1.1.

Fabian Krüger, Sebastian Lerch, Thordis Thorarinsdottir, and Tilmann Gneiting. Predictive inference based on Markov chain Monte Carlo output. *International Statistical Review*, 89(2):274–301, 2021. doi: 10.1111/insr.12405.

Ravin Kumar, Colin Carroll, Ari Hartikainen, and Osvaldo Martin. ArviZ a unified library for exploratory analysis of Bayesian models in Python. *Journal of Open Source Software*, 4(33):1143, 2019. doi: 10.21105/joss.01143.

Nikolas Kuschnig and Lukas Vashold. BVAR: Bayesian vector autoregressions with hierarchical prior selection in R. *Journal of Statistical Software*, 100(14):1–27, 2021. doi: 10.18637/jss.v100.i14.

Nikolas Kuschnig and Lukas Vashold. *BVAR: Hierarchical Prior Selection in R*, 2024. URL `https://CRAN.R-project.org/package=BVAR`. R package version 1.0.5.

Paul Labonne, Andrea Renzetti, and Joseph Oyegoke. *bvar: Bayesian Vector Autoregressions for Forecasting*. Bank of England, 2026. URL `https://github.com/bank-of-england/bvar`. Python package version 0.3.5.

Michele Lenza and Giorgio E. Primiceri. How to estimate a vector autoregression after March 2020. *Journal of Applied Econometrics*, 37(4):688–699, 2022. doi: 10.1002/jae.2895.

Robert B. Litterman. Forecasting with Bayesian vector autoregressions – five years of experience. *Journal of Business & Economic Statistics*, 4(1):25–38, 1986. doi: 10.1080/07350015.1986.10509491.

Rui Liu, Andrés Ramirez Hassan, and Tomasz Woźniak. *bvars: Bayesian Forecasting with Large Vector Autoregressions*, 2026. URL `https://CRAN.R-project.org/package=bvars`. R package version 1.1.

George Marsaglia and Wai Wan Tsang. A simple method for generating gamma variables. *ACM Transactions on Mathematical Software*, 26(3):363–372, 2000. doi: 10.1145/358407.358414.

William J. McCausland, Shirley Miller, and Denis Pelletier. Simulation smoothing for state–space models: A computational efficiency analysis. *Computational Statistics & Data Analysis*, 55(1): 199–212, 2011. doi: 10.1016/j.csda.2010.07.009.

Michael W. McCracken and Serena Ng. FRED-MD: A monthly database for macroeconomic research. *Journal of Business & Economic Statistics*, 34(4):574–589, 2016. doi: 10.1080/07350015.2015.1086655.

Michael W. McCracken and Serena Ng. FRED-QD: A quarterly database for macroeconomic research. *Federal Reserve Bank of St. Louis Review*, 103(1):1–44, 2021. doi: 10.20955/r.103.1-44.

Martin Modrák, Angie H. Moon, Shinyoung Kim, Paul-Christian Bürkner, Niko Huurre, Kateřina Faltejsková, Andrew Gelman, and Aki Vehtari. Simulation-based calibration checking for Bayesian computation: The choice of test quantities shapes sensitivity. *Bayesian Analysis*, 20 (2):461–488, 2025. doi: 10.1214/23-BA1404.

Franz X. Mohr. *bvartools: Bayesian Inference of Vector Autoregressive and Error Correction Models*, 2026. URL `https://CRAN.R-project.org/package=bvartools`. R package version 0.3.0.

Whitney K. Newey and Kenneth D. West. A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix. *Econometrica*, 55(3):703–708, 1987. doi: 10.2307/1913610.

Yasuhiro Omori, Siddhartha Chib, Neil Shephard, and Jouchi Nakajima. Stochastic volatility with leverage: Fast and efficient likelihood inference. *Journal of Econometrics*, 140(2):425–449, 2007. doi: 10.1016/j.jeconom.2006.07.008.

Du Phan, Neeraj Pradhan, and Martin Jankowiak. Composable effects for flexible and accelerated probabilistic programming in NumPyro, 2019. URL `https://arxiv.org/abs/1912.11554`. arXiv:1912.11554. NeurIPS 2019 Program Transformations for Machine Learning Workshop.

Thomas Pinder. *impulso: Bayesian Vector Autoregression (VAR) in Python*, 2026. URL `https://github.com/QuantClimate/Impulso`. Python package version 0.1.3.

Giorgio E. Primiceri. Time varying structural vector autoregressions and monetary policy. *Review of Economic Studies*, 72(3):821–852, 2005. doi: 10.1111/j.1467-937X.2005.00353.x.

PyMC Developers. *pymc-extras*, 2026. URL `https://github.com/pymc-devs/pymc-extras`. Python package version 0.15.1.

Håvard Rue. Fast sampling of Gaussian Markov random fields. *Journal of the Royal Statistical Society B*, 63(2):325–338, 2001. doi: 10.1111/1467-9868.00288.

Teemu Säilynoja, Paul-Christian Bürkner, and Aki Vehtari. Graphical test for discrete uniformity and its applications in goodness-of-fit evaluation and multiple sample comparison. *Statistics and Computing*, 32(2):32, 2022. doi: 10.1007/s11222-022-10090-6.

Teemu Säilynoja, Marvin Schmitt, Paul-Christian Bürkner, and Aki Vehtari. Posterior SBC: Simulation-based calibration checking conditional on data, 2025. URL `https://arxiv.org/abs/2502.03279`. arXiv:2502.03279.

John K. Salmon, Mark A. Moraes, Ron O. Dror, and David E. Shaw. Parallel random numbers: As easy as 1, 2, 3. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis (SC '11)*, pages 1–12. ACM, 2011. doi: 10.1145/2063384.2063405.

Michael Scheuerer and Thomas M. Hamill. Variogram-based proper scoring rules for probabilistic forecasts of multivariate quantities. *Monthly Weather Review*, 143(4):1321–1334, 2015. doi: 10.1175/MWR-D-14-00269.1.

Skipper Seabold and Josef Perktold. Statsmodels: Econometric and statistical modeling with Python. In *Proceedings of the 9th Python in Science Conference*, pages 92–96, 2010. doi: 10.25080/Majora-92bf1922-011.

Charles Shaw. *srvar-toolkit: Shadow-Rate VAR Toolkit for Bayesian Macroeconomic Forecasting*, 2026. URL `https://pypi.org/project/srvar-toolkit/`. Python package version 0.4.0.

Christopher A. Sims and Tao Zha. Bayesian methods for dynamic multivariate models. *International Economic Review*, 39(4):949–968, 1998. doi: 10.2307/2527347.

Christopher A. Sims and Tao Zha. Were there regime switches in U.S. monetary policy? *American Economic Review*, 96(1):54–81, 2006. doi: 10.1257/000282806776157678.

James H. Stock and Mark W. Watson. Core inflation and trend inflation. *Review of Economics and Statistics*, 98(4):770–784, 2016. doi: 10.1162/REST_a_00608.

Jonathan R. Stroud, Peter Müller, and Nicholas G. Polson. Nonlinear state-space models with state-dependent variances. *Journal of the American Statistical Association*, 98(462):377–386, 2003. doi: 10.1198/016214503000161.

Sean Talts, Michael Betancourt, Daniel Simpson, Aki Vehtari, and Andrew Gelman. Validating Bayesian inference algorithms with simulation-based calibration, 2018. URL `https://arxiv.org/abs/1804.06788`. arXiv:1804.06788.

David A. van Dyk and Taeyoung Park. Partially collapsed Gibbs samplers: Theory and methods. *Journal of the American Statistical Association*, 103(482):790–796, 2008. doi: 10.1198/016214508000000409.

Aki Vehtari, Andrew Gelman, Daniel Simpson, Bob Carpenter, and Paul-Christian Bürkner. Rank-normalization, folding, and localization: An improved $\widehat{R}$ for assessing convergence of MCMC (with discussion). *Bayesian Analysis*, 16(2):667–718, 2021. doi: 10.1214/20-BA1221.

Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020. doi: 10.1038/s41592-019-0686-2.

Xiaolei Wang and Tomasz Woźniak. *bsvarSIGNs: Bayesian SVARs with Sign, Zero, and Narrative Restrictions*, 2026. URL `https://CRAN.R-project.org/package=bsvarSIGNs`. R package version 3.0.

James T. Wilson, Viacheslav Borovitskiy, Alexander Terenin, Peter Mostowsky, and Marc Peter Deisenroth. Efficiently sampling functions from Gaussian process posteriors. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 10292–10302, 2020. URL `https://proceedings.mlr.press/v119/wilson20a.html`.

Tomasz Woźniak. *bsvars: Bayesian Estimation of Structural Vector Autoregressive Models*, 2026. URL `https://CRAN.R-project.org/package=bsvars`. R package version 4.0.

Qiang Ye. Computing singular values of diagonally dominant matrices to high relative accuracy. *Mathematics of Computation*, 77(264):2195–2230, 2008. doi: 10.1090/S0025-5718-08-02112-1.

Yaming Yu and Xiao-Li Meng. To center or not to center: That is not the question – an ancillarity-sufficiency interweaving strategy (ASIS) for boosting MCMC efficiency. *Journal of Computational and Graphical Statistics*, 20(3):531–570, 2011. doi: 10.1198/jcgs.2011.203main.

Francesco Zanetta and Sam Allen. *scoringrules: A Python Library for Probabilistic Forecast Evaluation*, 2024. URL `https://github.com/frazane/scoringrules`. Python package version 0.11.0.